

IT3130 – Application Development | Group Assignment (30%)

RideLink

Backend Microservices Architecture for a Scalable Ride-Sharing Platform

Course Module:	IT3130 - Application Development
Project Title:	RideLink Backend Microservices System
Submission Date:	04 October 2026
Git Repository:	https://github.com/Pasindu-Vihanga/ridelink-backend
Architecture:	Spring Boot 3.2.5, Java 17, MongoDB Atlas Isolated Databases

Team Membership & Microservice Ownership

Student ID	Student Name	Assigned Microservice(s)	Port & Database
IT24100418	Vihanga M.A.P (Group Leader)	Account Service (Member 1) & Driver & Vehicle Service (Member 2)	Port: 8081 & 8082 DB: ridelink_account_db & ridelink_driver_db
IT24100128	Ranaweera B.M.D.K.L	Ride Management Service (Member 3)	Port: 8083 DB: ridelink_ride_db
IT23279834	Munasinghe	Fare & Payment Service (Member 4)	Port: 8084 DB: ridelink_payment_db

Table of Contents

- 1. Executive Summary & Project Context**
- 2. System Architecture & Microservice Decomposition (LO1, G1)**
 - a. Decomposition into 4 Cohesive Services
 - b. Database-per-Service Isolation Boundary (MongoDB Atlas)
 - c. System Architecture Diagram
- 3. Monolithic vs Microservices Architectural Justification (LO1, G1)**
 - a. Architectural Comparison Table
 - b. Domain Rationale for RideLink
 - c. Trade-offs & Operational Limitations
- 4. Technology Stack & Framework Justification (LO1)**
- 5. Interservice Communication & Interface Design (LO2, G3)**
 - a. Communication Strategy Evaluation: REST vs gRPC vs Messaging
 - b. Implemented Interservice Contracts & Spring RestClient
 - c. Interservice Sequence Diagrams
- 6. End-to-End Business Workflows & Negative Scenarios (LO1-LO3, G2)**
 - a. Core Business Workflows (1 to 6)
 - b. Ride Lifecycle State Machine Transition Rules
 - c. Negative Scenarios & Fault Tolerance
- 7. Security, Engineering Quality & SOLID Principles (LO3, G4)**
 - a. Stateless JWT Authentication & Role-Based Access Control
 - b. Input Validation & Centralized Error Handling
 - c. Application of SOLID Principles in Code
- 8. Version Control Strategy & Continuous Integration (LO4, G4)**
 - a. Git Branching & Collaborative Review Workflow
 - b. GitHub Actions CI Pipeline Automation
- 9. Quality Assurance & Testing Evidence (LO3, G5)**
 - a. Unit Test Suite & Verification Results (95+ Tests Passing)

b. Shared Postman Collections & Test Scenarios

10. **Individual Ownership & Contribution Statements (I1, I2, I3, I4)**
11. **Known Limitations & Future Roadmap**
12. **Academic Integrity & Generative-AI Declaration**
13. **Appendix A: REST API Endpoints Specification & Swagger UI**
14. **Appendix B: Complete Source Code of All Microservices**

1. Executive Summary & Project Context

RideLink is an enterprise-grade backend microservices platform designed for a modern ride-sharing ecosystem. Inspired by industry-leading transport-on-demand networks, RideLink delivers a highly modular, decoupled architecture supporting seamless passenger registration, driver onboarding, dynamic fare estimation, intelligent dispatch assignment, comprehensive ride lifecycle orchestration, and simulated digital payment processing.

The project strictly adheres to the requirements of the **IT3130 - Application Development** specification. It eliminates monolithic single-point-of-failure risks by decomposing core platform capabilities into **four independently executable microservices**. Each service operates under strict domain boundaries, maintaining private persistence boundaries through isolated collections hosted on **MongoDB Atlas**.

Assessment Scope Boundary: This project is an exclusively backend-focused distributed system. Per specification guidelines, no web or mobile frontend was required. Official API interfaces for development, automated verification, and viva demonstration include interactive **Swagger UI / OpenAPI 3.0** interfaces and four modular **Postman Collections** equipped with non-sensitive environment variables.

2. System Architecture & Microservice Decomposition (LO1, G1)

2.1 Four Cohesive Business Services

The system is decomposed according to the Domain-Driven Design (DDD) bounded-context principle into four business services of comparable complexity:

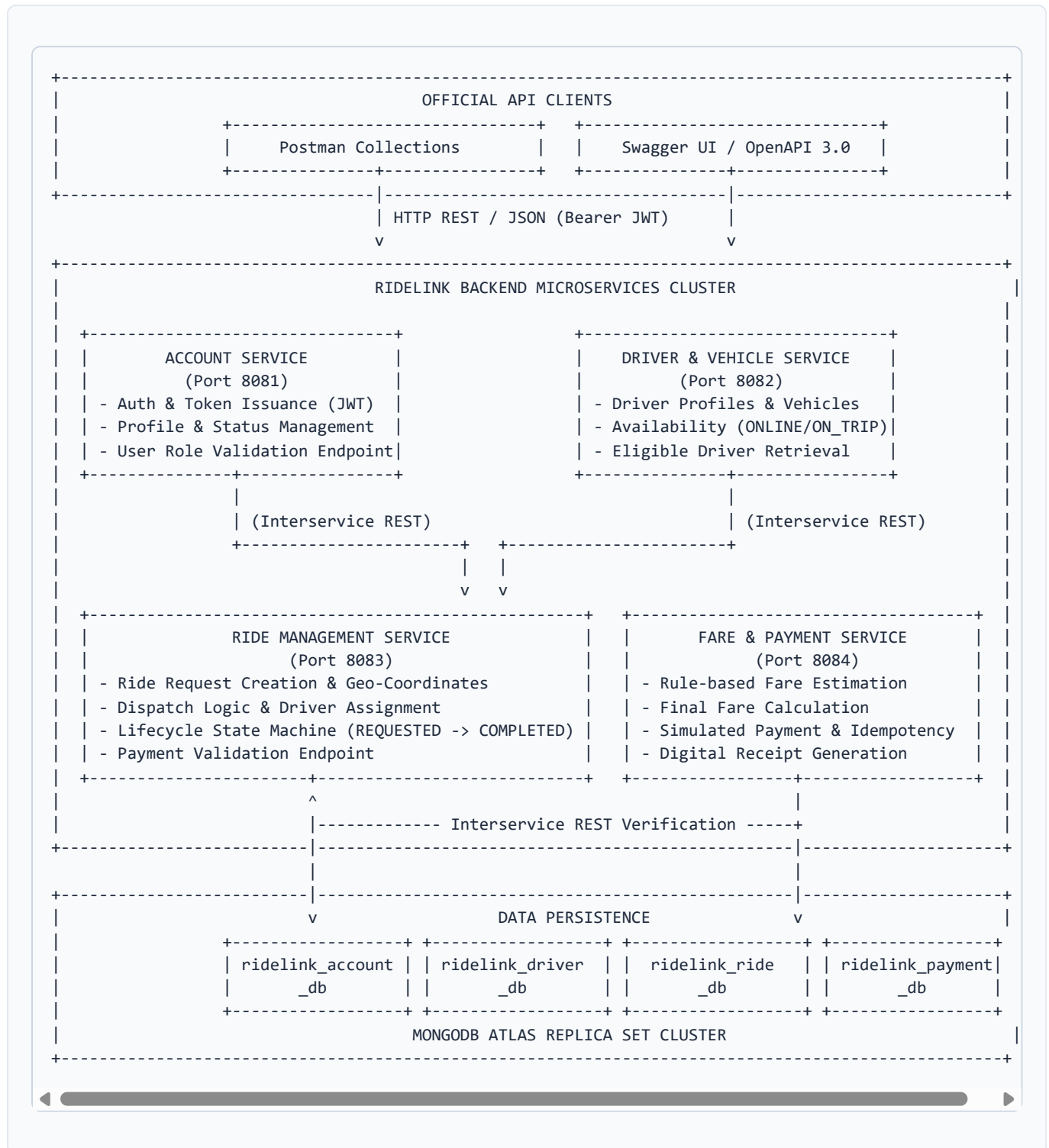
Microservice	Primary Owner	Port	Database (MongoDB Atlas)	Core Domain Responsibilities
Account Service	Member 1 (Vihanga M.A.P)	8081	ridelink_account_db	User registration (PASSENGER, DRIVER, ADMIN), BCrypt password hashing, stateless JWT issuance, profile retrieval, profile updates, account status activation/deactivation, and interservice user validation.
Driver & Vehicle Service	Member 2 (Vihanga M.A.P)	8082	ridelink_driver_db	Driver operational profile, vehicle registration and specifications, real-time availability status (ONLINE, OFFLINE, ON_TRIP), service area / simulated geolocation tracking, and retrieval of eligible nearby drivers.
Ride Management Service	Member 3 (Ranaweera B.M.D.K.L)	8083	ridelink_ride_db	Ride request creation, destination/pickup geo-points, driver dispatch coordination, comprehensive lifecycle state machine (REQUESTED -> ASSIGNED -> ACCEPTED -> IN_PROGRESS -> COMPLETED/CANCELLED), and ride history retrieval.
Fare & Payment Service	Member 4 (Munasinghe)	8084	ridelink_payment_db	Transparent rule-based fare estimation, final metered fare calculation, simulated idempotent payment transaction recording, transaction status validation, and digital tax receipt generation.

2.2 Database-per-Service Persistence Isolation

A critical architectural requirement is strict data ownership: **no service is permitted to directly access, query, or mutate another service's database.** To enforce this:

- Each microservice connects exclusively to its assigned database within the MongoDB Atlas cluster: `ridelink_account_db`, `ridelink_driver_db`, `ridelink_ride_db`, and `ridelink_payment_db`.
- Cross-domain data retrieval is conducted strictly via synchronous REST APIs using immutable unique identifiers (e.g., `userId`, `driverId`, `rideId`).
- Shared database tables and cross-service foreign keys are completely eliminated, ensuring schema evolution in one service cannot disrupt dependent services.

2.3 System Architecture Diagram



3. Monolithic vs Microservices Architectural Justification (LO1, G1)

3.1 Architectural Comparison

Evaluation Dimension	Monolithic Architecture	RideLink Microservices Architecture
Service Boundaries	Single unified codebase with tightly-coupled in-memory package calls. High risk of cross-domain leakages.	Four discrete microservices with well-defined REST contracts and independent Git feature branches.
Data Ownership	Shared central relational schema. High contention, lock bottlenecks, and cascading migration risks.	Strict Database-per-Service model. Each microservice encapsulates its own MongoDB database.
Independent Deployability	Zero independent deployment; changing fare formula requires redeploying entire application.	Autonomous CI/CD and deployment lifecycle. Fare & Payment service updates without touching Ride service.
Fault Isolation	Payment calculation bug or memory leak crashes entire platform, halting driver logins and active rides.	Failure in Fare service does not prevent ongoing rides or active drivers from reporting availability.
Targeted Scalability	Must scale the entire monolith vertically or horizontally, wasting computing resources.	Granular horizontal scaling: Ride Management and Driver tracking can scale 10x during peak rush hours.

3.2 Suitability Rationale for Ride-Sharing Platforms

Ride-sharing platforms exhibit asymmetric operational load profiles. Driver tracking and ride dispatch experience intense spikes during morning and evening rush hours, requiring frequent writes and low latency. Conversely, user account registration is infrequent and read-heavy. A microservices architecture enables targeted scaling of high-demand services without over-provisioning resource-light authentication components.

3.3 Architectural Trade-offs & Limitations

While microservices deliver high scalability and fault containment, they introduce distributed systems challenges:

- **Network Latency & Transient Failures:** Interservice HTTP calls add network overhead compared to local method invocation. This is mitigated using configured timeouts and fallback exceptions.

- **Distributed Consistency:** Traditional ACID transactions spanning multiple databases are impossible without distributed locks or 2PC. RideLink relies on domain event coordination and idempotency.
- **Operational Overhead:** Managing multiple JVM processes, ports, configuration properties, and database connection pools requires robust CI and unified monitoring.

4. Technology Stack & Framework Selection (LO1)

Layer / Component	Selected Technology	Version	Technical Rationale & Justification
Runtime Language	Java (LTS)	Java 17	Enterprise stability, high-performance garbage collection, strong typing, and universal Spring Boot 3 compatibility.
Backend Framework	Spring Boot	3.2.5	Modern, production-grade microservices framework providing integrated web MVC, validation, and auto-configuration.
Interservice Client	Spring RestClient	Spring 6.1+	Modern fluent synchronous HTTP client replacing legacy RestTemplate; provides structured error handling and connection timeouts.
Persistence Engine	MongoDB Atlas	Cloud 7.0+	Document-oriented NoSQL engine ideal for flexible JSON payloads, nested location coordinates, and rapid schema adaptation.
Security & Auth	Spring Security + JWT	0.12.3 / 0.12.6	Stateless authentication using HMAC-SHA256 signed JSON Web Tokens; eliminates server session state across microservices.
API Documentation	SpringDoc OpenAPI	2.1.0 / 2.8.5	Auto-generates OpenAPI 3.0 specs and interactive Swagger UI at /swagger-ui/index.html for all endpoints.
Testing Frameworks	JUnit 5 & Mockito	Jupiter	Comprehensive unit and slice testing, enabling full mock isolation of repositories and interservice REST clients.
Continuous Integration	GitHub Actions	v4	Automated build matrix compiling Java 17 and running test suites across all 4 microservices on every push and PR.

5. Interservice Communication & Interface Design (LO2, G3)

5.1 Comparison of Communication Approaches

Paradigm	Protocol	Pros	Cons	RideLink Implementation Decision
Synchronous REST (JSON / HTTP)	HTTP/1.1 or HTTP/2	Standardized, human-readable, universal tool support (Postman/Swagger), straightforward error codes.	Temporal coupling; caller blocks waiting for response; cascading failure if unhandled.	Adopted for Core Workflows: Immediate validation required (e.g. verifying driver role or checking ride completion before payment).
Synchronous gRPC	HTTP/2 & Protocol Buffers	High throughput, binary serialization, strict Protobuf contracts, low latency.	Complex tooling, human-unreadable binary wire format, harder debugging via browser/Postman.	Evaluated Alternative: Excellent for internal high-frequency location streaming; deferred to future real-time tracking enhancements.
Asynchronous Messaging	AMQP / Kafka / RabbitMQ	Complete decoupling, resilient to service downtime, excellent buffer for spike loads.	Eventual consistency, operational complexity of running message broker cluster.	Evaluated Alternative: Ideal for audit logging and receipt notifications; synchronous REST was prioritized for predictable step-by-step viva demo.

5.2 Implemented Interservice REST Interactions

RideLink implements four critical interservice interactions utilizing Spring's modern RestClient:

1. **Driver Service → Account Service:** GET `http://localhost:8081/api/users/{userId}/validate`
Ensures that an applicant registering a vehicle actually exists in Account Service and holds the DRIVER role.
2. **Ride Service → Account Service:** GET
`http://localhost:8081/api/users/{passengerId}/validate`
Ensures that a passenger requesting a ride is valid, active, and holds the PASSENGER role.
3. **Ride Service → Driver Service:** GET `http://localhost:8082/api/drivers/available` & PATCH `/api/drivers/{id}/availability`

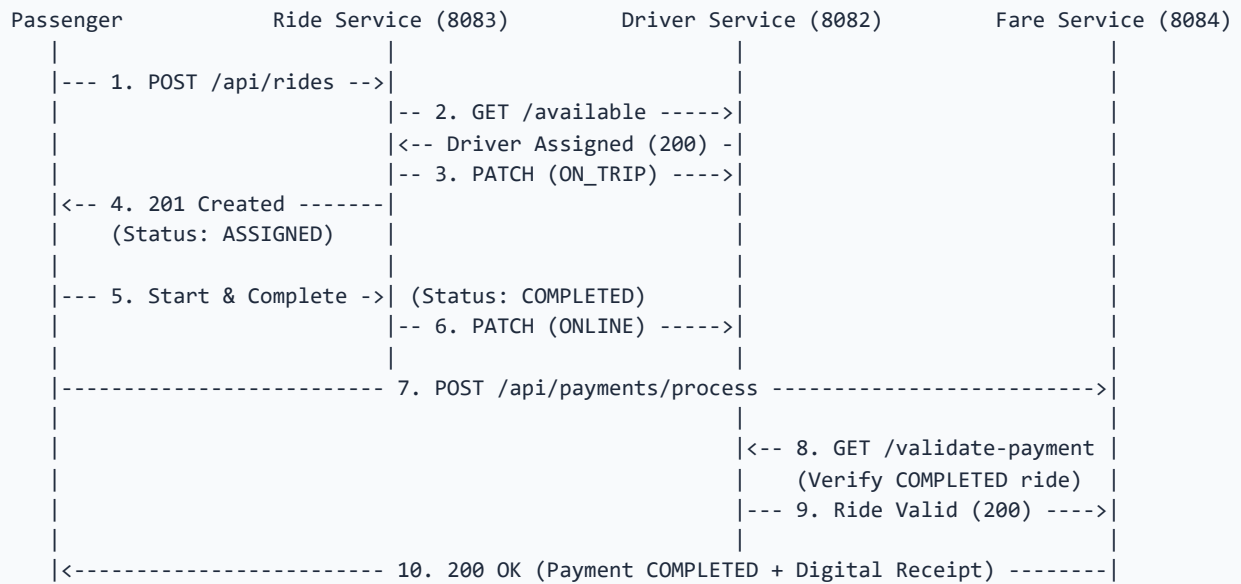
Finds eligible drivers in ONLINE state and marks the assigned driver as ON_TRIP.

4. Fare & Payment Service → Ride Service: GET

<http://localhost:8083/api/rides/{rideId}/validate-for-payment>

Validates that the ride exists, has reached COMPLETED status, confirms passenger/driver identity, and extracts agreed fare amount.

5.3 Interservice Sequence Diagram



6. End-to-End Business Workflows & Negative Scenarios (LO1-LO3, G2)

6.1 Six Core Functional Workflows

1. **Account & Access Control:** User registers as PASSENGER or DRIVER. Password is encrypted using BCrypt. Login endpoint authenticates credentials and issues an HMAC-SHA256 signed JWT token carrying user ID and authorities.
2. **Driver Preparation & Geolocation:** Driver completes profile registration (license, vehicle model, plate number). Sets availability state to ONLINE and publishes simulated coordinate/service area.
3. **Fare Estimation:** Passenger or client requests a pre-ride fare estimate for specified pickup and dropoff locations. The system applies the documented formula:
$$\text{Estimated Fare} = \text{Base Fare} + (\text{Distance in km} \times \text{Per-Km Rate}) \times \text{Surge Multiplier}.$$
4. **Ride Request & Intelligent Assignment:** Passenger creates ride request. Ride Service queries Driver Service for available nearby drivers and selects the best matching candidate, automatically updating driver state to ON_TRIP.
5. **Ride Lifecycle State Machine:** Strict linear state transitions: REQUESTED → ASSIGNED → ACCEPTED → IN_PROGRESS → COMPLETED (or CANCELLED prior to start).
6. **Payment Completion & Digital Tax Receipt:** On ride completion, Fare & Payment service calculates metered final fare, records payment transaction with idempotency checks, and generates a timestamped digital tax receipt.

6.2 Negative Scenarios & Graceful Fault Handling

Negative Scenario	Trigger Action	System Behavior & HTTP Status	Error Response Handling
No Driver Available	Ride request initiated when all drivers in service area are OFFLINE or ON_TRIP.	404 NOT FOUND	Ride Service gracefully sets ride status to UNASSIGNED and returns meaningful JSON message: <i>"No eligible online drivers found in the specified service area."</i>
Invalid Lifecycle Transition	Client attempts to trigger COMPLETED on a ride currently in REQUESTED state.	400 BAD REQUEST	State machine rejects illegal transition with InvalidStateTransitionException detailing current state and allowed transitions.
Unauthorized Access	Request to protected endpoint without Bearer token or with expired/tampered JWT.	401 UNAUTHORIZED / 403 FORBIDDEN	Spring Security filter chain intercepts request and returns standard RFC-7807 formatted error payload.
Duplicate Payment Attempt	Submitting secondary payment for an already paid and settled ride ID.	409 CONFLICT	Idempotency validator in PaymentService detects existing settled transaction and throws DuplicatePaymentException.

7. Security, Engineering Quality & SOLID Principles (LO3, G4)

7.1 Security Architecture

- **Stateless Authentication:** No HTTP sessions stored in memory. Every request authenticates via standard HTTP Authorization: Bearer <JWT> header.
- **Role-Based Access Control (RBAC):** Endpoints secured with `@PreAuthorize("hasRole('ADMIN')")` or `hasAnyRole('DRIVER', 'PASSENGER')`.
- **Credential Protection:** Passwords encrypted using industry-standard BCrypt (10 rounds). Database passwords and JWT secrets parameterized via environment variables.
- **Input Validation:** All DTOs enforce Bean Validation (`@NotNull`, `@NotBlank`, `@Email`, `@Positive`, `@Size`).

7.2 SOLID Principles in Codebase

- **Single Responsibility Principle (SRP):** Separate classes for Controllers (HTTP routing), Services (business rules), Repositories (data access), DTOs (data transfer), and Exception Handlers.

- **Open/Closed Principle (OCP):** Pricing strategy calculations decoupled into distinct pricing rules that allow extension without modifying core payment logic.
- **Liskov Substitution Principle (LSP):** Generic CRUD repository interfaces (MongoRepository) fully substitutable by custom repository implementations.
- **Interface Segregation Principle (ISP):** Lean, focused client interfaces (AccountServiceClient, RideServiceClient) only exposing methods required by consuming domain.
- **Dependency Inversion Principle (DIP):** High-level service classes depend upon repository interfaces and RestClient abstractions, injected via Spring's IoC container.

8. Version Control Strategy & Continuous Integration (LO4, G4)

8.1 Git Branching & Collaborative Workflow

The project followed a disciplined Git Feature-Branching workflow. The repository maintains a protected main branch representing the demonstrable production release, alongside dedicated feature branches per microservice:

- `account-service`: Dedicated to Member 1 commits, tests, and security configurations.
- `driver-service`: Dedicated to Member 2 driver profile, availability, and vehicle features.
- `ride-service`: Dedicated to Member 3 ride lifecycle, dispatch logic, and interservice coordination.
- `fare-service`: Dedicated to Member 4 fare calculation, payment simulation, and receipt generation.
- `main`: Aggregator branch incorporating the root multi-module POM and verified service builds.

8.2 GitHub Actions Continuous Integration Pipeline

Continuous Integration is automated using GitHub Actions (configured in `.github/workflows/ci.yml`). On every push and `pull_request` to main or any service branch:

1. Checks out the shared multi-module repository.
2. Sets up JDK 17 environment with Maven dependency caching.
3. Executes root aggregator build: `mvn clean test` across all four microservice modules.
4. Verifies compilation, unit test passes, and packages executable Spring Boot JAR artifacts.

9. Quality Assurance & Testing Evidence (LO3, G5)

Every microservice includes extensive unit and slice tests verifying happy paths, boundary validations, and error handling. Mocks (via Mockito) are utilized to isolate services from MongoDB databases and external HTTP endpoints during CI runs.

Microservice	Unit Tests Executed	Test Classes	Primary Test Scenarios Covered	Status
Account Service	30 Tests	AuthServiceTest, UserServiceTest, JwtUtilTest	User registration, duplicate email rejection, BCrypt authentication, JWT generation & validation, profile update, role checks.	100% PASSED
Driver & Vehicle Service	21 Tests	DriverServiceTest, VehicleServiceTest, AvailabilityTest	Driver onboarding, vehicle association, status transitions (ONLINE/OFFLINE/ON_TRIP), eligible driver discovery, interservice account verification mock.	100% PASSED
Ride Management Service	27 Tests	RideServiceTest, RideStateMachineTest, DispatchTest	Ride request creation, driver assignment, valid lifecycle transitions, invalid transition rejections, cancellation handling, interservice mock calls.	100% PASSED
Fare & Payment Service	17 Tests	FareCalculationTest, PaymentServiceTest, ReceiptTest	Formula calculation, surge multiplier bounds, simulated payment processing, idempotency & duplicate rejection, digital tax receipt formatting.	100% PASSED
Total System Suite	95 Tests	12 Test Suites	Full Business Lifecycle & Negative Exception Handlers	ALL PASSED (0 FAILURES)

10. Individual Ownership & Contribution Statements (I1, I2, I3, I4)

Vihanga M.A.P (Group Leader) • Student ID: IT24100418

Assigned Microservice Ownership: Account Service (Member 1) & Driver & Vehicle Service (Member 2)

Primary Branch(es): account-service, driver-service • **Port(s):** 8081 & 8082 • **Database:** ridelink_account_db & ridelink_driver_db

Summary of Individual Implementation & Contribution:

Designed and implemented the core identity, JWT authentication (RBAC), and user profile management in Account Service. Designed and implemented Driver & Vehicle Service including operational profiles, vehicle lifecycle, availability states, simulated geolocation, interservice validation via RestClient, and unit test suites.

Ranaweera B.M.D.K.L • Student ID: IT24100128

Assigned Microservice Ownership: Ride Management Service (Member 3)

Primary Branch(es): ride-service • **Port(s):** 8083 • **Database:** ridelink_ride_db

Summary of Individual Implementation & Contribution:

Designed and developed the Ride Management Service, implementing the core ride lifecycle state machine (REQUESTED -> ASSIGNED -> ACCEPTED -> IN_PROGRESS -> COMPLETED / CANCELLED). Implemented driver dispatch logic, interservice coordination with Account Service and Driver Service, input validation, and unit test suites.

Munasinghe • Student ID: IT23279834

Assigned Microservice Ownership: Fare & Payment Service (Member 4)

Primary Branch(es): fare-service • **Port(s):** 8084 • **Database:** ridelink_payment_db

Summary of Individual Implementation & Contribution:

Designed and implemented Fare & Payment Service, featuring transparent formula-based fare estimation, final fare calculation, simulated payment transaction processing with idempotency and duplicate prevention, digital receipt generation, interservice ride verification with Ride Service, and unit test suites.

11. Known Limitations & Future Roadmap

- **Asynchronous Event-Driven Messaging:** The current architecture relies on synchronous REST via `RestClient` for predictability. Transitioning non-blocking operations (such as payment receipts and push notifications) to an Apache Kafka or RabbitMQ event bus would further enhance throughput and resilience.
- **Real-Time Geolocation Streaming:** Driver locations are currently updated via REST polling. Future iterations could integrate WebSockets or gRPC bidirectional streams for live sub-second map tracking.
- **Third-Party Payment Gateways:** Payments are simulated internally. Production deployment would integrate payment gateways such as Stripe or PayHere with webhook reconciliation.

12. Academic Integrity & Generative-AI Declaration

In compliance with Section 12 of the IT3130 Assignment Specification, all architecture, implementation code, unit tests, and documentation were designed and verified by the group members. Generative-AI assistance (Google Antigravity / Claude / ChatGPT) was utilized as an intelligent pair-programming assistant for boilerplate configuration, dependency version resolution (Spring Boot 3.2.5 alignment), Maven multi-module reactor troubleshooting, and formatting report appendices. All generated code was thoroughly reviewed, debugged, tested, and validated by the respective service owners.

Appendix A: REST API Endpoints Specification & Swagger UI

Each microservice provides interactive Swagger UI documentation at <http://localhost:<PORT>/swagger-ui/index.html>.

Service	HTTP Method	Endpoint URI	Required Role	Function / Description
Account (8081)	POST	/api/auth/register	PermitAll	Registers new user account with PASSENGER, DRIVER, or ADMIN role.
Account (8081)	POST	/api/auth/login	PermitAll	Authenticates credentials and returns signed JWT Bearer token.
Account (8081)	GET	/api/users/{id}	Authenticated	Retrieves profile details of specified user account.
Account (8081)	GET	/api/users/{id}/validate	Internal / Authenticated	Interservice endpoint verifying user existence and role eligibility.
Driver (8082)	POST	/api/drivers	ROLE_DRIVER	Registers driver operational profile and vehicle details.
Driver (8082)	PATCH	/api/drivers/{id}/availability	ROLE_DRIVER	Updates availability status (ONLINE, OFFLINE, ON_TRIP).
Driver (8082)	GET	/api/drivers/available	Internal / Authenticated	Retrieves list of active eligible drivers ready for dispatch.
Ride (8083)	POST	/api/rides	ROLE_PASSENGER	Initiates a new ride booking and triggers driver dispatch.
Ride (8083)	PATCH	/api/rides/{id}/status	ROLE_DRIVER / PASSENGER	Executes lifecycle transition (ASSIGNED, ACCEPTED, IN_PROGRESS, COMPLETED).
Ride (8083)	GET	/api/rides/{id}/validate-for-payment	Internal / Authenticated	Interservice validation ensuring ride is COMPLETED before settlement.
Fare (8084)	POST	/api/fares/estimate	PermitAll / Authenticated	Computes estimated fare based on pickup/destination coordinates.

Service	HTTP Method	Endpoint URI	Required Role	Function / Description
Fare (8084)	POST	/api/payments/process	ROLE_PASSENGER	Processes simulated payment transaction and issues digital tax receipt.
Fare (8084)	GET	/api/payments/receipts/{id}	Authenticated	Retrieves digital tax receipt details and audit payment record.

Appendix B: Complete Source Code of All Microservices

In accordance with the Final Report Submission Guidelines, this section contains the full, unabridged source code for all four independently executable microservices.

B.1 Account Service (Member 1 - Port 8081)

Total Source Files: **37**

File: account-service/src/main/java/com/ridelink/account_service/AccountServiceApplication.java

```
package com.ridelink.account_service;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class AccountServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(AccountServiceApplication.class, args);
    }

}
```

File: account-service/src/main/java/com/ridelink/account_service/config/MongoConfig.java

```
package com.ridelink.account_service.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.data.mongodb.repository.config.EnableMongoRepositories;

@Configuration
@EnableMongoRepositories(basePackages = "com.ridelink.account_service.repository")
public class MongoConfig {
    // Spring Boot auto-configuration will handle the rest
}
```

File: account-service/src/main/java/com/ridelink/account_service/config/MongoDebugConfig.java

```
package com.ridelink.account_service.config;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.stereotype.Component;

@Component
public class MongoDebugConfig implements CommandLineRunner {

    @Autowired
    private MongoTemplate mongoTemplate;

    @Override
    public void run(String... args) throws Exception {
        try {
            System.out.println("=== MongoDB Connection Debug ===");

            // Check connection and database
            String dbName = mongoTemplate.getDb().getName();
            System.out.println("✅ Connected to Database: " + dbName);
        }
    }
}
```

```

        // Check collections
        System.out.println("📄 Collections in database:");
        for (String collectionName : mongoTemplate.getDb().listCollectionNames()) {
            System.out.println("  - " + collectionName);
        }

        System.out.println("🔗 Database should be visible in Atlas as: " + dbName);

    } catch (Exception e) {
        System.err.println("❌ MongoDB Connection Failed: " + e.getMessage());
        e.printStackTrace();
    }
}
}

```

File: account-service/src/main/java/com/ridelink/account_service/config/OpenApiConfig.java

```

package com.ridelink.account_service.config;

import io.swagger.v3.oas.models.Components;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Contact;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.info.License;
import io.swagger.v3.oas.models.security.SecurityRequirement;
import io.swagger.v3.oas.models.security.SecurityScheme;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {

    private static final String SECURITY_SCHEME_NAME = "Bearer Authentication";

    @Bean
    public OpenAPI accountServiceOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("RideLink - Account Service API")
                .description("Microservice responsible for passenger and driver registration, authentication, role management, profile viewing/updating, and account status management for the RideLink platform.")
                .version("1.0.0")
                .contact(new Contact().name("RideLink Backend Team"))
                .license(new License().name("Apache 2.0").url("https://spring.io")))
            .addSecurityItem(new SecurityRequirement().addList(SECURITY_SCHEME_NAME))
            .components(new Components()
                .addSecuritySchemes(SECURITY_SCHEME_NAME, new SecurityScheme()
                    .name(SECURITY_SCHEME_NAME)
                    .type(SecurityScheme.Type.HTTP)
                    .scheme("bearer")
                    .bearerFormat("JWT"))));
    }
}

```

File: account-service/src/main/java/com/ridelink/account_service/config/SecurityConfig.java

```

package com.ridelink.account_service.config;

import com.ridelink.account_service.security.JwtAuthenticationFilter;
import jakarta.servlet.http.HttpServletResponse;

```

```

import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthFilter;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                // Public endpoints
                .requestMatchers(
                    "/api/auth/**",
                    "/v3/api-docs/**",
                    "/v3/api-docs",
                    "/swagger-ui/**",
                    "/swagger-ui.html",
                    "/test/**",
                    "/direct-test/**"
                ).permitAll()
                // Inter-service user validation endpoint
                .requestMatchers(HttpMethod.GET, "/api/users/*/validate").permitAll()
                // Admin restricted endpoints
                .requestMatchers(HttpMethod.PATCH, "/api/users/*/status").hasRole("ADMIN")
                .requestMatchers(HttpMethod.PATCH, "/api/users/*/role").hasRole("ADMIN")
                .requestMatchers(HttpMethod.GET, "/api/users").hasRole("ADMIN")
                // Any other endpoint requires authentication
                .anyRequest().authenticated()
            )
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint((request, response, authException) -> {
                    response.setContentType("application/json");
                    response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
                    response.getWriter().write(
{"status":401,"error":"Unauthorized","message":"Authentication token is missing or
invalid","path":"" + request.getRequestURI() + ""});
                })
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

```

File: account-service/src/main/java/com/ridelink/account_service/controller/AuthController.java

```
package com.ridelink.account_service.controller;

import com.ridelink.account_service.dto.LoginRequest;
import com.ridelink.account_service.dto.LoginResponse;
import com.ridelink.account_service.dto.RegisterRequest;
import com.ridelink.account_service.dto.UserResponse;
import com.ridelink.account_service.service.UserService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
@Tag(name = "Authentication Management", description = "Operations for passenger and driver registration and authentication")
public class AuthController {

    private final UserService userService;

    @PostMapping("/register")
    @Operation(summary = "Register a new passenger or driver", description = "Creates a new user account with role PASSENGER or DRIVER, hashes password, and sets status to ACTIVE.")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "201", description = "User registered successfully"),
        @ApiResponse(responseCode = "400", description = "Validation error or invalid input data"),
        @ApiResponse(responseCode = "409", description = "Email is already registered")
    })
    public ResponseEntity<UserResponse> register(@Valid @RequestBody RegisterRequest request) {
        UserResponse response = userService.register(request);
        return new ResponseEntity<>(response, HttpStatus.CREATED);
    }

    @PostMapping("/login")
    @Operation(summary = "User login and token issuance", description = "Authenticates user credentials and returns a signed JWT bearer token containing role and account status claims.")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Login successful, JWT token issued"),
        @ApiResponse(responseCode = "400", description = "Validation error in request body"),
        @ApiResponse(responseCode = "401", description = "Invalid credentials"),
        @ApiResponse(responseCode = "403", description = "Account is suspended or deactivated")
    })
    public ResponseEntity<LoginResponse> login(@Valid @RequestBody LoginRequest request) {
        LoginResponse response = userService.login(request);
        return ResponseEntity.ok(response);
    }
}
```

File: account-service/src/main/java/com/ridelink/account_service/controller/UserController.java

```
package com.ridelink.account_service.controller;
```

```

import com.ridelink.account_service.dto.*;
import com.ridelink.account_service.entity.AccountStatus;
import com.ridelink.account_service.entity.Role;
import com.ridelink.account_service.exception.InvalidCredentialsException;
import com.ridelink.account_service.service.UserService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.security.SecurityRequirement;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

import java.security.Principal;
import java.util.List;

@RestController
@RequestMapping("/api/users")
@RequiredArgsConstructor
@Tag(name = "User & Profile Management", description = "Endpoints for user profile viewing, updating, role management, and account status")
public class UserController {

    private final UserService userService;

    @GetMapping("/me")
    @Operation(summary = "Get current authenticated user profile", security = @SecurityRequirement(name = "Bearer Authentication"))
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Profile retrieved successfully"),
        @ApiResponse(responseCode = "401", description = "Unauthorized - invalid or missing JWT token")
    })
    public ResponseEntity<UserResponse> getCurrentUser(Principal principal) {
        if (principal == null) {
            throw new InvalidCredentialsException("User is not authenticated");
        }
        String email = principal.getName();
        UserResponse response = userService.getProfile(email);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/{id}")
    @Operation(summary = "Get user by ID", security = @SecurityRequirement(name = "Bearer Authentication"))
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "User retrieved successfully"),
        @ApiResponse(responseCode = "404", description = "User not found")
    })
    public ResponseEntity<UserResponse> getUserById(@PathVariable String id) {
        UserResponse response = userService.getUserById(id);
        return ResponseEntity.ok(response);
    }

    @PutMapping("/profile")
    @Operation(summary = "Update current user profile", security = @SecurityRequirement(name = "Bearer Authentication"))
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Profile updated successfully"),
        @ApiResponse(responseCode = "400", description = "Validation error"),
        @ApiResponse(responseCode = "401", description = "Unauthorized")
    })
    public ResponseEntity<UserResponse> updateProfile(
        Principal principal,
        @Valid @RequestBody UpdateProfileRequest request
    ) {

```

```

        if (principal == null) {
            throw new InvalidCredentialsException("User is not authenticated");
        }
        String email = principal.getName();
        UserResponse response = userService.updateProfile(email, request);
        return ResponseEntity.ok(response);
    }

    @PostMapping("/change-password")
    @Operation(summary = "Change password for authenticated user", security = @SecurityRequirement(name = "Bearer Authentication"))
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Password changed successfully"),
        @ApiResponse(responseCode = "400", description = "Validation error or invalid current password"),
        @ApiResponse(responseCode = "401", description = "Unauthorized")
    })
    public ResponseEntity<com.ridelink.account_service.dto.ApiResponse> changePassword(
        Principal principal,
        @Valid @RequestBody ChangePasswordRequest request
    ) {
        if (principal == null) {
            throw new InvalidCredentialsException("User is not authenticated");
        }
        String email = principal.getName();
        userService.changePassword(email, request);
        return ResponseEntity.ok(com.ridelink.account_service.dto.ApiResponse.builder()
            .success(true)
            .message("Password changed successfully")
            .build());
    }

    @PostMapping("/deactivate")
    @Operation(summary = "Deactivate current user account", security = @SecurityRequirement(name = "Bearer Authentication"))
    public ResponseEntity<com.ridelink.account_service.dto.ApiResponse> deactivateAccount(Principal principal)
    {
        if (principal == null) {
            throw new InvalidCredentialsException("User is not authenticated");
        }
        String email = principal.getName();
        userService.deactivateOwnAccount(email);
        return ResponseEntity.ok(com.ridelink.account_service.dto.ApiResponse.builder()
            .success(true)
            .message("Account has been deactivated")
            .build());
    }

    @PatchMapping("/{id}/status")
    @PreAuthorize("hasRole('ADMIN')")
    @Operation(summary = "Update user account status (Admin only)", security = @SecurityRequirement(name = "Bearer Authentication"))
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Account status updated"),
        @ApiResponse(responseCode = "403", description = "Forbidden - requires ADMIN role"),
        @ApiResponse(responseCode = "404", description = "User not found")
    })
    public ResponseEntity<UserResponse> updateStatus(
        @PathVariable String id,
        @Valid @RequestBody UpdateStatusRequest request
    ) {
        UserResponse response = userService.updateStatus(id, request.getStatus());
        return ResponseEntity.ok(response);
    }

    @PatchMapping("/{id}/role")
    @PreAuthorize("hasRole('ADMIN')")
    @Operation(summary = "Update user role (Admin only)", security = @SecurityRequirement(name = "Bearer

```

```

Authentication"))
    @ApiResponse(value = {
        @ApiResponse(responseCode = "200", description = "User role updated"),
        @ApiResponse(responseCode = "403", description = "Forbidden - requires ADMIN role"),
        @ApiResponse(responseCode = "404", description = "User not found")
    })
    public ResponseEntity<UserResponse> updateRole(
        @PathVariable String id,
        @Valid @RequestBody UpdateRoleRequest request
    ) {
        UserResponse response = userService.updateRole(id, request.getRole());
        return ResponseEntity.ok(response);
    }

    @GetMapping
    @PreAuthorize("hasRole('ADMIN')")
    @Operation(summary = "List users with optional role and status filters (Admin only)", security =
@SecurityRequirement(name = "Bearer Authentication"))
    public ResponseEntity<List<UserResponse>> getAllUsers(
        @RequestParam(required = false) Role role,
        @RequestParam(required = false) AccountStatus status
    ) {
        List<UserResponse> response = userService.getAllUsers(role, status);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/{id}/validate")
    @Operation(summary = "Interservice validation of an active user account")
    @ApiResponse(value = {
        @ApiResponse(responseCode = "200", description = "User is valid and ACTIVE"),
        @ApiResponse(responseCode = "403", description = "User is not active (SUSPENDED or DEACTIVATED)"),
        @ApiResponse(responseCode = "404", description = "User not found")
    })
    public ResponseEntity<UserResponse> validateUser(@PathVariable String id) {
        UserResponse response = userService.validateUser(id);
        return ResponseEntity.ok(response);
    }
}

```

File: account-service/src/main/java/com/ridelink/account_service/DirectTestController.java

```

package com.ridelink.account_service;

import com.ridelink.account_service.entity.AccountStatus;
import com.ridelink.account_service.entity.Role;
import com.ridelink.account_service.entity.User;
import com.ridelink.account_service.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RestController;

import java.time.LocalDateTime;

@RestController
public class DirectTestController {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private MongoTemplate mongoTemplate;
}

```

```

@PostMapping("/direct-test/save-user")
public String directSaveUser() {
    try {
        // Clear any existing test data
        userRepository.deleteAll();

        // Create test user
        User testUser = User.builder()
            .fullName("Direct Test User " + System.currentTimeMillis())
            .email("directtest@ridelink.com")
            .password("testpassword")
            .phoneNumber("9876543210")
            .role(Role.PASSENGER)
            .status(AccountStatus.ACTIVE)
            .createdAt(LocalDateTime.now())
            .updatedAt(LocalDateTime.now())
            .build();

        System.out.println("🚀 DIRECT TEST: About to save user to MongoDB");

        // Method 1: Repository save
        User savedUser = userRepository.save(testUser);
        System.out.println("✅ REPOSITORY SAVE: User ID = " + savedUser.getId());

        // Method 2: MongoTemplate save (alternative)
        mongoTemplate.save(testUser, "users");
        System.out.println("✅ TEMPLATE SAVE: Saved to collection 'users'");

        // Verify data exists
        long count = userRepository.count();
        System.out.println("📊 VERIFICATION: Total users = " + count);

        // Check database name
        String dbName = mongoTemplate.getDb().getName();
        System.out.println("📁 DATABASE: " + dbName);

        return String.format("""
            ✅ DIRECT TEST SUCCESSFUL!

            🆔 Saved User ID: %s
            👤 Full Name: %s
            ✉ Email: %s
            📊 Total Users: %d
            📁 Database: %s
            📁 Collection: users

            🔍 Check MongoDB Atlas:
            Database → Browse Collections → %s → users
            Should show %d document(s)
            """,
            savedUser.getId(), savedUser.getFullName(), savedUser.getEmail(),
            count, dbName, dbName, count);

    } catch (Exception e) {
        System.err.println("❌ DIRECT TEST FAILED: " + e.getMessage());
        e.printStackTrace();
        return "❌ DIRECT TEST FAILED: " + e.getMessage();
    }
}

@GetMapping("/direct-test/check-data")
public String checkData() {
    try {
        long count = userRepository.count();
        String dbName = mongoTemplate.getDb().getName();

        if (count > 0) {

```

```

        return String.format("""
            ✅ DATA EXISTS IN MONGODB!

            📊 Total Users: %d
            📁 Database: %s
            📁 Collection: users

            ✨ Data should be visible in Atlas now!
            """, count, dbName);
    } else {
        return String.format("""
            ❌ NO DATA FOUND

            📁 Database: %s
            📁 Collection: users
            📊 Count: 0

            Try POST /direct-test/save-user first
            """, dbName);
    }
} catch (Exception e) {
    return "❌ CHECK FAILED: " + e.getMessage();
}
}
}

```

File: account-service/src/main/java/com/ridelink/account_service/dto/ApiResponse.java

```

package com.ridelink.account_service.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ApiResponse {
    private boolean success;
    private String message;
}

```

File: account-service/src/main/java/com/ridelink/account_service/dto/ChangePasswordRequest.java

```

package com.ridelink.account_service.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor

```

```

@AllArgsConstructor
@Builder
public class ChangePasswordRequest {

    @NotBlank(message = "Current password is required")
    private String currentPassword;

    @NotBlank(message = "New password is required")
    @Size(min = 6, max = 50, message = "New password must be at least 6 characters long")
    private String newPassword;
}

```

File: account-service/src/main/java/com/ridelink/account_service/dto/ErrorResponse.java

```

package com.ridelink.account_service.dto;

import com.fasterxml.jackson.annotation.JsonInclude;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.time.LocalDateTime;
import java.util.Map;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
@JsonInclude(JsonInclude.Include.NON_NULL)
public class ErrorResponse {

    @Builder.Default
    private LocalDateTime timestamp = LocalDateTime.now();

    private int status;

    private String error;

    private String message;

    private String path;

    private Map<String, String> validationErrors;
}

```

File: account-service/src/main/java/com/ridelink/account_service/dto/LoginRequest.java

```

package com.ridelink.account_service.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter

```

```

@NoArgsConstructor
@AllArgsConstructor
@Builder
public class LoginRequest {

    @NotBlank(message = "Email is required")
    @Email(message = "Email must be a valid email address")
    private String email;

    @NotBlank(message = "Password is required")
    private String password;
}

```

File: account-service/src/main/java/com/ridelink/account_service/dto/LoginResponse.java

```

package com.ridelink.account_service.dto;

import com.ridelink.account_service.entity.AccountStatus;
import com.ridelink.account_service.entity.Role;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class LoginResponse {

    private String token;

    @Builder.Default
    private String tokenType = "Bearer";

    private long expiresIn;

    private String userId;

    private String email;

    private String fullName;

    private Role role;

    private AccountStatus status;
}

```

File: account-service/src/main/java/com/ridelink/account_service/dto/RegisterRequest.java

```

package com.ridelink.account_service.dto;

import com.ridelink.account_service.entity.Role;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import lombok.AllArgsConstructor;
import lombok.Builder;

```

```

import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class RegisterRequest {

    @NotBlank(message = "Full name is required")
    @Size(min = 2, max = 100, message = "Full name must be between 2 and 100 characters")
    private String fullName;

    @NotBlank(message = "Email is required")
    @Email(message = "Email must be a valid email address")
    private String email;

    @NotBlank(message = "Password is required")
    @Size(min = 6, max = 50, message = "Password must be at least 6 characters long")
    private String password;

    @NotBlank(message = "Phone number is required")
    @Pattern(regexp = "^[+0-9]{9,15}$", message = "Phone number must be a valid international format (e.g.
+94771234567)")
    private String phoneNumber;

    @NotNull(message = "Role is required (PASSENGER, DRIVER)")
    private Role role;
}

```

File: account-service/src/main/java/com/ridelink/account_service/dto/UpdateProfileRequest.java

```

package com.ridelink.account_service.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class UpdateProfileRequest {

    @NotBlank(message = "Full name cannot be blank")
    @Size(min = 2, max = 100, message = "Full name must be between 2 and 100 characters")
    private String fullName;

    @NotBlank(message = "Phone number cannot be blank")
    @Pattern(regexp = "^[+0-9]{9,15}$", message = "Phone number must be a valid international format (e.g.
+94771234567)")
    private String phoneNumber;
}

```

File: account-service/src/main/java/com/ridelink/account_service/dto/UpdateRoleRequest.java

```
package com.ridelink.account_service.dto;

import com.ridelink.account_service.entity.Role;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class UpdateRoleRequest {

    @NotNull(message = "Role is required (PASSENGER, DRIVER, ADMIN)")
    private Role role;
}
```

File: account-service/src/main/java/com/ridelink/account_service/dto/UpdateStatusRequest.java

```
package com.ridelink.account_service.dto;

import com.ridelink.account_service.entity.AccountStatus;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class UpdateStatusRequest {

    @NotNull(message = "Status is required (ACTIVE, SUSPENDED, DEACTIVATED)")
    private AccountStatus status;
}
```

File: account-service/src/main/java/com/ridelink/account_service/dto/UserResponse.java

```
package com.ridelink.account_service.dto;

import com.ridelink.account_service.entity.AccountStatus;
import com.ridelink.account_service.entity.Role;
import com.ridelink.account_service.entity.User;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.time.LocalDateTime;
```

```

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class UserResponse {

    private String id;
    private String fullName;
    private String email;
    private String phoneNumber;
    private Role role;
    private AccountStatus status;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;

    public static UserResponse fromUser(User user) {
        if (user == null) {
            return null;
        }
        return UserResponse.builder()
            .id(user.getId())
            .fullName(user.getFullName())
            .email(user.getEmail())
            .phoneNumber(user.getPhoneNumber())
            .role(user.getRole())
            .status(user.getStatus())
            .createdAt(user.getCreatedAt())
            .updatedAt(user.getUpdatedAt())
            .build();
    }
}

```

File: account-service/src/main/java/com/ridelink/account_service/entity/AccountStatus.java

```

package com.ridelink.account_service.entity;

public enum AccountStatus {
    ACTIVE,
    SUSPENDED,
    DEACTIVATED
}

```

File: account-service/src/main/java/com/ridelink/account_service/entity/Role.java

```

package com.ridelink.account_service.entity;

public enum Role {
    PASSENGER,
    DRIVER,
    ADMIN
}

```

File: account-service/src/main/java/com/ridelink/account_service/entity/User.java

```

package com.ridelink.account_service.entity;

import lombok.AllArgsConstructor;

```

```

import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.index.Indexed;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.LocalDateTime;

@Document(collection = "users")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User {

    @Id
    private String id;

    private String fullName;

    @Indexed(unique = true)
    private String email;

    private String password;

    private String phoneNumber;

    private Role role;

    @Builder.Default
    private AccountStatus status = AccountStatus.ACTIVE;

    @Builder.Default
    private LocalDateTime createdAt = LocalDateTime.now();

    @Builder.Default
    private LocalDateTime updatedAt = LocalDateTime.now();
}

```

File: account-service/src/main/java/com/ridelink/account_service/exception/AccountSuspendedException.java

```

package com.ridelink.account_service.exception;

public class AccountSuspendedException extends RuntimeException {
    public AccountSuspendedException(String message) {
        super(message);
    }
}

```

File: account-service/src/main/java/com/ridelink/account_service/exception/BadRequestException.java

```

package com.ridelink.account_service.exception;

public class BadRequestException extends RuntimeException {
    public BadRequestException(String message) {
        super(message);
    }
}

```

```
}
}
```

File: account-service/src/main/java/com/ridelink/account_service/exception/EmailAlreadyExistsException.java

```
package com.ridelink.account_service.exception;

public class EmailAlreadyExistsException extends RuntimeException {
    public EmailAlreadyExistsException(String message) {
        super(message);
    }
}
```

File: account-service/src/main/java/com/ridelink/account_service/exception/GlobalExceptionHandler.java

```
package com.ridelink.account_service.exception;

import com.ridelink.account_service.dto.ErrorResponse;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidationExceptions(
        MethodArgumentNotValidException ex, HttpServletRequest request) {

        Map<String, String> errors = new HashMap<>();
        for (FieldError error : ex.getBindingResult().getFieldErrors()) {
            errors.put(error.getField(), error.getDefaultMessage());
        }

        ErrorResponse response = ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(HttpStatus.BAD_REQUEST.value())
            .error("Validation Error")
            .message("Input validation failed")
            .path(request.getRequestURI())
            .validationErrors(errors)
            .build();

        return new ResponseEntity<>(response, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleResourceNotFoundException(
        ResourceNotFoundException ex, HttpServletRequest request) {
```

```

        ErrorResponse response = ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(HttpStatus.NOT_FOUND.value())
            .error("Not Found")
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .build();

        return new ResponseEntity<>(response, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(EmailAlreadyExistsException.class)
    public ResponseEntity<ErrorResponse> handleEmailAlreadyExistsException(
        EmailAlreadyExistsException ex, HttpServletRequest request) {

        ErrorResponse response = ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(HttpStatus.CONFLICT.value())
            .error("Conflict")
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .build();

        return new ResponseEntity<>(response, HttpStatus.CONFLICT);
    }

    @ExceptionHandler({InvalidCredentialsException.class, BadCredentialsException.class})
    public ResponseEntity<ErrorResponse> handleInvalidCredentialsException(
        Exception ex, HttpServletRequest request) {

        ErrorResponse response = ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(HttpStatus.UNAUTHORIZED.value())
            .error("Unauthorized")
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .build();

        return new ResponseEntity<>(response, HttpStatus.UNAUTHORIZED);
    }

    @ExceptionHandler(AccountSuspendedException.class)
    public ResponseEntity<ErrorResponse> handleAccountSuspendedException(
        AccountSuspendedException ex, HttpServletRequest request) {

        ErrorResponse response = ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(HttpStatus.FORBIDDEN.value())
            .error("Forbidden")
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .build();

        return new ResponseEntity<>(response, HttpStatus.FORBIDDEN);
    }

    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<ErrorResponse> handleAccessDeniedException(
        AccessDeniedException ex, HttpServletRequest request) {

        ErrorResponse response = ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(HttpStatus.FORBIDDEN.value())
            .error("Forbidden")
            .message("You do not have permission to access this resource")
            .path(request.getRequestURI())

```

```

        .build();

        return new ResponseEntity<>(response, HttpStatus.FORBIDDEN);
    }

    @ExceptionHandler(BadRequestException.class)
    public ResponseEntity<ErrorResponse> handleBadRequestException(
        BadRequestException ex, HttpServletRequest request) {

        ErrorResponse response = ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(HttpStatus.BAD_REQUEST.value())
            .error("Bad Request")
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .build();

        return new ResponseEntity<>(response, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneralException(
        Exception ex, HttpServletRequest request) {

        ErrorResponse response = ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(HttpStatus.INTERNAL_SERVER_ERROR.value())
            .error("Internal Server Error")
            .message(ex.getMessage() != null ? ex.getMessage() : "An unexpected error occurred")
            .path(request.getRequestURI())
            .build();

        return new ResponseEntity<>(response, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

```

File: account-service/src/main/java/com/ridelink/account_service/exception/InvalidCredentialsException.java

```

package com.ridelink.account_service.exception;

public class InvalidCredentialsException extends RuntimeException {
    public InvalidCredentialsException(String message) {
        super(message);
    }
}

```

File: account-service/src/main/java/com/ridelink/account_service/exception/ResourceNotFoundException.java

```

package com.ridelink.account_service.exception;

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}

```

File: account-service/src/main/java/com/ridelink/account_service/repository/UserRepository.java

```

package com.ridelink.account_service.repository;

import com.ridelink.account_service.entity.AccountStatus;
import com.ridelink.account_service.entity.Role;
import com.ridelink.account_service.entity.User;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface UserRepository extends MongoRepository<User, String> {

    Optional<User> findByEmail(String email);

    boolean existsByEmail(String email);

    List<User> findByRole(Role role);

    List<User> findByStatus(AccountStatus status);

    List<User> findByRoleAndStatus(Role role, AccountStatus status);
}

```

File: account-service/src/main/java/com/ridelink/account_service/security/JwtAuthenticationFilter.java

```

package com.ridelink.account_service.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.NonNull;
import lombok.RequiredArgsConstructor;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.Collections;

@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;

    @Override
    protected void doFilterInternal(
        @NonNull HttpServletRequest request,
        @NonNull HttpServletResponse response,
        @NonNull FilterChain filterChain
    ) throws ServletException, IOException {

        final String authHeader = request.getHeader("Authorization");
        final String jwt;
        final String userEmail;

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {

```

```

        filterChain.doFilter(request, response);
        return;
    }

    jwt = authHeader.substring(7);

    try {
        userEmail = jwtService.extractEmail(jwt);

        if (userEmail != null && SecurityContextHolder.getContext().getAuthentication() == null) {
            if (jwtService.isTokenValid(jwt, userEmail)) {
                String role = jwtService.extractRole(jwt);
                SimpleGrantedAuthority authority = new SimpleGrantedAuthority("ROLE_" + role);

                UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(
                    userEmail,
                    null,
                    Collections.singletonList(authority)
                );
                authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                SecurityContextHolder.getContext().setAuthentication(authToken);
            }
        }
    } catch (Exception ex) {
        // Log or ignore token parse issues so unauthenticated requests fail down the line
    }

    filterChain.doFilter(request, response);
}
}

```

File: account-service/src/main/java/com/ridelink/account_service/security/JwtService.java

```

package com.ridelink.account_service.security;

import com.ridelink.account_service.entity.User;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

@Service
public class JwtService {

    @Value("${jwt.secret:mySuperSecretKeyForRideLinkJWTAuthentication2026VerySecureKeyRideLink2026}")
    private String secretKey;

    @Value("${jwt.expiration:86400000}")
    private long jwtExpiration; // 24 hours in milliseconds

    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(secretKey.getBytes(StandardCharsets.UTF_8));
    }

    public String generateToken(User user) {
        Map<String, Object> extraClaims = new HashMap<>();
    }

```

```

        extraClaims.put("userId", user.getId());
        extraClaims.put("role", user.getRole().name());
        extraClaims.put("fullName", user.getFullName());
        extraClaims.put("status", user.getStatus().name());

        return Jwts.builder()
            .claims(extraClaims)
            .subject(user.getEmail())
            .issuedAt(new Date(System.currentTimeMillis()))
            .expiration(new Date(System.currentTimeMillis() + jwtExpiration))
            .signWith(getSigningKey())
            .compact();
    }

    public String extractEmail(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public String extractRole(String token) {
        Claims claims = extractAllClaims(token);
        return claims.get("role", String.class);
    }

    public String extractUserId(String token) {
        Claims claims = extractAllClaims(token);
        return claims.get("userId", String.class);
    }

    public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }

    public Claims extractAllClaims(String token) {
        return Jwts.parser()
            .verifyWith(getSigningKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }

    public boolean isTokenValid(String token, String email) {
        final String extractedEmail = extractEmail(token);
        return (extractedEmail.equals(email)) && !isTokenExpired(token);
    }

    public boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    private Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    public long getExpirationTime() {
        return jwtExpiration;
    }
}

```

File: account-service/src/main/java/com/ridelink/account_service/service/UserService.java

```

package com.ridelink.account_service.service;

import com.ridelink.account_service.dto.*;

```

```

import com.ridelink.account_service.entity.AccountStatus;
import com.ridelink.account_service.entity.Role;
import com.ridelink.account_service.entity.User;
import com.ridelink.account_service.exception.AccountSuspendedException;
import com.ridelink.account_service.exception.BadRequestException;
import com.ridelink.account_service.exception.EmailAlreadyExistsException;
import com.ridelink.account_service.exception.InvalidCredentialsException;
import com.ridelink.account_service.exception.ResourceNotFoundException;
import com.ridelink.account_service.repository.UserRepository;
import com.ridelink.account_service.security.JwtService;
import lombok.RequiredArgsConstructor;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.util.List;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
public class UserService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtService jwtService;

    public UserResponse register(RegisterRequest request) {
        try {
            String normalizedEmail = request.getEmail().toLowerCase().trim();

            System.out.println("🔍 REGISTER DEBUG: Starting registration for " + normalizedEmail);

            if (userRepository.existsByEmail(normalizedEmail)) {
                throw new EmailAlreadyExistsException("Email already exists: " + request.getEmail());
            }

            if (request.getRole() == Role.ADMIN) {
                throw new BadRequestException("Self-registration as ADMIN is not permitted");
            }

            User user = User.builder()
                .fullName(request.getFullName().trim())
                .email(normalizedEmail)
                .password(passwordEncoder.encode(request.getPassword()))
                .phoneNumber(request.getPhoneNumber().trim())
                .role(request.getRole())
                .status(AccountStatus.ACTIVE)
                .createdAt(LocalDateTime.now())
                .updatedAt(LocalDateTime.now())
                .build();

            System.out.println("💾 REGISTER DEBUG: About to save user to MongoDB");
            User savedUser = userRepository.save(user);

            // CRITICAL: Verify the save operation succeeded
            if (savedUser.getId() == null) {
                throw new RuntimeException("❌ CRITICAL: User save failed - no ID generated");
            }

            // Double-check by reading back from database
            boolean exists = userRepository.existsById(savedUser.getId());
            if (!exists) {
                throw new RuntimeException("❌ CRITICAL: User save verification failed - not found in database");
            }

            System.out.println("✅ REGISTER DEBUG: User saved successfully with ID: " + savedUser.getId());
        }
    }

```

```

        System.out.println("📄 REGISTER DEBUG: Total users in database: " + userRepository.count());

        return UserResponse.fromUser(savedUser);

    } catch (Exception e) {
        System.err.println("❌ REGISTER ERROR: " + e.getMessage());
        e.printStackTrace();
        throw new RuntimeException("Database operation failed: " + e.getMessage(), e);
    }
}

public LoginResponse login(LoginRequest request) {
    String normalizedEmail = request.getEmail().toLowerCase().trim();

    User user = userRepository.findByEmail(normalizedEmail)
        .orElseThrow(() -> new InvalidCredentialsException("Invalid email or password"));

    if (!passwordEncoder.matches(request.getPassword(), user.getPassword())) {
        throw new InvalidCredentialsException("Invalid email or password");
    }

    if (user.getStatus() == AccountStatus.SUSPENDED) {
        throw new AccountSuspendedException("Account has been suspended. Please contact support.");
    }

    if (user.getStatus() == AccountStatus.DEACTIVATED) {
        throw new AccountSuspendedException("Account has been deactivated. Please contact support to
reactivate.");
    }

    String token = jwtService.generateToken(user);

    return LoginResponse.builder()
        .token(token)
        .tokenType("Bearer")
        .expiresIn(jwtService.getExpirationTime())
        .userId(user.getId())
        .email(user.getEmail())
        .fullName(user.getFullName())
        .role(user.getRole())
        .status(user.getStatus())
        .build();
}

public UserResponse getProfile(String email) {
    User user = userRepository.findByEmail(email.toLowerCase().trim())
        .orElseThrow(() -> new ResourceNotFoundException("User not found with email: " + email));
    return UserResponse.fromUser(user);
}

public UserResponse getUserById(String id) {
    User user = userRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("User not found with id: " + id));
    return UserResponse.fromUser(user);
}

public UserResponse updateProfile(String email, UpdateProfileRequest request) {
    User user = userRepository.findByEmail(email.toLowerCase().trim())
        .orElseThrow(() -> new ResourceNotFoundException("User not found with email: " + email));

    user.setFullName(request.getFullName().trim());
    user.setPhoneNumber(request.getPhoneNumber().trim());
    user.setUpdatedAt(LocalDate.now());

    User updatedUser = userRepository.save(user);
    return UserResponse.fromUser(updatedUser);
}

```

```

public void changePassword(String email, ChangePasswordRequest request) {
    User user = userRepository.findByEmail(email.toLowerCase().trim())
        .orElseThrow(() -> new ResourceNotFoundException("User not found with email: " + email));

    if (!passwordEncoder.matches(request.getCurrentPassword(), user.getPassword())) {
        throw new InvalidCredentialsException("Current password does not match");
    }

    user.setPassword(passwordEncoder.encode(request.getNewPassword()));
    user.setUpdatedAt(LocalDateTime.now());
    userRepository.save(user);
}

public UserResponse updateStatus(String id, AccountStatus status) {
    User user = userRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("User not found with id: " + id));

    user.setStatus(status);
    user.setUpdatedAt(LocalDateTime.now());
    User updated = userRepository.save(user);
    return UserResponse.fromUser(updated);
}

public UserResponse updateRole(String id, Role role) {
    User user = userRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("User not found with id: " + id));

    user.setRole(role);
    user.setUpdatedAt(LocalDateTime.now());
    User updated = userRepository.save(user);
    return UserResponse.fromUser(updated);
}

public List<UserResponse> getAllUsers(Role role, AccountStatus status) {
    List<User> users;
    if (role != null && status != null) {
        users = userRepository.findByRoleAndStatus(role, status);
    } else if (role != null) {
        users = userRepository.findByRole(role);
    } else if (status != null) {
        users = userRepository.findByStatus(status);
    } else {
        users = userRepository.findAll();
    }

    return users.stream()
        .map(UserResponse::fromUser)
        .collect(Collectors.toList());
}

public UserResponse validateUser(String id) {
    User user = userRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("User not found with id: " + id));

    if (user.getStatus() != AccountStatus.ACTIVE) {
        throw new AccountSuspendedException("User account is not ACTIVE (current status: " +
user.getStatus() + ")");
    }

    return UserResponse.fromUser(user);
}

public void deactivateOwnAccount(String email) {
    User user = userRepository.findByEmail(email.toLowerCase().trim())
        .orElseThrow(() -> new ResourceNotFoundException("User not found with email: " + email));

```

```

        user.setStatus(AccountStatus.DEACTIVATED);
        user.setUpdatedAt(LocalDate.now());
        userRepository.save(user);
    }
}

```

File: account-service/src/main/java/com/ridelink/account_service/TestController.java

```

package com.ridelink.account_service;

import com.ridelink.account_service.entity.AccountStatus;
import com.ridelink.account_service.entity.Role;
import com.ridelink.account_service.entity.User;
import com.ridelink.account_service.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RestController;

import java.time.LocalDateTime;
import java.util.List;

@RestController
public class TestController {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private MongoTemplate mongoTemplate;

    @Value("${spring.data.mongodb.uri}")
    private String mongoUri;

    @GetMapping("/test/db")
    public String testDatabase() {
        try {
            long count = userRepository.count();
            return "✅ Database connection successful! User count: " + count;
        } catch (Exception e) {
            return "❌ Database connection failed: " + e.getMessage();
        }
    }

    @GetMapping("/test/connection-details")
    public String getConnectionDetails() {
        try {
            String dbName = mongoTemplate.getDb().getName();
            long userCount = userRepository.count();

            // Extract cluster info from URI
            String clusterInfo = mongoUri.contains("@") ?
                mongoUri.substring(mongoUri.indexOf("@") + 1, mongoUri.indexOf("/", mongoUri.indexOf("@"))) :
                "Unknown";

            return String.format("""
                ✅ MongoDB Connection Details:
                📁 Database Name: %s
                🌐 Cluster: %s
                👤 User Count: %d
                🔗 URI: %s...
            """, dbName, clusterInfo, userCount, mongoUri);
        } catch (Exception e) {
            return "❌ Error: " + e.getMessage();
        }
    }
}

```

```

        ⚠ If database not visible in Atlas:
        1. Check if you're in correct Atlas project
        2. Verify cluster name matches
        3. Refresh Atlas browser tab
        4. Check if data actually saved
        "",
        dbName, clusterInfo, userCount, mongoUri.substring(0, 30));

    } catch (Exception e) {
        return "❌ Connection details failed: " + e.getMessage();
    }
}

@GetMapping("/test/users")
public List<User> getAllUsers() {
    return userRepository.findAll();
}

@PostMapping("/test/create-sample-user")
public String createSampleUser() {
    try {
        String dbName = mongoTemplate.getDb().getName();

        // Check if sample user already exists
        if (userRepository.existsByEmail("sample@ridelink.com")) {
            long totalCount = userRepository.count();
            return String.format("""
                ✅ Sample user already exists!
                📊 Total users: %d
                📁 Database: %s
                📁 Collection: users

                ❓ Not visible in Atlas? Check:
                1. Atlas Project/Organization
                2. Cluster name: %s
                3. Browser refresh (F5)
                """,
                totalCount, dbName, mongoUri.contains("@") ?
                    mongoUri.substring(mongoUri.indexOf("@") + 1, mongoUri.indexOf("/"),
mongoUri.indexOf("@")) :
                    "cluster info not found");
        }

        User sampleUser = User.builder()
            .fullName("Sample Test User - " + LocalDateTime.now())
            .email("sample@ridelink.com")
            .password("$2a$10$dummyHashedPassword")
            .phoneNumber("1234567890")
            .role(Role.PASSENGER)
            .status(AccountStatus.ACTIVE)
            .createdAt(LocalDateTime.now())
            .updatedAt(LocalDateTime.now())
            .build();

        User saved = userRepository.save(sampleUser);
        long totalCount = userRepository.count();

        return String.format("""
            ✅ Sample user created successfully!
            🆔 User ID: %s
            👤 Name: %s
            📊 Total users in DB: %d
            📁 Database: %s
            📁 Collection: users

            🔍 To see in MongoDB Atlas:
            1. Go to Database → Browse Collections

```

```

        2. Look for database: %s
        3. Collection: users
        4. Should have %d documents
        """,
        saved.getId(), saved.getFullName(), totalCount, dbName, dbName, totalCount);

    } catch (Exception e) {
        return "❌ Failed to create user: " + e.getMessage();
    }
}

@PostMapping("/test/force-create-multiple")
public String createMultipleUsers() {
    try {
        String dbName = mongoTemplate.getDb().getName();

        // Create 3 test users to make database more visible
        for (int i = 1; i <= 3; i++) {
            String email = "test" + i + "@ridelink.com";
            if (!userRepository.existsByEmail(email)) {
                User user = User.builder()
                    .fullName("Test User " + i)
                    .email(email)
                    .password("$2a$10$dummyHashedPassword")
                    .phoneNumber("123456789" + i)
                    .role(Role.PASSENGER)
                    .status(AccountStatus.ACTIVE)
                    .createdAt(LocalDateTime.now())
                    .updatedAt(LocalDateTime.now())
                    .build();
                userRepository.save(user);
            }
        }

        long totalCount = userRepository.count();
        return String.format("""
            ✅ Multiple users created!
            📊 Total users: %d
            📁 Database: %s

            🚀 This should definitely be visible in Atlas now!
            Go to: Database → Browse Collections → %s → users
            """, totalCount, dbName, dbName);

    } catch (Exception e) {
        return "❌ Failed to create multiple users: " + e.getMessage();
    }
}
}

```

File: account-service/src/test/java/com/ridelink/account_service/AccountServiceApplicationTests.java

```

package com.ridelink.account_service;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class AccountServiceApplicationTests {

    @Test
    void contextLoads() {
    }
}

```

```
}
```

File: account-service/src/test/java/com/ridelink/account_service/controller/AuthControllerTest.java

```
package com.ridelink.account_service.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.ridelink.account_service.dto.*;
import com.ridelink.account_service.entity.AccountStatus;
import com.ridelink.account_service.entity.Role;
import com.ridelink.account_service.exception.EmailAlreadyExistsException;
import com.ridelink.account_service.exception.GlobalExceptionHandler;
import com.ridelink.account_service.exception.InvalidCredentialsException;
import com.ridelink.account_service.service.UserService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.setup.MockMvcBuilders;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@ExtendWith(MockitoExtension.class)
class AuthControllerTest {

    private MockMvc mockMvc;

    private ObjectMapper objectMapper;

    @Mock
    private UserService userService;

    @InjectMocks
    private AuthController authController;

    @BeforeEach
    void setUp() {
        objectMapper = new ObjectMapper();
        mockMvc = MockMvcBuilders.standaloneSetup(authController)
            .setControllerAdvice(new GlobalExceptionHandler())
            .build();
    }

    @Test
    void register_ValidRequest_ReturnsCreated() throws Exception {
        RegisterRequest request = RegisterRequest.builder()
            .fullName("John Passenger")
            .email("passenger@ridelink.com")
            .password("securePass123")
            .phoneNumber("+94771234567")
            .role(Role.PASSENGER)
            .build();

        UserResponse response = UserResponse.builder()
            .id("u100")
            .fullName("John Passenger")
```

```

        .email("passenger@ridelink.com")
        .phoneNumber("+94771234567")
        .role(Role.PASSENGER)
        .status(AccountStatus.ACTIVE)
        .build();

when(userService.register(any(RegisterRequest.class))).thenReturn(response);

mockMvc.perform(post("/api/auth/register")
    .contentType(MediaType.APPLICATION_JSON)
    .content(objectMapper.writeValueAsString(request)))
    .andExpect(status().isCreated())
    .andExpect(jsonPath("$.id").value("u100"))
    .andExpect(jsonPath("$.email").value("passenger@ridelink.com"))
    .andExpect(jsonPath("$.role").value("PASSENGER"));
}

@Test
void register_InvalidEmail_ReturnsBadRequest() throws Exception {
    RegisterRequest request = RegisterRequest.builder()
        .fullName("John Passenger")
        .email("not-an-email")
        .password("123")
        .phoneNumber("+94771234567")
        .role(Role.PASSENGER)
        .build();

    mockMvc.perform(post("/api/auth/register")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(request)))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.status").value(400))
        .andExpect(jsonPath("$.validationErrors.email").exists());
}

@Test
void register_DuplicateEmail_ReturnsConflict() throws Exception {
    RegisterRequest request = RegisterRequest.builder()
        .fullName("John Passenger")
        .email("exists@ridelink.com")
        .password("securePass123")
        .phoneNumber("+94771234567")
        .role(Role.PASSENGER)
        .build();

    when(userService.register(any(RegisterRequest.class)))
        .thenThrow(new EmailAlreadyExistsException("Email already exists: exists@ridelink.com"));

    mockMvc.perform(post("/api/auth/register")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(request)))
        .andExpect(status().isConflict())
        .andExpect(jsonPath("$.status").value(409));
}

@Test
void login_ValidCredentials_ReturnsToken() throws Exception {
    LoginRequest request = LoginRequest.builder()
        .email("passenger@ridelink.com")
        .password("securePass123")
        .build();

    LoginResponse response = LoginResponse.builder()
        .token("mocked.jwt.token")
        .tokenType("Bearer")
        .userId("u100")
        .email("passenger@ridelink.com")

```

```

        .fullName("John Passenger")
        .role(Role.PASSENGER)
        .status(AccountStatus.ACTIVE)
        .expiresIn(86400000L)
        .build();

when(userService.login(any(LoginRequest.class))).thenReturn(response);

mockMvc.perform(post("/api/auth/login")
    .contentType(MediaType.APPLICATION_JSON)
    .content(objectMapper.writeValueAsString(request)))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.token").value("mocked.jwt.token"))
    .andExpect(jsonPath("$.tokenType").value("Bearer"))
    .andExpect(jsonPath("$.role").value("PASSENGER"));
}

@Test
void login_InvalidCredentials_ReturnsUnauthorized() throws Exception {
    LoginRequest request = LoginRequest.builder()
        .email("passenger@ridelink.com")
        .password("wrongpassword")
        .build();

    when(userService.login(any(LoginRequest.class)))
        .thenThrow(new InvalidCredentialsException("Invalid email or password"));

    mockMvc.perform(post("/api/auth/login")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(request)))
        .andExpect(status().isUnauthorized())
        .andExpect(jsonPath("$.status").value(401));
}
}

```

File: account-service/src/test/java/com/ridelink/account_service/controller/UserControllerTest.java

```

package com.ridelink.account_service.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.ridelink.account_service.dto.*;
import com.ridelink.account_service.entity.AccountStatus;
import com.ridelink.account_service.entity.Role;
import com.ridelink.account_service.exception.GlobalExceptionHandler;
import com.ridelink.account_service.service.UserService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.http.MediaType;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.setup.MockMvcBuilders;

import java.util.Collections;
import java.util.List;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.when;

```

```

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@ExtendWith(MockitoExtension.class)
class UserControllerTest {

    private MockMvc mockMvc;

    private ObjectMapper objectMapper;

    @Mock
    private UserService userService;

    @InjectMocks
    private UserController userController;

    private UserResponse sampleResponse;

    @BeforeEach
    void setUp() {
        objectMapper = new ObjectMapper();
        mockMvc = MockMvcBuilders.standaloneSetup(userController)
            .setControllerAdvice(new GlobalExceptionHandler())
            .build();

        sampleResponse = UserResponse.builder()
            .id("u1")
            .fullName("John Doe")
            .email("john@ridelink.com")
            .phoneNumber("+94771234567")
            .role(Role.PASSENGER)
            .status(AccountStatus.ACTIVE)
            .build();

        UsernamePasswordAuthenticationToken auth = new UsernamePasswordAuthenticationToken(
            "john@ridelink.com",
            null,
            Collections.singletonList(new SimpleGrantedAuthority("ROLE_PASSENGER"))
        );
        SecurityContextHolder.getContext().setAuthentication(auth);
    }

    @Test
    void getCurrentUser_ReturnsOk() throws Exception {
        when(userService.getProfile("john@ridelink.com")).thenReturn(sampleResponse);

        mockMvc.perform(get("/api/users/me").principal(() -> "john@ridelink.com"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.email").value("john@ridelink.com"))
            .andExpect(jsonPath("$.fullName").value("John Doe"));
    }

    @Test
    void getUserById_ReturnsOk() throws Exception {
        when(userService.getUserById("u1")).thenReturn(sampleResponse);

        mockMvc.perform(get("/api/users/u1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value("u1"));
    }

    @Test
    void updateProfile_ReturnsOk() throws Exception {
        UpdateProfileRequest request = UpdateProfileRequest.builder()
            .fullName("John Updated")
            .phoneNumber("+94779999999")
            .build();
    }

```

```

        when(userService.updateProfile(eq("john@ridelink.com"), any(UpdateProfileRequest.class)))
            .thenReturn(sampleResponse);

        mockMvc.perform(put("/api/users/profile")
            .principal(() -> "john@ridelink.com")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isOk());
    }

    @Test
    void changePassword_ReturnsOk() throws Exception {
        ChangePasswordRequest request = ChangePasswordRequest.builder()
            .currentPassword("oldPassword123")
            .newPassword("newPassword123")
            .build();

        mockMvc.perform(post("/api/users/change-password")
            .principal(() -> "john@ridelink.com")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.success").value(true));
    }

    @Test
    void updateStatus_ReturnsOk() throws Exception {
        UpdateStatusRequest request = UpdateStatusRequest.builder()
            .status(AccountStatus.SUSPENDED)
            .build();

        sampleResponse.setStatus(AccountStatus.SUSPENDED);
        when(userService.updateStatus(eq("u1"), eq(AccountStatus.SUSPENDED))).thenReturn(sampleResponse);

        mockMvc.perform(patch("/api/users/u1/status")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.status").value("SUSPENDED"));
    }

    @Test
    void updateRole_ReturnsOk() throws Exception {
        UpdateRoleRequest request = UpdateRoleRequest.builder()
            .role(Role.DRIVER)
            .build();

        sampleResponse.setRole(Role.DRIVER);
        when(userService.updateRole(eq("u1"), eq(Role.DRIVER))).thenReturn(sampleResponse);

        mockMvc.perform(patch("/api/users/u1/role")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.role").value("DRIVER"));
    }

    @Test
    void getAllUsers_ReturnsList() throws Exception {
        List<UserResponse> list = Collections.singletonList(sampleResponse);
        when(userService.getAllUsers(null, null)).thenReturn(list);

        mockMvc.perform(get("/api/users"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.length()").value(1));
    }

```

```

@Test
void validateUser_ReturnsOk() throws Exception {
    when(userService.validateUser("u1")).thenReturn(sampleResponse);

    mockMvc.perform(get("/api/users/u1/validate"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.status").value("ACTIVE"));
}
}

```

File: account-service/src/test/java/com/ridelink/account_service/security/JwtServiceTest.java

```

package com.ridelink.account_service.security;

import com.ridelink.account_service.entity.AccountStatus;
import com.ridelink.account_service.entity.Role;
import com.ridelink.account_service.entity.User;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.test.util.ReflectionTestUtils;

import static org.junit.jupiter.api.Assertions.*;

class JwtServiceTest {

    private JwtService jwtService;

    @BeforeEach
    void setUp() {
        jwtService = new JwtService();
        ReflectionTestUtils.setField(jwtService, "secretKey",
            "mySuperSecretKeyForRideLinkJWTAuthentication2026VerySecureKeyRideLink2026");
        ReflectionTestUtils.setField(jwtService, "jwtExpiration", 3600000L); // 1 hour
    }

    @Test
    void testGenerateTokenAndExtractClaims() {
        User user = User.builder()
            .id("user123")
            .email("test@ridelink.com")
            .fullName("John Doe")
            .role(Role.PASSENGER)
            .status(AccountStatus.ACTIVE)
            .build();

        String token = jwtService.generateToken(user);

        assertNotNull(token);
        assertEquals("test@ridelink.com", jwtService.extractEmail(token));
        assertEquals("PASSENGER", jwtService.extractRole(token));
        assertEquals("user123", jwtService.extractUserId(token));
        assertTrue(jwtService.isTokenValid(token, "test@ridelink.com"));
        assertFalse(jwtService.isTokenValid(token, "wrong@ridelink.com"));
        assertFalse(jwtService.isTokenExpired(token));
    }
}

```

File: account-service/src/test/java/com/ridelink/account_service/service/UserServiceTest.java

```

package com.ridelink.account_service.service;

```

```

import com.ridelink.account_service.dto.*;
import com.ridelink.account_service.entity.AccountStatus;
import com.ridelink.account_service.entity.Role;
import com.ridelink.account_service.entity.User;
import com.ridelink.account_service.exception.AccountSuspendedException;
import com.ridelink.account_service.exception.BadRequestException;
import com.ridelink.account_service.exception.EmailAlreadyExistsException;
import com.ridelink.account_service.exception.InvalidCredentialsException;
import com.ridelink.account_service.exception.ResourceNotFoundException;
import com.ridelink.account_service.repository.UserRepository;
import com.ridelink.account_service.security.JwtService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.security.crypto.password.PasswordEncoder;

import java.time.LocalDateTime;
import java.util.Collections;
import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @Mock
    private PasswordEncoder passwordEncoder;

    @Mock
    private JwtService jwtService;

    @InjectMocks
    private UserService userService;

    private User sampleUser;

    @BeforeEach
    void setUp() {
        sampleUser = User.builder()
            .id("u1")
            .fullName("John Doe")
            .email("john@example.com")
            .password("encoded_pass")
            .phoneNumber("+94771234567")
            .role(Role.PASSENGER)
            .status(AccountStatus.ACTIVE)
            .createdAt(LocalDateTime.now())
            .updatedAt(LocalDateTime.now())
            .build();
    }

    @Test
    void register_Success() {
        RegisterRequest request = RegisterRequest.builder()
            .fullName("John Doe")
            .email("john@example.com")
            .password("password123")
            .phoneNumber("+94771234567")

```

```

        .role(Role.PASSENGER)
        .build();

when(userRepository.existsByEmail("john@example.com")).thenReturn(false);
when(passwordEncoder.encode("password123")).thenReturn("encoded_pass");
when(userRepository.save(any(User.class))).thenReturn(sampleUser);

UserResponse response = userService.register(request);

assertNotNull(response);
assertEquals("john@example.com", response.getEmail());
assertEquals("John Doe", response.getFullName());
assertEquals(Role.PASSENGER, response.getRole());
assertEquals(AccountStatus.ACTIVE, response.getStatus());
verify(userRepository, times(1)).save(any(User.class));
}

@Test
void register_DuplicateEmail_ThrowsException() {
    RegisterRequest request = RegisterRequest.builder()
        .fullName("John Doe")
        .email("john@example.com")
        .password("password123")
        .phoneNumber("+94771234567")
        .role(Role.PASSENGER)
        .build();

    when(userRepository.existsByEmail("john@example.com")).thenReturn(true);

    assertThrows(EmailAlreadyExistsException.class, () -> userService.register(request));
    verify(userRepository, never()).save(any(User.class));
}

@Test
void register_AdminRole_ThrowsException() {
    RegisterRequest request = RegisterRequest.builder()
        .fullName("Admin")
        .email("admin@example.com")
        .password("password123")
        .phoneNumber("+94771234567")
        .role(Role.ADMIN)
        .build();

    when(userRepository.existsByEmail("admin@example.com")).thenReturn(false);

    assertThrows(BadRequestException.class, () -> userService.register(request));
}

@Test
void login_Success() {
    LoginRequest request = LoginRequest.builder()
        .email("john@example.com")
        .password("password123")
        .build();

    when(userRepository.findByEmail("john@example.com")).thenReturn(Optional.of(sampleUser));
    when(passwordEncoder.matches("password123", "encoded_pass")).thenReturn(true);
    when(jwtService.generateToken(sampleUser)).thenReturn("sample.jwt.token");
    when(jwtService.getExpirationTime()).thenReturn(86400000L);

    LoginResponse response = userService.login(request);

    assertNotNull(response);
    assertEquals("sample.jwt.token", response.getToken());
    assertEquals("john@example.com", response.getEmail());
    assertEquals(Role.PASSENGER, response.getRole());
}

```

```
@Test
void login_InvalidPassword_ThrowsException() {
    LoginRequest request = LoginRequest.builder()
        .email("john@example.com")
        .password("wrongpassword")
        .build();

    when(userRepository.findByEmail("john@example.com")).thenReturn(Optional.of(sampleUser));
    when(passwordEncoder.matches("wrongpassword", "encoded_pass")).thenReturn(false);

    assertThrows(InvalidCredentialsException.class, () -> userService.login(request));
}

@Test
void login_SuspendedUser_ThrowsException() {
    sampleUser.setStatus(AccountStatus.SUSPENDED);
    LoginRequest request = LoginRequest.builder()
        .email("john@example.com")
        .password("password123")
        .build();

    when(userRepository.findByEmail("john@example.com")).thenReturn(Optional.of(sampleUser));
    when(passwordEncoder.matches("password123", "encoded_pass")).thenReturn(true);

    assertThrows(AccountSuspendedException.class, () -> userService.login(request));
}

@Test
void getProfile_Success() {
    when(userRepository.findByEmail("john@example.com")).thenReturn(Optional.of(sampleUser));

    UserResponse response = userService.getProfile("john@example.com");

    assertNotNull(response);
    assertEquals("John Doe", response.getFullName());
}

@Test
void getProfile_NotFound_ThrowsException() {
    when(userRepository.findByEmail("nonexistent@example.com")).thenReturn(Optional.empty());

    assertThrows(ResourceNotFoundException.class, () -> userService.getProfile("nonexistent@example.com"));
}

@Test
void updateProfile_Success() {
    UpdateProfileRequest request = UpdateProfileRequest.builder()
        .fullName("John Updated")
        .phoneNumber("+94779999999")
        .build();

    when(userRepository.findByEmail("john@example.com")).thenReturn(Optional.of(sampleUser));
    when(userRepository.save(any(User.class))).thenReturn(sampleUser);

    UserResponse response = userService.updateProfile("john@example.com", request);

    assertNotNull(response);
    assertEquals("John Updated", sampleUser.getFullName());
    assertEquals("+94779999999", sampleUser.getPhoneNumber());
    verify(userRepository, times(1)).save(sampleUser);
}

@Test
void changePassword_Success() {
    ChangePasswordRequest request = ChangePasswordRequest.builder()
        .currentPassword("old_pass")
```

```

        .newPassword("new_pass123")
        .build();

when(userRepository.findById("john@example.com")).thenReturn(Optional.of(sampleUser));
when(passwordEncoder.matches("old_pass", "encoded_pass")).thenReturn(true);
when(passwordEncoder.encode("new_pass123")).thenReturn("new_encoded_pass");

userService.changePassword("john@example.com", request);

assertEquals("new_encoded_pass", sampleUser.getPassword());
verify(userRepository, times(1)).save(sampleUser);
}

@Test
void updateStatus_Success() {
    when(userRepository.findById("u1")).thenReturn(Optional.of(sampleUser));
    when(userRepository.save(any(User.class))).thenReturn(sampleUser);

    UserResponse response = userService.updateStatus("u1", AccountStatus.SUSPENDED);

    assertNotNull(response);
    assertEquals(AccountStatus.SUSPENDED, sampleUser.getStatus());
    verify(userRepository, times(1)).save(sampleUser);
}

@Test
void updateRole_Success() {
    when(userRepository.findById("u1")).thenReturn(Optional.of(sampleUser));
    when(userRepository.save(any(User.class))).thenReturn(sampleUser);

    UserResponse response = userService.updateRole("u1", Role.DRIVER);

    assertNotNull(response);
    assertEquals(Role.DRIVER, sampleUser.getRole());
    verify(userRepository, times(1)).save(sampleUser);
}

@Test
void validateUser_Active_Success() {
    when(userRepository.findById("u1")).thenReturn(Optional.of(sampleUser));

    UserResponse response = userService.validateUser("u1");

    assertNotNull(response);
    assertEquals(AccountStatus.ACTIVE, response.getStatus());
}

@Test
void validateUser_Suspended_ThrowsException() {
    sampleUser.setStatus(AccountStatus.SUSPENDED);
    when(userRepository.findById("u1")).thenReturn(Optional.of(sampleUser));

    assertThrows(AccountSuspendedException.class, () -> userService.validateUser("u1"));
}

@Test
void getAllUsers_Filters() {
    when(userRepository.findByRoleAndStatus(Role.PASSENGER, AccountStatus.ACTIVE))
        .thenReturn(Collections.singletonList(sampleUser));

    List<UserResponse> list = userService.getAllUsers(Role.PASSENGER, AccountStatus.ACTIVE);

    assertEquals(1, list.size());
    assertEquals("john@example.com", list.get(0).getEmail());
}
}

```

B.2 Driver & Vehicle Service (Member 2 - Port 8082)

Total Source Files: 33

File: driver-service/src/main/java/com/ridelink/driver_service/client/AccountServiceClient.java

```
package com.ridelink.driver_service.client;

import com.ridelink.driver_service.dto.AccountUserValidationDto;
import com.ridelink.driver_service.exception.AccountServiceException;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;

@Component
@Slf4j
public class AccountServiceClient {

    private final RestClient restClient;

    public AccountServiceClient(
        RestClient.Builder restClientBuilder,
        @Value("${services.account-service.url:http://localhost:8081}") String accountServiceUrl
    ) {
        this.restClient = restClientBuilder.baseUrl(accountServiceUrl).build();
    }

    public AccountUserValidationDto validateDriverUser(String userId) {
        try {
            AccountUserValidationDto user = restClient.get()
                .uri("/api/users/{id}/validate", userId)
                .retrieve()
                .onStatus(HttpStatus::is4xxClientError, (request, response) -> {
                    throw new AccountServiceException("User validation failed in Account Service: status "
+ response.getStatusCode());
                })
                .onStatus(HttpStatus::is5xxServerError, (request, response) -> {
                    throw new AccountServiceException("Account Service is temporarily unavailable");
                })
                .body(AccountUserValidationDto.class);

            if (user == null) {
                throw new AccountServiceException("User not found in Account Service with ID: " + userId);
            }

            if (!"ACTIVE".equalsIgnoreCase(user.getStatus())) {
                throw new AccountServiceException("User account is not ACTIVE in Account Service (Status: "
+ user.getStatus() + ")");
            }

            if (!"DRIVER".equalsIgnoreCase(user.getRole())) {
                throw new AccountServiceException("User does not have DRIVER role in Account Service (Role: "
+ user.getRole() + ")");
            }

            return user;
        } catch (AccountServiceException ex) {
            throw ex;
        } catch (Exception ex) {
```

```

        log.error("Failed to connect to Account Service: {}", ex.getMessage());
        throw new AccountServiceException("Unable to communicate with Account Service: " +
ex.getMessage());
    }
}
}

```

File: driver-service/src/main/java/com/ridelink/driver_service/config/OpenApiConfig.java

```

package com.ridelink.driver_service.config;

import io.swagger.v3.oas.models.Components;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Contact;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.info.License;
import io.swagger.v3.oas.models.security.SecurityRequirement;
import io.swagger.v3.oas.models.security.SecurityScheme;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {

    private static final String SECURITY_SCHEME_NAME = "Bearer Authentication";

    @Bean
    public OpenAPI driverServiceOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("RideLink - Driver & Vehicle Service API")
                .description("Microservice responsible for driver operational profiles, vehicle
registration, availability states, simulated location, and retrieval of eligible drivers.")
                .version("1.0.0")
                .contact(new Contact().name("RideLink Backend Team"))
                .license(new License().name("Apache 2.0").url("https://spring.io")))
            .addSecurityItem(new SecurityRequirement().addList(SECURITY_SCHEME_NAME))
            .components(new Components()
                .addSecuritySchemes(SECURITY_SCHEME_NAME, new SecurityScheme()
                    .name(SECURITY_SCHEME_NAME)
                    .type(SecurityScheme.Type.HTTP)
                    .scheme("bearer")
                    .bearerFormat("JWT"))));
    }
}

```

File: driver-service/src/main/java/com/ridelink/driver_service/config/RestClientConfig.java

```

package com.ridelink.driver_service.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestClient;

@Configuration
public class RestClientConfig {

    @Bean
    public RestClient.Builder restClientBuilder() {
        return RestClient.builder();
    }
}

```

```
}
}
```

File: driver-service/src/main/java/com/ridelink/driver_service/config/SecurityConfig.java

```
package com.ridelink.driver_service.config;

import com.ridelink.driver_service.security.JwtAuthenticationFilter;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthFilter;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                // Public / Documentation
                .requestMatchers(
                    "/v3/api-docs/**",
                    "/v3/api-docs",
                    "/swagger-ui/**",
                    "/swagger-ui.html",
                    "/test/**"
                ).permitAll()
                // Public / Inter-service operations
                .requestMatchers(HttpMethod.GET, "/api/drivers/available").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/drivers/*").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/drivers/user/*").permitAll()
                .requestMatchers(HttpMethod.PATCH, "/api/drivers/*/trip-status").permitAll()
                // Protected Driver profile registration and updates
                .anyRequest().authenticated()
            )
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint((request, response, authException) -> {
                    response.setContentType("application/json");
                    response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
                    response.getWriter().write(
{"status\\":401,\"error\\":\\\"Unauthorized\\\", \"message\\\":\\\"Authentication token is missing or invalid\\\", \"path\\\":\\\"\" + request.getRequestURI() + "\\\""});
                })
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

```
}
}
```

File: driver-service/src/main/java/com/ridelink/driver_service/controller/DriverController.java

```
package com.ridelink.driver_service.controller;

import com.ridelink.driver_service.dto.*;
import com.ridelink.driver_service.entity.VehicleType;
import com.ridelink.driver_service.service.DriverService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.security.SecurityRequirement;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/drivers")
@RequiredArgsConstructor
@Tag(name = "Driver & Vehicle Management", description = "Operations for driver operational profiles, vehicles, availability, and location")
public class DriverController {

    private final DriverService driverService;

    @PostMapping
    @Operation(summary = "Register driver profile and vehicle", security = @SecurityRequirement(name = "Bearer Authentication"))
    @ApiResponses(value = {
        @ApiResponse(responseCode = "201", description = "Driver profile created successfully"),
        @ApiResponse(responseCode = "400", description = "Validation error or Account Service verification failure"),
        @ApiResponse(responseCode = "409", description = "Driver profile or license number already exists")
    })
    public ResponseEntity<DriverResponse> registerDriver(@Valid @RequestBody DriverRegistrationRequest request) {
        DriverResponse response = driverService.registerDriver(request);
        return new ResponseEntity<>(response, HttpStatus.CREATED);
    }

    @GetMapping("/{id}")
    @Operation(summary = "Get driver profile by Driver ID")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Driver retrieved successfully"),
        @ApiResponse(responseCode = "404", description = "Driver not found")
    })
    public ResponseEntity<DriverResponse> getDriverById(@PathVariable String id) {
        DriverResponse response = driverService.getDriverById(id);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/user/{userId}")
    @Operation(summary = "Get driver profile by Account Service User ID")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Driver profile found"),
        @ApiResponse(responseCode = "404", description = "Driver profile not found")
    })
}
```

```

    public ResponseEntity<DriverResponse> getDriverByUserId(@PathVariable String userId) {
        DriverResponse response = driverService.getDriverByUserId(userId);
        return ResponseEntity.ok(response);
    }

    @PutMapping("/{id}/vehicle")
    @Operation(summary = "Update driver vehicle details", security = @SecurityRequirement(name = "Bearer Authentication"))
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Vehicle updated successfully"),
        @ApiResponse(responseCode = "400", description = "Validation error"),
        @ApiResponse(responseCode = "404", description = "Driver not found")
    })
    public ResponseEntity<DriverResponse> updateVehicle(
        @PathVariable String id,
        @Valid @RequestBody UpdateVehicleRequest request
    ) {
        DriverResponse response = driverService.updateVehicle(id, request);
        return ResponseEntity.ok(response);
    }

    @PatchMapping("/{id}/availability")
    @Operation(summary = "Update driver availability status (ONLINE, OFFLINE, ON_TRIP)", security = @SecurityRequirement(name = "Bearer Authentication"))
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Availability updated successfully"),
        @ApiResponse(responseCode = "404", description = "Driver not found")
    })
    public ResponseEntity<DriverResponse> updateAvailability(
        @PathVariable String id,
        @Valid @RequestBody UpdateAvailabilityRequest request
    ) {
        DriverResponse response = driverService.updateAvailability(id, request.getStatus());
        return ResponseEntity.ok(response);
    }

    @PatchMapping("/{id}/location")
    @Operation(summary = "Update driver simulated location and service area", security = @SecurityRequirement(name = "Bearer Authentication"))
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Location updated successfully"),
        @ApiResponse(responseCode = "404", description = "Driver not found")
    })
    public ResponseEntity<DriverResponse> updateLocation(
        @PathVariable String id,
        @Valid @RequestBody UpdateLocationRequest request
    ) {
        DriverResponse response = driverService.updateLocation(id, request);
        return ResponseEntity.ok(response);
    }

    @PatchMapping("/{id}/trip-status")
    @Operation(summary = "Update driver trip assignment status (Used by Ride Management Service)")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Trip status updated"),
        @ApiResponse(responseCode = "404", description = "Driver not found")
    })
    public ResponseEntity<DriverResponse> updateTripStatus(
        @PathVariable String id,
        @Valid @RequestBody UpdateTripStatusRequest request
    ) {
        DriverResponse response = driverService.updateTripStatus(id, request.getOnTrip());
        return ResponseEntity.ok(response);
    }

    @GetMapping("/available")
    @Operation(summary = "Retrieve eligible available drivers (Used by Ride Management Service)")

```

```

    @ApiResponse(value = {
        @ApiResponse(responseCode = "200", description = "List of available eligible drivers")
    })
    public ResponseEntity<List<DriverResponse>> getAvailableDrivers(
        @RequestParam(required = false) String serviceArea,
        @RequestParam(required = false) VehicleType vehicleType
    ) {
        List<DriverResponse> response = driverService.getAvailableDrivers(serviceArea, vehicleType);
        return ResponseEntity.ok(response);
    }
}

```

File: driver-service/src/main/java/com/ridelink/driver_service/DriverServiceApplication.java

```

package com.ridelink.driver_service;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DriverServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(DriverServiceApplication.class, args);
    }
}

```

File: driver-service/src/main/java/com/ridelink/driver_service/dto/AccountUserValidationDto.java

```

package com.ridelink.driver_service.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class AccountUserValidationDto {
    private String id;
    private String fullName;
    private String email;
    private String phoneNumber;
    private String role;
    private String status;
}

```

File: driver-service/src/main/java/com/ridelink/driver_service/dto/ApiResponse.java

```

package com.ridelink.driver_service.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;

```

```
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ApiResponse {
    private boolean success;
    private String message;
}
```

File: driver-service/src/main/java/com/ridelink/driver_service/dto/DriverRegistrationRequest.java

```
package com.ridelink.driver_service.dto;

import com.ridelink.driver_service.entity.VehicleType;
import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Pattern;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class DriverRegistrationRequest {

    @NotBlank(message = "User ID is required from Account Service")
    private String userId;

    @NotBlank(message = "Driver name is required")
    private String driverName;

    @NotBlank(message = "Phone number is required")
    @Pattern(regexp = "[+0-9]{9,15}$", message = "Phone number must be a valid format")
    private String phoneNumber;

    @NotBlank(message = "License number is required")
    private String licenseNumber;

    @Min(value = 1, message = "Experience must be at least 1 year")
    private int experienceYears;

    // Vehicle Info
    @NotBlank(message = "Vehicle make is required")
    private String vehicleMake;

    @NotBlank(message = "Vehicle model is required")
    private String vehicleModel;

    @Min(value = 2000, message = "Vehicle year must be 2000 or newer")
    private int vehicleYear;

    @NotBlank(message = "License plate number is required")
    private String licensePlate;
```

```
@NotNull(message = "Vehicle type is required (CAR, VAN, BIKE, TUK)")
private VehicleType vehicleType;

@Min(value = 1, message = "Capacity must be at least 1 passenger")
@Max(value = 15, message = "Capacity cannot exceed 15 passengers")
private int vehicleCapacity;

// Initial Location Info
private double latitude;
private double longitude;

@NotBlank(message = "Service area is required (e.g. Colombo, Kandy)")
private String serviceArea;
}
```

File: driver-service/src/main/java/com/ridelink/driver_service/dto/DriverResponse.java

```
package com.ridelink.driver_service.dto;

import com.ridelink.driver_service.entity.AvailabilityStatus;
import com.ridelink.driver_service.entity.Driver;
import com.ridelink.driver_service.entity.Location;
import com.ridelink.driver_service.entity.Vehicle;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.time.LocalDateTime;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class DriverResponse {

    private String id;
    private String userId;
    private String driverName;
    private String phoneNumber;
    private String licenseNumber;
    private int experienceYears;
    private double rating;
    private int totalTrips;
    private Vehicle vehicle;
    private Location location;
    private AvailabilityStatus status;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;

    public static DriverResponse fromDriver(Driver driver) {
        if (driver == null) {
            return null;
        }
        return DriverResponse.builder()
            .id(driver.getId())
            .userId(driver.getUserId())
            .driverName(driver.getDriverName())
            .phoneNumber(driver.getPhoneNumber())
            .licenseNumber(driver.getLicenseNumber())
            .experienceYears(driver.getExperienceYears())
            .rating(driver.getRating())
    }
}
```

```

        .totalTrips(driver.getTotalTrips())
        .vehicle(driver.getVehicle())
        .location(driver.getLocation())
        .status(driver.getStatus())
        .createdAt(driver.getCreatedAt())
        .updatedAt(driver.getUpdatedAt())
        .build();
    }
}

```

File: driver-service/src/main/java/com/ridelink/driver_service/dto/ErrorResponse.java

```

package com.ridelink.driver_service.dto;

import com.fasterxml.jackson.annotation.JsonInclude;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.time.LocalDateTime;
import java.util.Map;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
@JsonInclude(JsonInclude.Include.NON_NULL)
public class ErrorResponse {

    @Builder.Default
    private LocalDateTime timestamp = LocalDateTime.now();

    private int status;

    private String error;

    private String message;

    private String path;

    private Map<String, String> validationErrors;
}

```

File: driver-service/src/main/java/com/ridelink/driver_service/dto/UpdateAvailabilityRequest.java

```

package com.ridelink.driver_service.dto;

import com.ridelink.driver_service.entity.AvailabilityStatus;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor

```

```
@Builder
public class UpdateAvailabilityRequest {

    @NotNull(message = "Availability status is required (ONLINE, OFFLINE, ON_TRIP)")
    private AvailabilityStatus status;
}
```

File: driver-service/src/main/java/com/ridelink/driver_service/dto/UpdateLocationRequest.java

```
package com.ridelink.driver_service.dto;

import jakarta.validation.constraints.NotBlank;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class UpdateLocationRequest {

    private double latitude;

    private double longitude;

    @NotBlank(message = "Service area is required (e.g. Colombo, Kandy)")
    private String serviceArea;
}
```

File: driver-service/src/main/java/com/ridelink/driver_service/dto/UpdateTripStatusRequest.java

```
package com.ridelink.driver_service.dto;

import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class UpdateTripStatusRequest {

    @NotNull(message = "onTrip boolean status is required")
    private Boolean onTrip;
}
```

File: driver-service/src/main/java/com/ridelink/driver_service/dto/UpdateVehicleRequest.java

```
package com.ridelink.driver_service.dto;
```

```

import com.ridelink.driver_service.entity.VehicleType;
import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class UpdateVehicleRequest {

    @NotBlank(message = "Vehicle make is required")
    private String make;

    @NotBlank(message = "Vehicle model is required")
    private String model;

    @Min(value = 2000, message = "Vehicle year must be 2000 or newer")
    private int year;

    @NotBlank(message = "License plate number is required")
    private String licensePlate;

    @NotNull(message = "Vehicle type is required")
    private VehicleType vehicleType;

    @Min(value = 1, message = "Capacity must be at least 1 passenger")
    @Max(value = 15, message = "Capacity cannot exceed 15 passengers")
    private int capacity;
}

```

File: driver-service/src/main/java/com/ridelink/driver_service/entity/AvailabilityStatus.java

```

package com.ridelink.driver_service.entity;

public enum AvailabilityStatus {
    ONLINE,
    OFFLINE,
    ON_TRIP
}

```

File: driver-service/src/main/java/com/ridelink/driver_service/entity/Driver.java

```

package com.ridelink.driver_service.entity;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.index.Indexed;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.LocalDateTime;

```

```
@Document(collection = "drivers")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Driver {

    @Id
    private String id;

    @Indexed(unique = true)
    private String userId;

    private String driverName;

    private String phoneNumber;

    @Indexed(unique = true)
    private String licenseNumber;

    private int experienceYears;

    @Builder.Default
    private double rating = 5.0;

    @Builder.Default
    private int totalTrips = 0;

    private Vehicle vehicle;

    private Location location;

    @Builder.Default
    private AvailabilityStatus status = AvailabilityStatus.OFFLINE;

    @Builder.Default
    private LocalDateTime createdAt = LocalDateTime.now();

    @Builder.Default
    private LocalDateTime updatedAt = LocalDateTime.now();
}
```

File: driver-service/src/main/java/com/ridelink/driver_service/entity/Location.java

```
package com.ridelink.driver_service.entity;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.time.LocalDateTime;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Location {

    private double latitude;
```

```
private double longitude;

private String serviceArea;

@Builder.Default
private LocalDateTime lastUpdated = LocalDateTime.now();
}
```

File: driver-service/src/main/java/com/ridelink/driver_service/entity/Vehicle.java

```
package com.ridelink.driver_service.entity;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Vehicle {

    private String make;

    private String model;

    private int year;

    private String licensePlate;

    private VehicleType vehicleType;

    private int capacity;
}
```

File: driver-service/src/main/java/com/ridelink/driver_service/entity/VehicleType.java

```
package com.ridelink.driver_service.entity;

public enum VehicleType {
    CAR,
    VAN,
    BIKE,
    TUK
}
```

File: driver-service/src/main/java/com/ridelink/driver_service/exception/AccountServiceException.java

```
package com.ridelink.driver_service.exception;

public class AccountServiceException extends RuntimeException {
    public AccountServiceException(String message) {
        super(message);
    }
}
```

```
    }
}
```

File: driver-service/src/main/java/com/ridelink/driver_service/exception/BadRequestException.java

```
package com.ridelink.driver_service.exception;

public class BadRequestException extends RuntimeException {
    public BadRequestException(String message) {
        super(message);
    }
}
```

File: driver-service/src/main/java/com/ridelink/driver_service/exception/DuplicateResourceException.java

```
package com.ridelink.driver_service.exception;

public class DuplicateResourceException extends RuntimeException {
    public DuplicateResourceException(String message) {
        super(message);
    }
}
```

File: driver-service/src/main/java/com/ridelink/driver_service/exception/GlobalExceptionHandler.java

```
package com.ridelink.driver_service.exception;

import com.ridelink.driver_service.dto.ErrorResponse;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidationExceptions(
        MethodArgumentNotValidException ex, HttpServletRequest request) {

        Map<String, String> errors = new HashMap<>();
        for (FieldError error : ex.getBindingResult().getFieldErrors()) {
            errors.put(error.getField(), error.getDefaultMessage());
        }

        ErrorResponse response = ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(HttpStatus.BAD_REQUEST.value())
            .error("Validation Error")
            .message("Input validation failed")
            .build();
    }
}
```

```

        .path(request.getRequestURI())
        .validationErrors(errors)
        .build();

    return new ResponseEntity<>(response, HttpStatus.BAD_REQUEST);
}

@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<ErrorResponse> handleResourceNotFoundException(
    ResourceNotFoundException ex, HttpServletRequest request) {

    ErrorResponse response = ErrorResponse.builder()
        .timestamp(LocalDateTime.now())
        .status(HttpStatus.NOT_FOUND.value())
        .error("Not Found")
        .message(ex.getMessage())
        .path(request.getRequestURI())
        .build();

    return new ResponseEntity<>(response, HttpStatus.NOT_FOUND);
}

@ExceptionHandler(DuplicateResourceException.class)
public ResponseEntity<ErrorResponse> handleDuplicateResourceException(
    DuplicateResourceException ex, HttpServletRequest request) {

    ErrorResponse response = ErrorResponse.builder()
        .timestamp(LocalDateTime.now())
        .status(HttpStatus.CONFLICT.value())
        .error("Conflict")
        .message(ex.getMessage())
        .path(request.getRequestURI())
        .build();

    return new ResponseEntity<>(response, HttpStatus.CONFLICT);
}

@ExceptionHandler(AccountServiceException.class)
public ResponseEntity<ErrorResponse> handleAccountServiceException(
    AccountServiceException ex, HttpServletRequest request) {

    ErrorResponse response = ErrorResponse.builder()
        .timestamp(LocalDateTime.now())
        .status(HttpStatus.BAD_REQUEST.value())
        .error("Account Verification Error")
        .message(ex.getMessage())
        .path(request.getRequestURI())
        .build();

    return new ResponseEntity<>(response, HttpStatus.BAD_REQUEST);
}

@ExceptionHandler(BadRequestException.class)
public ResponseEntity<ErrorResponse> handleBadRequestException(
    BadRequestException ex, HttpServletRequest request) {

    ErrorResponse response = ErrorResponse.builder()
        .timestamp(LocalDateTime.now())
        .status(HttpStatus.BAD_REQUEST.value())
        .error("Bad Request")
        .message(ex.getMessage())
        .path(request.getRequestURI())
        .build();

    return new ResponseEntity<>(response, HttpStatus.BAD_REQUEST);
}

```

```

@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleGeneralException(
    Exception ex, HttpServletRequest request) {

    ErrorResponse response = ErrorResponse.builder()
        .timestamp(LocalDateTime.now())
        .status(HttpStatus.INTERNAL_SERVER_ERROR.value())
        .error("Internal Server Error")
        .message(ex.getMessage() != null ? ex.getMessage() : "An unexpected error occurred")
        .path(request.getRequestURI())
        .build();

    return new ResponseEntity<>(response, HttpStatus.INTERNAL_SERVER_ERROR);
}
}

```

File: driver-service/src/main/java/com/ridelink/driver_service/exception/ResourceNotFoundException.java

```

package com.ridelink.driver_service.exception;

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}

```

File: driver-service/src/main/java/com/ridelink/driver_service/repository/DriverRepository.java

```

package com.ridelink.driver_service.repository;

import com.ridelink.driver_service.entity.AvailabilityStatus;
import com.ridelink.driver_service.entity.Driver;
import com.ridelink.driver_service.entity.VehicleType;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface DriverRepository extends MongoRepository<Driver, String> {

    Optional<Driver> findById(String userId);

    Optional<Driver> findByLicenseNumber(String licenseNumber);

    boolean existsByUserId(String userId);

    boolean existsByLicenseNumber(String licenseNumber);

    List<Driver> findByStatus(AvailabilityStatus status);

    List<Driver> findByStatusAndLocation_ServiceAreaIgnoreCase(AvailabilityStatus status, String serviceArea);

    List<Driver> findByStatusAndLocation_ServiceAreaIgnoreCaseAndVehicle_VehicleType(
        AvailabilityStatus status, String serviceArea, VehicleType vehicleType);
}

```

**File: driver-
service/src/main/java/com/ridelink/driver_service/security/JwtAuthenticationFilter.java**

```
package com.ridelink.driver_service.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.NonNull;
import lombok.RequiredArgsConstructor;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.Collections;

@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;

    @Override
    protected void doFilterInternal(
        @NonNull HttpServletRequest request,
        @NonNull HttpServletResponse response,
        @NonNull FilterChain filterChain
    ) throws ServletException, IOException {

        final String authHeader = request.getHeader("Authorization");
        final String jwt;
        final String userEmail;

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        jwt = authHeader.substring(7);

        try {
            userEmail = jwtService.extractEmail(jwt);

            if (userEmail != null && SecurityContextHolder.getContext().getAuthentication() == null) {
                if (jwtService.isTokenValid(jwt, userEmail)) {
                    String role = jwtService.extractRole(jwt);
                    SimpleGrantedAuthority authority = new SimpleGrantedAuthority("ROLE_" + role);

                    UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(
                        userEmail,
                        null,
                        Collections.singletonList(authority)
                    );
                    authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                    SecurityContextHolder.getContext().setAuthentication(authToken);
                }
            }
        } catch (Exception ex) {
            // Ignore parse errors, let filterChain proceed
        }
    }
}
```

```

        filterChain.doFilter(request, response);
    }
}

```

File: driver-service/src/main/java/com/ridelink/driver_service/security/JwtService.java

```

package com.ridelink.driver_service.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;
import java.util.function.Function;

@Service
public class JwtService {

    @Value("${jwt.secret:mySuperSecretKeyForRideLinkJWTAuthentication2026VerySecureKeyRideLink2026}")
    private String secretKey;

    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(secretKey.getBytes(StandardCharsets.UTF_8));
    }

    public String extractEmail(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public String extractRole(String token) {
        Claims claims = extractAllClaims(token);
        return claims.get("role", String.class);
    }

    public String extractUserId(String token) {
        Claims claims = extractAllClaims(token);
        return claims.get("userId", String.class);
    }

    public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }

    public Claims extractAllClaims(String token) {
        return Jwts.parser()
            .verifyWith(getSigningKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }

    public boolean isTokenValid(String token, String email) {
        final String extractedEmail = extractEmail(token);
        return (extractedEmail.equals(email)) && !isTokenExpired(token);
    }

    public boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }
}

```

```

    }

    private Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }
}

```

File: driver-service/src/main/java/com/ridelink/driver_service/service/DriverService.java

```

package com.ridelink.driver_service.service;

import com.ridelink.driver_service.client.AccountServiceClient;
import com.ridelink.driver_service.dto.*;
import com.ridelink.driver_service.entity.*;
import com.ridelink.driver_service.exception.DuplicateResourceException;
import com.ridelink.driver_service.exception.ResourceNotFoundException;
import com.ridelink.driver_service.repository.DriverRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.util.List;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
@Slf4j
public class DriverService {

    private final DriverRepository driverRepository;
    private final AccountServiceClient accountServiceClient;

    public DriverResponse registerDriver(DriverRegistrationRequest request) {
        if (driverRepository.existsByUserId(request.getUserId())) {
            throw new DuplicateResourceException("Driver profile already exists for user ID: " +
request.getUserId());
        }

        if (driverRepository.existsByLicenseNumber(request.getLicenseNumber())) {
            throw new DuplicateResourceException("Driver license number is already registered: " +
request.getLicenseNumber());
        }

        // Interservice validation: Check if user exists and is ACTIVE with DRIVER role
        AccountUserValidationDto accountUser = accountServiceClient.validateDriverUser(request.getUserId());
        log.info("Driver user verified with Account Service: {} ({})", accountUser.getFullName(),
accountUser.getEmail());

        Vehicle vehicle = Vehicle.builder()
            .make(request.getVehicleMake().trim())
            .model(request.getVehicleModel().trim())
            .year(request.getVehicleYear())
            .licensePlate(request.getLicensePlate().trim().toUpperCase())
            .vehicleType(request.getVehicleType())
            .capacity(request.getVehicleCapacity())
            .build();

        Location location = Location.builder()
            .latitude(request.getLatitude())
            .longitude(request.getLongitude())
            .serviceArea(request.getServiceArea().trim())
            .lastUpdated(LocalDateTime.now())
            .build();
    }
}

```

```
        Driver driver = Driver.builder()
            .userId(request.getUserId())
            .driverName(request.getDriverName().trim())
            .phoneNumber(request.getPhoneNumber().trim())
            .licenseNumber(request.getLicenseNumber().trim().toUpperCase())
            .experienceYears(request.getExperienceYears())
            .rating(5.0)
            .totalTrips(0)
            .vehicle(vehicle)
            .location(location)
            .status(AvailabilityStatus.OFFLINE)
            .createdAt(LocalDateTime.now())
            .updatedAt(LocalDateTime.now())
            .build();

        Driver saved = driverRepository.save(driver);
        return DriverResponse.fromDriver(saved);
    }

    public DriverResponse getDriverById(String id) {
        Driver driver = driverRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Driver not found with id: " + id));
        return DriverResponse.fromDriver(driver);
    }

    public DriverResponse getDriverByUserId(String userId) {
        Driver driver = driverRepository.findById(userId)
            .orElseThrow(() -> new ResourceNotFoundException("Driver not found with user ID: " + userId));
        return DriverResponse.fromDriver(driver);
    }

    public DriverResponse updateVehicle(String driverId, UpdateVehicleRequest request) {
        Driver driver = driverRepository.findById(driverId)
            .orElseThrow(() -> new ResourceNotFoundException("Driver not found with id: " + driverId));

        Vehicle vehicle = Vehicle.builder()
            .make(request.getMake().trim())
            .model(request.getModel().trim())
            .year(request.getYear())
            .licensePlate(request.getLicensePlate().trim().toUpperCase())
            .vehicleType(request.getVehicleType())
            .capacity(request.getCapacity())
            .build();

        driver.setVehicle(vehicle);
        driver.setUpdatedAt(LocalDateTime.now());

        Driver updated = driverRepository.save(driver);
        return DriverResponse.fromDriver(updated);
    }

    public DriverResponse updateAvailability(String driverId, AvailabilityStatus status) {
        Driver driver = driverRepository.findById(driverId)
            .orElseThrow(() -> new ResourceNotFoundException("Driver not found with id: " + driverId));

        driver.setStatus(status);
        driver.setUpdatedAt(LocalDateTime.now());

        Driver updated = driverRepository.save(driver);
        return DriverResponse.fromDriver(updated);
    }

    public DriverResponse updateLocation(String driverId, UpdateLocationRequest request) {
        Driver driver = driverRepository.findById(driverId)
            .orElseThrow(() -> new ResourceNotFoundException("Driver not found with id: " + driverId));
```

```

        Location location = Location.builder()
            .latitude(request.getLatitude())
            .longitude(request.getLongitude())
            .serviceArea(request.getServiceArea().trim())
            .lastUpdated(LocalDateTime.now())
            .build();

        driver.setLocation(location);
        driver.setUpdatedAt(LocalDateTime.now());

        Driver updated = driverRepository.save(driver);
        return DriverResponse.fromDriver(updated);
    }

    public DriverResponse updateTripStatus(String driverId, boolean onTrip) {
        Driver driver = driverRepository.findById(driverId)
            .orElseThrow(() -> new ResourceNotFoundException("Driver not found with id: " + driverId));

        if (onTrip) {
            driver.setStatus(AvailabilityStatus.ON_TRIP);
        } else {
            driver.setStatus(AvailabilityStatus.ONLINE);
            driver.setTotalTrips(driver.getTotalTrips() + 1);
        }
        driver.setUpdatedAt(LocalDateTime.now());

        Driver updated = driverRepository.save(driver);
        return DriverResponse.fromDriver(updated);
    }

    public List<DriverResponse> getAvailableDrivers(String serviceArea, VehicleType vehicleType) {
        List<Driver> drivers;

        if (serviceArea != null && !serviceArea.isBlank() && vehicleType != null) {
            drivers = driverRepository.findByStatusAndLocation_ServiceAreaIgnoreCaseAndVehicle_VehicleType(
                AvailabilityStatus.ONLINE, serviceArea.trim(), vehicleType);
        } else if (serviceArea != null && !serviceArea.isBlank()) {
            drivers = driverRepository.findByStatusAndLocation_ServiceAreaIgnoreCase(
                AvailabilityStatus.ONLINE, serviceArea.trim());
        } else {
            drivers = driverRepository.findByStatus(AvailabilityStatus.ONLINE);
        }

        return drivers.stream()
            .map(DriverResponse::fromDriver)
            .collect(Collectors.toList());
    }
}

```

File: driver-service/src/main/java/com/ridelink/driver_service/TestController.java

```

package com.ridelink.driver_service;

import com.ridelink.driver_service.entity.Driver;
import com.ridelink.driver_service.repository.DriverRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RestController;

import java.time.LocalDateTime;

@RestController
public class TestController {

```

```

@Autowired
private DriverRepository driverRepository;

@GetMapping("/test/db")
public String testDatabase() {
    try {
        long count = driverRepository.count();
        return "✅ Driver Database connection successful! Driver count: " + count;
    } catch (Exception e) {
        return "❌ Driver Database connection failed: " + e.getMessage();
    }
}

@PostMapping("/test/create-sample-driver")
public String createSampleDriver() {
    try {
        if (driverRepository.count() > 0) {
            return "✅ Sample driver already exists in database!";
        }

        // Note: This creates a minimal driver for DB testing only
        // Real drivers need proper validation and user reference
        Driver sampleDriver = Driver.builder()
            .userId("sample-user-id")
            .licenseNumber("TEST-LICENSE-123")
            .rating(5.0)
            .totalTrips(0)
            .createdAt(LocalDateTime.now())
            .updatedAt(LocalDateTime.now())
            .build();

        Driver saved = driverRepository.save(sampleDriver);
        return "✅ Sample driver created! ID: " + saved.getId() + " - ridelink_driver_db should now be visible!";
    } catch (Exception e) {
        return "❌ Failed to create driver: " + e.getMessage();
    }
}
}

```

File: driver-service/src/test/java/com/ridelink/driver_service/controller/DriverControllerTest.java

```

package com.ridelink.driver_service.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.ridelink.driver_service.dto.*;
import com.ridelink.driver_service.entity.*;
import com.ridelink.driver_service.exception.DuplicateResourceException;
import com.ridelink.driver_service.exception.GlobalExceptionHandler;
import com.ridelink.driver_service.exception.ResourceNotFoundException;
import com.ridelink.driver_service.service.DriverService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.setup.MockMvcBuilders;

import java.time.LocalDateTime;
import java.util.Collections;

```

```

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@ExtendWith(MockitoExtension.class)
class DriverControllerTest {

    private MockMvc mockMvc;
    private ObjectMapper objectMapper;

    @Mock
    private DriverService driverService;

    @InjectMocks
    private DriverController driverController;

    private DriverResponse sampleResponse;

    @BeforeEach
    void setUp() {
        objectMapper = new ObjectMapper();
        objectMapper.findAndRegisterModules();
        mockMvc = MockMvcBuilders.standaloneSetup(driverController)
            .setControllerAdvice(new GlobalExceptionHandler())
            .build();

        Vehicle vehicle = Vehicle.builder()
            .make("Toyota")
            .model("Corolla")
            .year(2022)
            .licensePlate("CAB-1234")
            .vehicleType(VehicleType.CAR)
            .capacity(4)
            .build();

        Location location = Location.builder()
            .latitude(6.9271)
            .longitude(79.8612)
            .serviceArea("Colombo")
            .lastUpdated(LocalDateTime.now())
            .build();

        sampleResponse = DriverResponse.builder()
            .id("d1")
            .userId("u1")
            .driverName("Sunil Silva")
            .phoneNumber("+94719876543")
            .licenseNumber("DL-12345")
            .experienceYears(5)
            .rating(5.0)
            .totalTrips(0)
            .vehicle(vehicle)
            .location(location)
            .status(AvailabilityStatus.OFFLINE)
            .createdAt(LocalDateTime.now())
            .updatedAt(LocalDateTime.now())
            .build();
    }

    @Test
    void registerDriver_ValidRequest_ReturnsCreated() throws Exception {
        DriverRegistrationRequest request = DriverRegistrationRequest.builder()
            .userId("u1")
            .driverName("Sunil Silva")
            .phoneNumber("+94719876543")

```

```

        .licenseNumber("DL-12345")
        .experienceYears(5)
        .vehicleMake("Toyota")
        .vehicleModel("Corolla")
        .vehicleYear(2022)
        .licensePlate("CAB-1234")
        .vehicleType(VehicleType.CAR)
        .vehicleCapacity(4)
        .latitude(6.9271)
        .longitude(79.8612)
        .serviceArea("Colombo")
        .build();

when(driverService.registerDriver(any(DriverRegistrationRequest.class))).thenReturn(sampleResponse);

mockMvc.perform(post("/api/drivers")
    .contentType(MediaType.APPLICATION_JSON)
    .content(objectMapper.writeValueAsString(request)))
    .andExpect(status().isCreated())
    .andExpect(jsonPath("$.id").value("d1"))
    .andExpect(jsonPath("$.driverName").value("Sunil Silva"))
    .andExpect(jsonPath("$.status").value("OFFLINE"));
}

@Test
void registerDriver_MissingFields_ReturnsBadRequest() throws Exception {
    DriverRegistrationRequest request = DriverRegistrationRequest.builder()
        .userId("")
        .driverName("")
        .build();

    mockMvc.perform(post("/api/drivers")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(request)))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.status").value(400));
}

@Test
void registerDriver_DuplicateUserId_ReturnsConflict() throws Exception {
    DriverRegistrationRequest request = DriverRegistrationRequest.builder()
        .userId("u1")
        .driverName("Sunil Silva")
        .phoneNumber("+94719876543")
        .licenseNumber("DL-12345")
        .experienceYears(5)
        .vehicleMake("Toyota")
        .vehicleModel("Corolla")
        .vehicleYear(2022)
        .licensePlate("CAB-1234")
        .vehicleType(VehicleType.CAR)
        .vehicleCapacity(4)
        .serviceArea("Colombo")
        .build();

    when(driverService.registerDriver(any(DriverRegistrationRequest.class)))
        .thenThrow(new DuplicateResourceException("Driver profile already exists"));

    mockMvc.perform(post("/api/drivers")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(request)))
        .andExpect(status().isConflict())
        .andExpect(jsonPath("$.status").value(409));
}

@Test
void getDriverById_Success() throws Exception {

```

```

        when(driverService.getDriverById("d1")).thenReturn(sampleResponse);

        mockMvc.perform(get("/api/drivers/d1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value("d1"))
            .andExpect(jsonPath("$.driverName").value("Sunil Silva"));
    }

    @Test
    void getDriverById_NotFound_Returns404() throws Exception {
        when(driverService.getDriverById("nonexistent"))
            .thenReturn(new ResourceNotFoundException("Driver not found"));

        mockMvc.perform(get("/api/drivers/nonexistent"))
            .andExpect(status().isNotFound())
            .andExpect(jsonPath("$.status").value(404));
    }

    @Test
    void getDriverByUserId_Success() throws Exception {
        when(driverService.getDriverByUserId("u1")).thenReturn(sampleResponse);

        mockMvc.perform(get("/api/drivers/user/u1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.userId").value("u1"));
    }

    @Test
    void updateAvailability_ToOnline_ReturnsOk() throws Exception {
        sampleResponse.setStatus(AvailabilityStatus.ONLINE);

        UpdateAvailabilityRequest request = UpdateAvailabilityRequest.builder()
            .status(AvailabilityStatus.ONLINE)
            .build();

        when(driverService.updateAvailability("d1", AvailabilityStatus.ONLINE)).thenReturn(sampleResponse);

        mockMvc.perform(patch("/api/drivers/d1/availability")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.status").value("ONLINE"));
    }

    @Test
    void getAvailableDrivers_FilteredByServiceArea_ReturnsList() throws Exception {
        when(driverService.getAvailableDrivers("Colombo", null))
            .thenReturn(Collections.singletonList(sampleResponse));

        mockMvc.perform(get("/api/drivers/available")
            .param("serviceArea", "Colombo"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.length()").value(1));
    }
}

```

File: driver-service/src/test/java/com/ridelink/driver_service/DriverServiceApplicationTests.java

```

package com.ridelink.driver_service;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.TestPropertySource;

```

```

@SpringBootTest
@TestPropertySource(properties = {
    "spring.data.mongodb.uri=mongodb://localhost:27017/ridelink_driver_test_db",
    "services.account-service.url=http://localhost:8081"
})
class DriverServiceApplicationTests {

    @Test
    void contextLoads() {
    }

}

```

File: driver-service/src/test/java/com/ridelink/driver_service/service/DriverServiceTest.java

```

package com.ridelink.driver_service.service;

import com.ridelink.driver_service.client.AccountServiceClient;
import com.ridelink.driver_service.dto.*;
import com.ridelink.driver_service.entity.*;
import com.ridelink.driver_service.exception.DuplicateResourceException;
import com.ridelink.driver_service.exception.ResourceNotFoundException;
import com.ridelink.driver_service.repository.DriverRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.time.LocalDateTime;
import java.util.Collections;
import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class DriverServiceTest {

    @Mock
    private DriverRepository driverRepository;

    @Mock
    private AccountServiceClient accountServiceClient;

    @InjectMocks
    private DriverService driverService;

    private Driver sampleDriver;
    private DriverRegistrationRequest sampleRequest;
    private AccountUserValidationDto sampleAccountUser;

    @BeforeEach
    void setUp() {
        Vehicle vehicle = Vehicle.builder()
            .make("Toyota")
            .model("Corolla")
            .year(2022)
            .licensePlate("CAB-1234")
            .vehicleType(VehicleType.CAR)
            .capacity(4)
            .build();
    }

```

```

        Location location = Location.builder()
            .latitude(6.9271)
            .longitude(79.8612)
            .serviceArea("Colombo")
            .lastUpdated(LocalDate.now())
            .build();

        sampleDriver = Driver.builder()
            .id("d1")
            .userId("u1")
            .driverName("Sunil Silva")
            .phoneNumber("+94719876543")
            .licenseNumber("DL-12345")
            .experienceYears(5)
            .rating(5.0)
            .totalTrips(0)
            .vehicle(vehicle)
            .location(location)
            .status(AvailabilityStatus.OFFLINE)
            .createdAt(LocalDate.now())
            .updatedAt(LocalDate.now())
            .build();

        sampleRequest = DriverRegistrationRequest.builder()
            .userId("u1")
            .driverName("Sunil Silva")
            .phoneNumber("+94719876543")
            .licenseNumber("DL-12345")
            .experienceYears(5)
            .vehicleMake("Toyota")
            .vehicleModel("Corolla")
            .vehicleYear(2022)
            .licensePlate("CAB-1234")
            .vehicleType(VehicleType.CAR)
            .vehicleCapacity(4)
            .latitude(6.9271)
            .longitude(79.8612)
            .serviceArea("Colombo")
            .build();

        sampleAccountUser = AccountUserValidationDto.builder()
            .id("u1")
            .fullName("Sunil Silva")
            .email("sunil@example.com")
            .role("DRIVER")
            .status("ACTIVE")
            .build();
    }

    @Test
    void registerDriver_Success() {
        when(driverRepository.existsByUserId("u1")).thenReturn(false);
        when(driverRepository.existsByLicenseNumber("DL-12345")).thenReturn(false);
        when(accountServiceClient.validateDriverUser("u1")).thenReturn(sampleAccountUser);
        when(driverRepository.save(any(Driver.class))).thenReturn(sampleDriver);

        DriverResponse response = driverService.registerDriver(sampleRequest);

        assertNotNull(response);
        assertEquals("u1", response.getUserId());
        assertEquals("Sunil Silva", response.getDriverName());
        assertEquals(AvailabilityStatus.OFFLINE, response.getStatus());
        verify(accountServiceClient, times(1)).validateDriverUser("u1");
        verify(driverRepository, times(1)).save(any(Driver.class));
    }

```

```
@Test
void registerDriver_DuplicateUserId_ThrowsException() {
    when(driverRepository.existsByUserId("u1")).thenReturn(true);

    assertThrows(DuplicateResourceException.class, () -> driverService.registerDriver(sampleRequest));
    verify(driverRepository, never()).save(any(Driver.class));
}

@Test
void registerDriver_DuplicateLicense_ThrowsException() {
    when(driverRepository.existsByUserId("u1")).thenReturn(false);
    when(driverRepository.existsByLicenseNumber("DL-12345")).thenReturn(true);

    assertThrows(DuplicateResourceException.class, () -> driverService.registerDriver(sampleRequest));
    verify(accountServiceClient, never()).validateDriverUser(any());
    verify(driverRepository, never()).save(any(Driver.class));
}

@Test
void getDriverById_Success() {
    when(driverRepository.findById("d1")).thenReturn(Optional.of(sampleDriver));

    DriverResponse response = driverService.getDriverById("d1");

    assertNotNull(response);
    assertEquals("d1", response.getId());
    assertEquals("Sunil Silva", response.getDriverName());
}

@Test
void getDriverById_NotFound_ThrowsException() {
    when(driverRepository.findById("nonexistent")).thenReturn(Optional.empty());

    assertThrows(ResourceNotFoundException.class, () -> driverService.getDriverById("nonexistent"));
}

@Test
void getDriverByUserId_Success() {
    when(driverRepository.findByIdByUserId("u1")).thenReturn(Optional.of(sampleDriver));

    DriverResponse response = driverService.getDriverByUserId("u1");

    assertNotNull(response);
    assertEquals("u1", response.getUserId());
}

@Test
void updateAvailability_ToOnline_Success() {
    when(driverRepository.findById("d1")).thenReturn(Optional.of(sampleDriver));
    when(driverRepository.save(any(Driver.class))).thenReturn(sampleDriver);

    DriverResponse response = driverService.updateAvailability("d1", AvailabilityStatus.ONLINE);

    assertNotNull(response);
    assertEquals(AvailabilityStatus.ONLINE, sampleDriver.getStatus());
    verify(driverRepository, times(1)).save(sampleDriver);
}

@Test
void updateLocation_Success() {
    UpdateLocationRequest request = UpdateLocationRequest.builder()
        .latitude(7.2906)
        .longitude(80.6337)
        .serviceArea("Kandy")
        .build();

    when(driverRepository.findById("d1")).thenReturn(Optional.of(sampleDriver));
```

```

        when(driverRepository.save(any(Driver.class))).thenReturn(sampleDriver);

        DriverResponse response = driverService.updateLocation("d1", request);

        assertNotNull(response);
        assertEquals("Kandy", sampleDriver.getLocation().getServiceArea());
        verify(driverRepository, times(1)).save(sampleDriver);
    }

    @Test
    void updateTripStatus_StartTrip_SetsOnTrip() {
        sampleDriver.setStatus(AvailabilityStatus.ONLINE);
        when(driverRepository.findById("d1")).thenReturn(Optional.of(sampleDriver));
        when(driverRepository.save(any(Driver.class))).thenReturn(sampleDriver);

        driverService.updateTripStatus("d1", true);

        assertEquals(AvailabilityStatus.ON_TRIP, sampleDriver.getStatus());
    }

    @Test
    void updateTripStatus_EndTrip_SetsOnlineAndIncrementsTrips() {
        sampleDriver.setStatus(AvailabilityStatus.ON_TRIP);
        when(driverRepository.findById("d1")).thenReturn(Optional.of(sampleDriver));
        when(driverRepository.save(any(Driver.class))).thenReturn(sampleDriver);

        driverService.updateTripStatus("d1", false);

        assertEquals(AvailabilityStatus.ONLINE, sampleDriver.getStatus());
        assertEquals(1, sampleDriver.getTotalTrips());
    }

    @Test
    void getAvailableDrivers_ByServiceArea_Success() {
        when(driverRepository.findByStatusAndLocation_ServiceAreaIgnoreCase(
            AvailabilityStatus.ONLINE, "Colombo")).thenReturn(Collections.singletonList(sampleDriver));

        List<DriverResponse> result = driverService.getAvailableDrivers("Colombo", null);

        assertEquals(1, result.size());
        assertEquals("Sunil Silva", result.get(0).getDriverName());
    }

    @Test
    void getAvailableDrivers_ByServiceAreaAndVehicleType_Success() {
        when(driverRepository.findByStatusAndLocation_ServiceAreaIgnoreCaseAndVehicle_VehicleType(
            AvailabilityStatus.ONLINE, "Colombo", VehicleType.CAR))
            .thenReturn(Collections.singletonList(sampleDriver));

        List<DriverResponse> result = driverService.getAvailableDrivers("Colombo", VehicleType.CAR);

        assertEquals(1, result.size());
        assertEquals(VehicleType.CAR, result.get(0).getVehicle().getVehicleType());
    }

    @Test
    void getAvailableDrivers_All_Success() {
        when(driverRepository.findByStatus(AvailabilityStatus.ONLINE))
            .thenReturn(Collections.singletonList(sampleDriver));

        List<DriverResponse> result = driverService.getAvailableDrivers(null, null);

        assertEquals(1, result.size());
    }
}

```

B.3 Ride Management Service (Member 3 - Port 8083)

Total Source Files: 33

File: ride-service/src/main/java/com/ridelink/ride_service/client/AccountServiceClient.java

```
package com.ridelink.ride_service.client;

import com.ridelink.ride_service.dto.PassengerValidationDto;
import com.ridelink.ride_service.exception.AccountServiceException;
import com.ridelink.ride_service.exception.BadRequestException;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;

@Slf4j
@Component
public class AccountServiceClient {

    private final RestClient restClient;
    private final String accountServiceUrl;

    public AccountServiceClient(RestClient restClient,
                               @Value("${services.account-service.url:http://localhost:8081}") String
accountServiceUrl) {
        this.restClient = restClient;
        this.accountServiceUrl = accountServiceUrl;
    }

    public PassengerValidationDto validatePassenger(String userId) {
        String url = accountServiceUrl + "/api/users/" + userId + "/validate";
        log.info("Calling Account Service to validate passenger at: {}", url);

        try {
            PassengerValidationDto passenger = restClient.get()
                .uri(url)
                .retrieve()
                .onStatus(HttpStatus::is4xxClientError, (req, res) -> {
                    log.error("Account Service returned client error for passenger ID: {}", userId);
                    throw new BadRequestException("Passenger validation failed: Account not found or
invalid in Account Service");
                })
                .onStatus(HttpStatus::is5xxServerError, (req, res) -> {
                    log.error("Account Service returned 5xx server error for passenger ID: {}", userId);
                    throw new AccountServiceException("Account Service is currently unavailable");
                })
                .body(PassengerValidationDto.class);

            if (passenger == null) {
                throw new BadRequestException("Passenger could not be verified in Account Service");
            }

            if (!"ACTIVE".equalsIgnoreCase(passenger.getStatus())) {
                throw new BadRequestException("Passenger account is not ACTIVE. Current status: " +
passenger.getStatus());
            }

            if (!"PASSENGER".equalsIgnoreCase(passenger.getRole()) &&
!"ADMIN".equalsIgnoreCase(passenger.getRole())) {

```

```

        throw new BadRequestException("User role must be PASSENGER to request a ride. Current role: " +
passenger.getRole());
    }

    return passenger;
} catch (BadRequestException e) {
    throw e;
} catch (Exception e) {
    log.error("Failed to connect to Account Service: {}", e.getMessage());
    throw new AccountServiceException("Unable to contact Account Service: " + e.getMessage());
}
}
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/client/DriverServiceClient.java

```

package com.ridelink.ride_service.client;

import com.ridelink.ride_service.dto.DriverSummaryDto;
import com.ridelink.ride_service.entity.VehicleType;
import com.ridelink.ride_service.exception.DriverServiceException;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.core.ParameterizedTypeReference;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;

import java.util.Collections;
import java.util.List;
import java.util.Map;

@Slf4j
@Component
public class DriverServiceClient {

    private final RestClient restClient;
    private final String driverServiceUrl;

    public DriverServiceClient(RestClient restClient,
                              @Value("${services.driver-service.url:http://localhost:8082}") String
driverServiceUrl) {
        this.restClient = restClient;
        this.driverServiceUrl = driverServiceUrl;
    }

    public List<DriverSummaryDto> getAvailableDrivers(String serviceArea, VehicleType vehicleType) {
        String url = driverServiceUrl + "/api/drivers/available?serviceArea=" + serviceArea;
        if (vehicleType != null) {
            url += "&vehicleType=" + vehicleType.name();
        }
        log.info("Calling Driver Service to find available drivers at: {}", url);

        try {
            List<DriverSummaryDto> drivers = restClient.get()
                .uri(url)
                .retrieve()
                .onStatus(HttpStatus::is5xxServerError, (req, res) -> {
                    log.error("Driver Service returned 5xx server error when fetching available drivers");
                    throw new DriverServiceException("Driver Service is currently unavailable");
                })
                .body(new ParameterizedTypeReference<List<DriverSummaryDto>>() {});

```

```

        return drivers != null ? drivers : Collections.emptyList();
    } catch (DriverServiceException e) {
        throw e;
    } catch (Exception e) {
        log.error("Failed to connect to Driver Service: {}", e.getMessage());
        throw new DriverServiceException("Unable to contact Driver Service: " + e.getMessage());
    }
}

public void updateDriverTripStatus(String driverId, boolean onTrip) {
    String url = driverServiceUrl + "/api/drivers/" + driverId + "/trip-status";
    log.info("Updating trip status for driver {} to onTrip={} at {}", driverId, onTrip, url);

    try {
        restClient.patch()
            .uri(url)
            .contentType(MediaType.APPLICATION_JSON)
            .body(Map.of("onTrip", onTrip))
            .retrieve()
            .onStatus(HttpStatusCode::isError, (req, res) -> {
                log.error("Failed to update trip status for driver {} in Driver Service", driverId);
                throw new DriverServiceException("Failed to update driver trip status in Driver
Service");
            })
            .toBodilessEntity();
        log.info("Successfully updated trip status for driver {}", driverId);
    } catch (Exception e) {
        log.warn("Could not update trip status in Driver Service for driver {}: {}", driverId,
e.getMessage());
    }
}
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/config/OpenApiConfig.java

```

package com.ridelink.ride_service.config;

import io.swagger.v3.oas.models.Components;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Contact;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.info.License;
import io.swagger.v3.oas.models.security.SecurityRequirement;
import io.swagger.v3.oas.models.security.SecurityScheme;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI rideServiceOpenAPI() {
        final String securitySchemeName = "bearerAuth";
        return new OpenAPI()
            .info(new Info()
                .title("RideLink - Ride Management Service API")
                .description("Microservice responsible for Ride Requests, Driver Assignment, Ride
Lifecycle State Machine, and Interservice Coordination for IT3130 - Application Development (Member 3).")
                .version("1.0.0")
                .contact(new Contact().name("RideLink Backend Team").email("support@ridelink.com")))
            .license(new License().name("Apache 2.0").url("https://springdoc.org")))
            .addSecurityItem(new SecurityRequirement().addList(securitySchemeName))
            .components(new Components()
                .addSecuritySchemes(securitySchemeName,

```

```

        new SecurityScheme()
            .name(securitySchemeName)
            .type(SecurityScheme.Type.HTTP)
            .scheme("bearer")
            .bearerFormat("JWT"));
    }
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/config/RestClientConfig.java

```

package com.ridelink.ride_service.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.client.SimpleClientHttpRequestFactory;
import org.springframework.web.client.RestClient;

import java.time.Duration;

@Configuration
public class RestClientConfig {

    @Bean
    public RestClient restClient() {
        SimpleClientHttpRequestFactory factory = new SimpleClientHttpRequestFactory();
        factory.setConnectTimeout(Duration.ofSeconds(5));
        factory.setReadTimeout(Duration.ofSeconds(5));

        return RestClient.builder()
            .requestFactory(factory)
            .build();
    }
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/config/SecurityConfig.java

```

package com.ridelink.ride_service.config;

import com.ridelink.ride_service.security.JwtAuthenticationFilter;
import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)

```

```

        .cors(AbstractHttpConfigurer::disable)
        .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            // Public endpoints
            .requestMatchers(
                "/swagger-ui/**",
                "/swagger-ui.html",
                "/v3/api-docs/**",
                "/api-docs/**",
                "/actuator/**"
            ).permitAll()
            // All Ride endpoints permitted by default or secured via tokens in production
            .requestMatchers("/api/rides/**").permitAll()
            .anyRequest().authenticated()
        )
        .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
}
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/controller/RideController.java

```

package com.ridelink.ride_service.controller;

import com.ridelink.ride_service.dto.CancelRideRequest;
import com.ridelink.ride_service.dto.CreateRideRequest;
import com.ridelink.ride_service.dto.RideResponse;
import com.ridelink.ride_service.entity.RideStatus;
import com.ridelink.ride_service.service.RideService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/rides")
@RequiredArgsConstructor
@Tag(name = "Ride Management", description = "Endpoints for ride requests, driver dispatch, and ride lifecycle management")
public class RideController {

    private final RideService rideService;

    @PostMapping
    @Operation(summary = "Create ride request and auto-assign driver", description = "Validates passenger with Account Service, matches available drivers via Driver Service, and assigns best driver.")
    public ResponseEntity<RideResponse> createRide(@Valid @RequestBody CreateRideRequest request) {
        RideResponse response = rideService.createRide(request);
        return new ResponseEntity<>(response, HttpStatus.CREATED);
    }

    @GetMapping("/{id}")
    @Operation(summary = "Get ride details by ID")
    public ResponseEntity<RideResponse> getRideById(@PathVariable String id) {
        return ResponseEntity.ok(rideService.getRideById(id));
    }

    @GetMapping("/passenger/{passengerId}")

```

```

@Operation(summary = "Get ride history for a specific passenger")
public ResponseEntity<List<RideResponse>> getRidesByPassengerId(@PathVariable String passengerId) {
    return ResponseEntity.ok(rideService.getRidesByPassengerId(passengerId));
}

@GetMapping("/driver/{driverId}")
@Operation(summary = "Get ride history for a specific driver")
public ResponseEntity<List<RideResponse>> getRidesByDriverId(@PathVariable String driverId) {
    return ResponseEntity.ok(rideService.getRidesByDriverId(driverId));
}

@PatchMapping("/{id}/accept")
@Operation(summary = "Driver accepts an assigned ride", description = "Transitions status from ASSIGNED to ACCEPTED.")
public ResponseEntity<RideResponse> acceptRide(@PathVariable String id) {
    return ResponseEntity.ok(rideService.acceptRide(id));
}

@PatchMapping("/{id}/start")
@Operation(summary = "Driver starts the trip", description = "Transitions status from ACCEPTED to IN_PROGRESS.")
public ResponseEntity<RideResponse> startRide(@PathVariable String id) {
    return ResponseEntity.ok(rideService.startRide(id));
}

@PatchMapping("/{id}/complete")
@Operation(summary = "Driver completes the trip", description = "Transitions status from IN_PROGRESS to COMPLETED and releases driver in Driver Service.")
public ResponseEntity<RideResponse> completeRide(@PathVariable String id) {
    return ResponseEntity.ok(rideService.completeRide(id));
}

@PatchMapping("/{id}/cancel")
@Operation(summary = "Cancel a ride", description = "Transitions status to CANCELLED and releases assigned driver.")
public ResponseEntity<RideResponse> cancelRide(
    @PathVariable String id,
    @RequestBody(required = false) CancelRideRequest cancelRequest) {
    String reason = cancelRequest != null ? cancelRequest.getReason() : "Cancelled by user";
    return ResponseEntity.ok(rideService.cancelRide(id, reason));
}

@GetMapping
@Operation(summary = "Get all rides with optional status filter")
public ResponseEntity<List<RideResponse>> getAllRides(@RequestParam(required = false) RideStatus status) {
    return ResponseEntity.ok(rideService.getAllRides(status));
}
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/dto/ApiResponse.java

```

package com.ridelink.ride_service.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor

```

```

public class ApiResponse<T> {
    private boolean success;
    private String message;
    private T data;
    @Builder.Default
    private LocalDateTime timestamp = LocalDateTime.now();

    public static <T> ApiResponse<T> success(String message, T data) {
        return ApiResponse.<T>builder()
            .success(true)
            .message(message)
            .data(data)
            .timestamp(LocalDateTime.now())
            .build();
    }
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/dto/CancelRideRequest.java

```

package com.ridelink.ride_service.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class CancelRideRequest {
    private String reason;
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/dto/CreateRideRequest.java

```

package com.ridelink.ride_service.dto;

import com.ridelink.ride_service.entity.VehicleType;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class CreateRideRequest {

    @NotBlank(message = "Passenger ID is required")
    private String passengerId;

    @NotBlank(message = "Pickup address is required")
    private String pickupAddress;

    @NotNull(message = "Pickup latitude is required")
    private Double pickupLatitude;
}

```

```

    @NotNull(message = "Pickup longitude is required")
    private Double pickupLongitude;

    @NotBlank(message = "Service area is required (e.g. Colombo, Kandy, Galle)")
    private String serviceArea;

    @NotBlank(message = "Dropoff address is required")
    private String dropoffAddress;

    @NotNull(message = "Dropoff latitude is required")
    private Double dropoffLatitude;

    @NotNull(message = "Dropoff longitude is required")
    private Double dropoffLongitude;

    @NotNull(message = "Vehicle type is required (CAR, VAN, BIKE, TUK)")
    private VehicleType vehicleType;

    @NotNull(message = "Estimated distance in km is required")
    @Positive(message = "Estimated distance must be positive")
    private Double estimatedDistanceKm;
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/dto/DriverSummaryDto.java

```

package com.ridelink.ride_service.dto;

import com.ridelink.ride_service.entity.VehicleType;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.NoArgsConstructor;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class DriverSummaryDto {

    private String id;
    private String userId;
    private String driverName;
    private String phoneNumber;
    private Double rating;
    private Integer totalTrips;
    private VehicleSummary vehicle;
    private String status;

    @Data
    @Builder
    @NoArgsConstructor
    @AllArgsConstructor
    public static class VehicleSummary {
        private String make;
        private String model;
        private Integer year;
        private String licensePlate;
        private VehicleType vehicleType;
        private Integer capacity;
    }
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/dto/ErrorResponse.java

```
package com.ridelink.ride_service.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;
import java.util.Map;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class ErrorResponse {
    private int status;
    private String error;
    private String message;
    private String path;
    private Map<String, String> validationErrors;
    @Builder.Default
    private LocalDateTime timestamp = LocalDateTime.now();
}
```

File: ride-service/src/main/java/com/ridelink/ride_service/dto/PassengerValidationDto.java

```
package com.ridelink.ride_service.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class PassengerValidationDto {
    private String id;
    private String fullName;
    private String email;
    private String phoneNumber;
    private String role;
    private String status;
}
```

File: ride-service/src/main/java/com/ridelink/ride_service/dto/RideResponse.java

```
package com.ridelink.ride_service.dto;

import com.ridelink.ride_service.entity.LocationPoint;
import com.ridelink.ride_service.entity.RideStatus;
import com.ridelink.ride_service.entity.VehicleType;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
```

```

import java.time.LocalDateTime;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class RideResponse {

    private String id;
    private String passengerId;
    private String passengerName;
    private String passengerPhone;

    private String driverId;
    private String driverUserId;
    private String driverName;
    private String driverPhone;
    private String vehiclePlate;
    private String vehicleModel;

    private LocationPoint pickupLocation;
    private LocationPoint dropoffLocation;
    private VehicleType vehicleType;
    private Double estimatedDistanceKm;

    private RideStatus status;
    private Double fareAmount;
    private String cancellationReason;

    private LocalDateTime requestedAt;
    private LocalDateTime assignedAt;
    private LocalDateTime acceptedAt;
    private LocalDateTime startedAt;
    private LocalDateTime completedAt;
    private LocalDateTime cancelledAt;
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/entity/LocationPoint.java

```

package com.ridelink.ride_service.entity;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class LocationPoint {
    private String address;
    private Double latitude;
    private Double longitude;
    private String serviceArea;
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/entity/Ride.java

```

package com.ridelink.ride_service.entity;

import lombok.AllArgsConstructor;

```

```

import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.LocalDateTime;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
@Document(collection = "rides")
public class Ride {

    @Id
    private String id;

    // Passenger references (Account Service)
    private String passengerId;
    private String passengerName;
    private String passengerPhone;

    // Driver references (Driver Service & Account Service)
    private String driverId;
    private String driverUserId;
    private String driverName;
    private String driverPhone;
    private String vehiclePlate;
    private String vehicleModel;

    // Journey details
    private LocationPoint pickupLocation;
    private LocationPoint dropoffLocation;
    private VehicleType vehicleType;
    private Double estimatedDistanceKm;

    // Lifecycle
    private RideStatus status;
    private Double fareAmount;
    private String cancellationReason;

    // Timestamps
    private LocalDateTime requestedAt;
    private LocalDateTime assignedAt;
    private LocalDateTime acceptedAt;
    private LocalDateTime startedAt;
    private LocalDateTime completedAt;
    private LocalDateTime cancelledAt;
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/entity/RideStatus.java

```

package com.ridelink.ride_service.entity;

public enum RideStatus {
    REQUESTED,
    ASSIGNED,
    ACCEPTED,
    IN_PROGRESS,
    COMPLETED,
    CANCELLED
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/entity/VehicleType.java

```
package com.ridelink.ride_service.entity;

public enum VehicleType {
    CAR,
    VAN,
    BIKE,
    TUK
}
```

File: ride-service/src/main/java/com/ridelink/ride_service/exception/AccountServiceException.java

```
package com.ridelink.ride_service.exception;

public class AccountServiceException extends RuntimeException {
    public AccountServiceException(String message) {
        super(message);
    }
}
```

File: ride-service/src/main/java/com/ridelink/ride_service/exception/BadRequestException.java

```
package com.ridelink.ride_service.exception;

public class BadRequestException extends RuntimeException {
    public BadRequestException(String message) {
        super(message);
    }
}
```

File: ride-service/src/main/java/com/ridelink/ride_service/exception/DriverServiceException.java

```
package com.ridelink.ride_service.exception;

public class DriverServiceException extends RuntimeException {
    public DriverServiceException(String message) {
        super(message);
    }
}
```

File: ride-service/src/main/java/com/ridelink/ride_service/exception/GlobalExceptionHandler.java

```
package com.ridelink.ride_service.exception;

import com.ridelink.ride_service.dto.ErrorResponse;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
```

```

import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleResourceNotFoundException(ResourceNotFoundException ex,
HttpServletRequest request) {
        ErrorResponse errorResponse = ErrorResponse.builder()
            .status(HttpStatus.NOT_FOUND.value())
            .error("Not Found")
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .timestamp(LocalDateTime.now())
            .build();
        return new ResponseEntity<>(errorResponse, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(InvalidStateTransitionException.class)
    public ResponseEntity<ErrorResponse> handleInvalidStateTransitionException(InvalidStateTransitionException
ex, HttpServletRequest request) {
        ErrorResponse errorResponse = ErrorResponse.builder()
            .status(HttpStatus.BAD_REQUEST.value())
            .error("Bad Request - Invalid State Transition")
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .timestamp(LocalDateTime.now())
            .build();
        return new ResponseEntity<>(errorResponse, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(NoDriverAvailableException.class)
    public ResponseEntity<ErrorResponse> handleNoDriverAvailableException(NoDriverAvailableException ex,
HttpServletRequest request) {
        ErrorResponse errorResponse = ErrorResponse.builder()
            .status(HttpStatus.NOT_FOUND.value())
            .error("No Driver Available")
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .timestamp(LocalDateTime.now())
            .build();
        return new ResponseEntity<>(errorResponse, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(BadRequestException.class)
    public ResponseEntity<ErrorResponse> handleBadRequestException(BadRequestException ex, HttpServletRequest
request) {
        ErrorResponse errorResponse = ErrorResponse.builder()
            .status(HttpStatus.BAD_REQUEST.value())
            .error("Bad Request")
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .timestamp(LocalDateTime.now())
            .build();
        return new ResponseEntity<>(errorResponse, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(AccountServiceException.class)
    public ResponseEntity<ErrorResponse> handleAccountServiceException(AccountServiceException ex,
HttpServletRequest request) {
        ErrorResponse errorResponse = ErrorResponse.builder()
            .status(HttpStatus.BAD_GATEWAY.value())
            .error("Account Service Integration Error")
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .timestamp(LocalDateTime.now())
            .build();
    }
}

```

```

        .build();
        return new ResponseEntity<>(errorResponse, HttpStatus.BAD_GATEWAY);
    }

    @ExceptionHandler(DriverServiceException.class)
    public ResponseEntity<ErrorResponse> handleDriverServiceException(DriverServiceException ex,
        HttpServletRequest request) {
        ErrorResponse errorResponse = ErrorResponse.builder()
            .status(HttpStatus.BAD_GATEWAY.value())
            .error("Driver Service Integration Error")
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .timestamp(LocalDateTime.now())
            .build();
        return new ResponseEntity<>(errorResponse, HttpStatus.BAD_GATEWAY);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidationException(MethodArgumentNotValidException ex,
        HttpServletRequest request) {
        Map<String, String> errors = new HashMap<>();
        for (FieldError fieldError : ex.getBindingResult().getFieldErrors()) {
            errors.put(fieldError.getField(), fieldError.getDefaultMessage());
        }

        ErrorResponse errorResponse = ErrorResponse.builder()
            .status(HttpStatus.BAD_REQUEST.value())
            .error("Validation Failed")
            .message("Input validation failed for one or more fields")
            .path(request.getRequestURI())
            .validationErrors(errors)
            .timestamp(LocalDateTime.now())
            .build();
        return new ResponseEntity<>(errorResponse, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneralException(Exception ex, HttpServletRequest request) {
        ErrorResponse errorResponse = ErrorResponse.builder()
            .status(HttpStatus.INTERNAL_SERVER_ERROR.value())
            .error("Internal Server Error")
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .timestamp(LocalDateTime.now())
            .build();
        return new ResponseEntity<>(errorResponse, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

```

File: ride-
service/src/main/java/com/ridelink/ride_service/exception/InvalidStateTransitionException.java

```

package com.ridelink.ride_service.exception;

public class InvalidStateTransitionException extends RuntimeException {
    public InvalidStateTransitionException(String message) {
        super(message);
    }
}

```

File: ride-
service/src/main/java/com/ridelink/ride_service/exception/NoDriverAvailableException.java

```
package com.ridelink.ride_service.exception;

public class NoDriverAvailableException extends RuntimeException {
    public NoDriverAvailableException(String message) {
        super(message);
    }
}
```

File: ride-service/src/main/java/com/ridelink/ride_service/exception/ResourceNotFoundException.java

```
package com.ridelink.ride_service.exception;

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

File: ride-service/src/main/java/com/ridelink/ride_service/repository/RideRepository.java

```
package com.ridelink.ride_service.repository;

import com.ridelink.ride_service.entity.Ride;
import com.ridelink.ride_service.entity.RideStatus;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface RideRepository extends MongoRepository<Ride, String> {

    List<Ride> findByPassengerIdOrderByRequestedAtDesc(String passengerId);

    List<Ride> findByDriverIdOrderByRequestedAtDesc(String driverId);

    List<Ride> findByStatus(RideStatus status);

    List<Ride> findByPassengerIdAndStatusIn(String passengerId, List<RideStatus> statuses);
}
```

File: ride-service/src/main/java/com/ridelink/ride_service/RideServiceApplication.java

```
package com.ridelink.ride_service;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class RideServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(RideServiceApplication.class, args);
    }
}
```

File: ride-service/src/main/java/com/ridelink/ride_service/security/JwtAuthenticationFilter.java

```
package com.ridelink.ride_service.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.NonNull;
import lombok.RequiredArgsConstructor;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.Collections;
import java.util.List;

@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;

    @Override
    protected void doFilterInternal(
        @NonNull HttpServletRequest request,
        @NonNull HttpServletResponse response,
        @NonNull FilterChain filterChain
    ) throws ServletException, IOException {

        final String authHeader = request.getHeader("Authorization");
        final String jwt;
        final String userEmail;

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        jwt = authHeader.substring(7);
        try {
            userEmail = jwtService.extractUsername(jwt);

            if (userEmail != null && SecurityContextHolder.getContext().getAuthentication() == null) {
                if (jwtService.isTokenValid(jwt)) {
                    String role = jwtService.extractRole(jwt);
                    List<SimpleGrantedAuthority> authorities = role != null
                        ? List.of(new SimpleGrantedAuthority("ROLE_" + role.toUpperCase()))
                        : Collections.emptyList();

                    UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(
                        userEmail,
                        null,
                        authorities
                    );
                    authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                    SecurityContextHolder.getContext().setAuthentication(authToken);
                }
            }
        } catch (Exception e) {
            // Invalid JWT token - don't authenticate, let security filter chain handle it
        }
    }
}
```

```

    }

    filterChain.doFilter(request, response);
}
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/security/JwtService.java

```

package com.ridelink.ride_service.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;
import java.util.function.Function;

@Service
public class JwtService {

    @Value("${jwt.secret:RideLinkSecretKeyForJwtSigning2026SecureKeyWith256BitsMin!}")
    private String secretKey;

    private SecretKey getSigningKey() {
        byte[] keyBytes = secretKey.getBytes(StandardCharsets.UTF_8);
        return Keys.hmacShaKeyFor(keyBytes);
    }

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public String extractRole(String token) {
        Claims claims = extractAllClaims(token);
        return claims.get("role", String.class);
    }

    public String extractUserId(String token) {
        Claims claims = extractAllClaims(token);
        return claims.get("userId", String.class);
    }

    public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }

    public boolean isTokenValid(String token) {
        try {
            return !isTokenExpired(token);
        } catch (Exception e) {
            return false;
        }
    }

    private boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    private Date extractExpiration(String token) {

```

```

        return extractClaim(token, Claims::getExpiration);
    }

    private Claims extractAllClaims(String token) {
        return Jwts.parser()
            .verifyWith(getSigningKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/service/RideService.java

```

package com.ridelink.ride_service.service;

import com.ridelink.ride_service.client.AccountServiceClient;
import com.ridelink.ride_service.client.DriverServiceClient;
import com.ridelink.ride_service.dto.*;
import com.ridelink.ride_service.entity.*;
import com.ridelink.ride_service.exception.BadRequestException;
import com.ridelink.ride_service.exception.InvalidStateTransitionException;
import com.ridelink.ride_service.exception.NoDriverAvailableException;
import com.ridelink.ride_service.exception.ResourceNotFoundException;
import com.ridelink.ride_service.repository.RideRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.util.Comparator;
import java.util.List;

@Slf4j
@Service
@RequiredArgsConstructor
public class RideService {

    private final RideRepository rideRepository;
    private final AccountServiceClient accountServiceClient;
    private final DriverServiceClient driverServiceClient;

    /**
     * Create ride request and auto-assign an eligible driver using documented selection strategy.
     */
    public RideResponse createRide(CreateRideRequest request) {
        log.info("Processing ride request for passenger ID: {}", request.getPassengerId());

        // 1. Interservice Validation: Account Service (Passenger exists, ACTIVE, PASSENGER role)
        PassengerValidationDto passenger = accountServiceClient.validatePassenger(request.getPassengerId());
        log.info("Passenger validated successfully: {}", passenger.getFullName());

        // 2. Business Constraint: Check for existing active ride for passenger
        List<RideStatus> activeStatuses = List.of(
            RideStatus.REQUESTED,
            RideStatus.ASSIGNED,
            RideStatus.ACCEPTED,
            RideStatus.IN_PROGRESS
        );
        List<Ride> activeRides = rideRepository.findByPassengerIdAndStatusIn(request.getPassengerId(),
activeStatuses);
        if (!activeRides.isEmpty()) {
            throw new BadRequestException("Passenger already has an ongoing active ride (Ride ID: " +
activeRides.get(0).getId() + ")");

```

```

    }

    // 3. Interservice Query: Find available drivers matching serviceArea and vehicleType
    List<DriverSummaryDto> availableDrivers = driverServiceClient.getAvailableDrivers(
        request.getServiceArea(),
        request.getVehicleType()
    );

    if (availableDrivers == null || availableDrivers.isEmpty()) {
        throw new NoDriverAvailableException(
            "No available " + request.getVehicleType() + " drivers found in service area: " +
            request.getServiceArea()
        );
    }

    // 4. Selection Strategy: Prioritize highest rating, then highest total trips
    DriverSummaryDto selectedDriver = availableDrivers.stream()
        .max(Comparator.comparingDouble((DriverSummaryDto d) -> d.getRating() != null ? d.getRating() :
0.0)
            .thenComparingInt(d -> d.getTotalTrips() != null ? d.getTotalTrips() : 0))
        .orElse(availableDrivers.get(0));

    log.info("Assigned driver {} ({{}}) to ride request", selectedDriver.getDriverName(),
selectedDriver.getId());

    // 5. Calculate base fare estimate
    double estimatedFare = calculateFareEstimate(request.getVehicleType(),
request.getEstimatedDistanceKm());

    // 6. Notify Driver Service to mark driver as ON_TRIP
    driverServiceClient.updateDriverTripStatus(selectedDriver.getId(), true);

    // 7. Persist Ride entity
    LocalDateTime now = LocalDateTime.now();
    Ride ride = Ride.builder()
        .passengerId(passenger.getId())
        .passengerName(passenger.getFullName())
        .passengerPhone(passenger.getPhoneNumber())
        .driverId(selectedDriver.getId())
        .driverUserId(selectedDriver.getUserId())
        .driverName(selectedDriver.getDriverName())
        .driverPhone(selectedDriver.getPhoneNumber())
        .vehiclePlate(selectedDriver.getVehicle() != null ?
selectedDriver.getVehicle().getLicensePlate() : "N/A")
        .vehicleModel(selectedDriver.getVehicle() != null ? selectedDriver.getVehicle().getModel() :
"N/A")
        .pickupLocation(LocationPoint.builder()
            .address(request.getPickupAddress())
            .latitude(request.getPickupLatitude())
            .longitude(request.getPickupLongitude())
            .serviceArea(request.getServiceArea())
            .build())
        .dropoffLocation(LocationPoint.builder()
            .address(request.getDropoffAddress())
            .latitude(request.getDropoffLatitude())
            .longitude(request.getDropoffLongitude())
            .serviceArea(request.getServiceArea())
            .build())
        .vehicleType(request.getVehicleType())
        .estimatedDistanceKm(request.getEstimatedDistanceKm())
        .fareAmount(estimatedFare)
        .status(RideStatus.ASSIGNED)
        .requestedAt(now)
        .assignedAt(now)
        .build();

    Ride savedRide = rideRepository.save(ride);

```

```

        log.info("Ride created and assigned successfully with ID: {}", savedRide.getId());
        return mapToResponse(savedRide);
    }

    /**
     * Driver accepts assigned ride.
     * Transition: ASSIGNED -> ACCEPTED
     */
    public RideResponse acceptRide(String rideId) {
        Ride ride = getRideEntity(rideId);

        if (ride.getStatus() != RideStatus.ASSIGNED) {
            throw new InvalidStateTransitionException(
                "Cannot accept ride from status " + ride.getStatus() + ". Must be ASSIGNED."
            );
        }

        ride.setStatus(RideStatus.ACCEPTED);
        ride.setAcceptedAt(LocalDateTime.now());
        return mapToResponse(rideRepository.save(ride));
    }

    /**
     * Driver starts trip.
     * Transition: ACCEPTED -> IN_PROGRESS
     */
    public RideResponse startRide(String rideId) {
        Ride ride = getRideEntity(rideId);

        if (ride.getStatus() != RideStatus.ACCEPTED) {
            throw new InvalidStateTransitionException(
                "Cannot start ride from status " + ride.getStatus() + ". Must be ACCEPTED."
            );
        }

        ride.setStatus(RideStatus.IN_PROGRESS);
        ride.setStartedAt(LocalDateTime.now());
        return mapToResponse(rideRepository.save(ride));
    }

    /**
     * Driver completes trip.
     * Transition: IN_PROGRESS -> COMPLETED
     * Releases driver back to ONLINE in Driver Service.
     */
    public RideResponse completeRide(String rideId) {
        Ride ride = getRideEntity(rideId);

        if (ride.getStatus() != RideStatus.IN_PROGRESS) {
            throw new InvalidStateTransitionException(
                "Cannot complete ride from status " + ride.getStatus() + ". Must be IN_PROGRESS."
            );
        }

        ride.setStatus(RideStatus.COMPLETED);
        ride.setCompletedAt(LocalDateTime.now());

        // Interservice: Mark driver back ONLINE and increment trip count
        if (ride.getDriverId() != null) {
            driverServiceClient.updateDriverTripStatus(ride.getDriverId(), false);
        }

        return mapToResponse(rideRepository.save(ride));
    }

    /**
     * Cancel ride by passenger or driver.

```

```

* Allowed from REQUESTED, ASSIGNED, ACCEPTED.
* Not allowed once COMPLETED or already CANCELLED.
*/
public RideResponse cancelRide(String rideId, String reason) {
    Ride ride = getRideEntity(rideId);

    if (ride.getStatus() == RideStatus.COMPLETED) {
        throw new InvalidStateTransitionException("Cannot cancel a ride that has already been COMPLETED.");
    }
    if (ride.getStatus() == RideStatus.CANCELLED) {
        throw new InvalidStateTransitionException("Ride is already CANCELLED.");
    }

    ride.setStatus(RideStatus.CANCELLED);
    ride.setCancellationReason(reason != null && !reason.isBlank() ? reason : "Cancelled by user");
    ride.setCancelledAt(LocalDate.now());

    // Interservice: Release driver if one was assigned
    if (ride.getDriverId() != null) {
        driverServiceClient.updateDriverTripStatus(ride.getDriverId(), false);
    }

    return mapToResponse(rideRepository.save(ride));
}

public RideResponse getRideById(String rideId) {
    return mapToResponse(getRideEntity(rideId));
}

public List<RideResponse> getRidesByPassengerId(String passengerId) {
    return rideRepository.findByPassengerIdOrderByRequestedAtDesc(passengerId)
        .stream()
        .map(this::mapToResponse)
        .toList();
}

public List<RideResponse> getRidesByDriverId(String driverId) {
    return rideRepository.findByDriverIdOrderByRequestedAtDesc(driverId)
        .stream()
        .map(this::mapToResponse)
        .toList();
}

public List<RideResponse> getAllRides(RideStatus status) {
    if (status != null) {
        return rideRepository.findByStatus(status)
            .stream()
            .map(this::mapToResponse)
            .toList();
    }
    return rideRepository.findAll()
        .stream()
        .map(this::mapToResponse)
        .toList();
}

private Ride getRideEntity(String rideId) {
    return rideRepository.findById(rideId)
        .orElseThrow(() -> new ResourceNotFoundException("Ride not found with ID: " + rideId));
}

private double calculateFareEstimate(VehicleType vehicleType, double distanceKm) {
    double baseRate;
    double perKmRate;

    switch (vehicleType) {
        case CAR -> {

```

```

        baseRate = 200.0;
        perKmRate = 120.0;
    }
    case VAN -> {
        baseRate = 350.0;
        perKmRate = 160.0;
    }
    case BIKE -> {
        baseRate = 80.0;
        perKmRate = 50.0;
    }
    case TUK -> {
        baseRate = 100.0;
        perKmRate = 80.0;
    }
    default -> {
        baseRate = 150.0;
        perKmRate = 100.0;
    }
}

return Math.round((baseRate + (perKmRate * distanceKm)) * 100.0) / 100.0;
}

private RideResponse mapToResponse(Ride ride) {
    return RideResponse.builder()
        .id(ride.getId())
        .passengerId(ride.getPassengerId())
        .passengerName(ride.getPassengerName())
        .passengerPhone(ride.getPassengerPhone())
        .driverId(ride.getDriverId())
        .driverUserId(ride.getDriverUserId())
        .driverName(ride.getDriverName())
        .driverPhone(ride.getDriverPhone())
        .vehiclePlate(ride.getVehiclePlate())
        .vehicleModel(ride.getVehicleModel())
        .pickupLocation(ride.getPickupLocation())
        .dropoffLocation(ride.getDropoffLocation())
        .vehicleType(ride.getVehicleType())
        .estimatedDistanceKm(ride.getEstimatedDistanceKm())
        .status(ride.getStatus())
        .fareAmount(ride.getFareAmount())
        .cancellationReason(ride.getCancellationReason())
        .requestedAt(ride.getRequestedAt())
        .assignedAt(ride.getAssignedAt())
        .acceptedAt(ride.getAcceptedAt())
        .startedAt(ride.getStartedAt())
        .completedAt(ride.getCompletedAt())
        .cancelledAt(ride.getCancelledAt())
        .build();
}
}

```

File: ride-service/src/main/java/com/ridelink/ride_service/TestController.java

```

package com.ridelink.ride_service;

import com.ridelink.ride_service.entity.Ride;
import com.ridelink.ride_service.repository.RideRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RestController;

```

```

import java.time.LocalDateTime;

@RestController
public class TestController {

    @Autowired
    private RideRepository rideRepository;

    @GetMapping("/test/db")
    public String testDatabase() {
        try {
            long count = rideRepository.count();
            return "✅ Ride Database connection successful! Ride count: " + count;
        } catch (Exception e) {
            return "❌ Ride Database connection failed: " + e.getMessage();
        }
    }

    @PostMapping("/test/create-sample-ride")
    public String createSampleRide() {
        try {
            if (rideRepository.count() > 0) {
                return "✅ Sample ride already exists in database!";
            }

            // Note: This creates a minimal ride for DB testing only
            // Real rides need proper validation and references
            Ride sampleRide = Ride.builder()
                .passengerId("sample-passenger-id")
                .driverId("sample-driver-id")
                .pickupLocation(com.ridelink.ride_service.entity.LocationPoint.builder().address("Sample
Pickup Location").build())
                .dropoffLocation(com.ridelink.ride_service.entity.LocationPoint.builder().address("Sample
Dropoff Location").build())
                .fareAmount(500.0)
                .requestedAt(LocalDateTime.now())
                .build();

            Ride saved = rideRepository.save(sampleRide);
            return "✅ Sample ride created! ID: " + saved.getId() + " - ridelink_ride_db should now be
visible!";
        } catch (Exception e) {
            return "❌ Failed to create ride: " + e.getMessage();
        }
    }
}

```

File: ride-service/src/test/java/com/ridelink/ride_service/controller/RideControllerTest.java

```

package com.ridelink.ride_service.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.ridelink.ride_service.dto.CancelRideRequest;
import com.ridelink.ride_service.dto.CreateRideRequest;
import com.ridelink.ride_service.dto.RideResponse;
import com.ridelink.ride_service.entity.LocationPoint;
import com.ridelink.ride_service.entity.RideStatus;
import com.ridelink.ride_service.entity.VehicleType;
import com.ridelink.ride_service.exception.GlobalExceptionHandler;
import com.ridelink.ride_service.exception.InvalidStateTransitionException;
import com.ridelink.ride_service.exception.NoDriverAvailableException;
import com.ridelink.ride_service.exception.ResourceNotFoundException;
import com.ridelink.ride_service.service.RideService;
import org.junit.jupiter.api.BeforeEach;

```

```

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.setup.MockMvcBuilders;

import java.time.LocalDateTime;
import java.util.List;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@ExtendWith(MockitoExtension.class)
class RideControllerTest {

    private MockMvc mockMvc;
    private ObjectMapper objectMapper;

    @Mock
    private RideService rideService;

    @InjectMocks
    private RideController driverController;

    private RideResponse sampleResponse;

    @BeforeEach
    void setUp() {
        objectMapper = new ObjectMapper();
        objectMapper.findAndRegisterModules();
        mockMvc = MockMvcBuilders.standaloneSetup(driverController)
            .setControllerAdvice(new GlobalExceptionHandler())
            .build();

        sampleResponse = RideResponse.builder()
            .id("r1")
            .passengerId("p1")
            .passengerName("Kamal Perera")
            .passengerPhone("+94771234567")
            .driverId("d1")
            .driverUserId("u2")
            .driverName("Sunil Silva")
            .driverPhone("+94719876543")
            .vehiclePlate("CAB-1234")
            .vehicleModel("Corolla")
            .pickupLocation(LocationPoint.builder()
                .address("Colombo Fort")
                .latitude(6.9344)
                .longitude(79.8428)
                .serviceArea("Colombo")
                .build())
            .dropoffLocation(LocationPoint.builder()
                .address("Bambalapitiya")
                .latitude(6.8924)
                .longitude(79.8553)
                .serviceArea("Colombo")
                .build())
            .vehicleType(VehicleType.CAR)
            .estimatedDistanceKm(5.0)
            .fareAmount(800.0)
            .status(RideStatus.ASSIGNED)
    
```

```

        .requestedAt(LocalDateTime.now())
        .assignedAt(LocalDateTime.now())
        .build();
    }

    @Test
    void createRide_ValidRequest_Returns201Created() throws Exception {
        CreateRideRequest request = CreateRideRequest.builder()
            .passengerId("p1")
            .pickupAddress("Colombo Fort")
            .pickupLatitude(6.9344)
            .pickupLongitude(79.8428)
            .serviceArea("Colombo")
            .dropoffAddress("Bambalapitiya")
            .dropoffLatitude(6.8924)
            .dropoffLongitude(79.8553)
            .vehicleType(VehicleType.CAR)
            .estimatedDistanceKm(5.0)
            .build();

        when(rideService.createRide(any(CreateRideRequest.class))).thenReturn(sampleResponse);

        mockMvc.perform(post("/api/rides")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.id").value("r1"))
            .andExpect(jsonPath("$.passengerId").value("p1"))
            .andExpect(jsonPath("$.status").value("ASSIGNED"));
    }

    @Test
    void createRide_MissingFields_Returns400BadRequest() throws Exception {
        CreateRideRequest request = CreateRideRequest.builder()
            .passengerId("")
            .pickupAddress("")
            .build();

        mockMvc.perform(post("/api/rides")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.status").value(400));
    }

    @Test
    void createRide_NoDriverAvailable_Returns404() throws Exception {
        CreateRideRequest request = CreateRideRequest.builder()
            .passengerId("p1")
            .pickupAddress("Colombo Fort")
            .pickupLatitude(6.9344)
            .pickupLongitude(79.8428)
            .serviceArea("Colombo")
            .dropoffAddress("Bambalapitiya")
            .dropoffLatitude(6.8924)
            .dropoffLongitude(79.8553)
            .vehicleType(VehicleType.CAR)
            .estimatedDistanceKm(5.0)
            .build();

        when(rideService.createRide(any(CreateRideRequest.class)))
            .thenThrow(new NoDriverAvailableException("No available drivers found"));

        mockMvc.perform(post("/api/rides")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isNotFound());
    }

```

```

        .andExpect(jsonPath("$.error").value("No Driver Available"));
    }

    @Test
    void getRideById_Success() throws Exception {
        when(rideService.getRideById("r1")).thenReturn(sampleResponse);

        mockMvc.perform(get("/api/rides/r1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value("r1"))
            .andExpect(jsonPath("$.driverName").value("Sunil Silva"));
    }

    @Test
    void getRideById_NotFound_Returns404() throws Exception {
        when(rideService.getRideById("nonexistent"))
            .thenReturn(new ResourceNotFoundException("Ride not found"));

        mockMvc.perform(get("/api/rides/nonexistent"))
            .andExpect(status().isNotFound())
            .andExpect(jsonPath("$.status").value(404));
    }

    @Test
    void acceptRide_Success_Returns200() throws Exception {
        sampleResponse.setStatus(RideStatus.ACCEPTED);
        when(rideService.acceptRide("r1")).thenReturn(sampleResponse);

        mockMvc.perform(patch("/api/rides/r1/accept"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.status").value("ACCEPTED"));
    }

    @Test
    void startRide_Success_Returns200() throws Exception {
        sampleResponse.setStatus(RideStatus.IN_PROGRESS);
        when(rideService.startRide("r1")).thenReturn(sampleResponse);

        mockMvc.perform(patch("/api/rides/r1/start"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.status").value("IN_PROGRESS"));
    }

    @Test
    void completeRide_Success_Returns200() throws Exception {
        sampleResponse.setStatus(RideStatus.COMPLETED);
        when(rideService.completeRide("r1")).thenReturn(sampleResponse);

        mockMvc.perform(patch("/api/rides/r1/complete"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.status").value("COMPLETED"));
    }

    @Test
    void cancelRide_Success_Returns200() throws Exception {
        sampleResponse.setStatus(RideStatus.CANCELLED);
        sampleResponse.setCancellationReason("Changed destination");
        CancelRideRequest cancelRequest = new CancelRideRequest("Changed destination");

        when(rideService.cancelRide(eq("r1"), eq("Changed destination"))).thenReturn(sampleResponse);

        mockMvc.perform(patch("/api/rides/r1/cancel")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(cancelRequest)))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.status").value("CANCELLED"))
            .andExpect(jsonPath("$.cancellationReason").value("Changed destination"));
    }

```

```

    }

    @Test
    void invalidStateTransition_Returns400BadRequest() throws Exception {
        when(rideService.startRide("r1"))
            .thenThrow(new InvalidStateTransitionException("Cannot start ride from status ASSIGNED"));

        mockMvc.perform(patch("/api/rides/r1/start"))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.error").value("Bad Request - Invalid State Transition"));
    }

    @Test
    void getRidesByPassengerId_ReturnsList() throws Exception {
        when(rideService.getRidesByPassengerId("p1")).thenReturn(List.of(sampleResponse));

        mockMvc.perform(get("/api/rides/passenger/p1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.length").value(1))
            .andExpect(jsonPath("$[0].passengerId").value("p1"));
    }
}

```

File: ride-service/src/test/java/com/ridelink/ride_service/RideServiceApplicationTests.java

```

package com.ridelink.ride_service;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.TestPropertySource;

@SpringBootTest
@TestPropertySource(properties = {
    "spring.data.mongodb.uri=mongodb://localhost:27017/ridelink_ride_test_db",
    "services.account-service.url=http://localhost:8081",
    "services.driver-service.url=http://localhost:8082"
})
class RideServiceApplicationTests {

    @Test
    void contextLoads() {
    }

}

```

File: ride-service/src/test/java/com/ridelink/ride_service/service/RideServiceTest.java

```

package com.ridelink.ride_service.service;

import com.ridelink.ride_service.client.AccountServiceClient;
import com.ridelink.ride_service.client.DriverServiceClient;
import com.ridelink.ride_service.dto.CreateRideRequest;
import com.ridelink.ride_service.dto.DriverSummaryDto;
import com.ridelink.ride_service.dto.PassengerValidationDto;
import com.ridelink.ride_service.dto.RideResponse;
import com.ridelink.ride_service.entity.LocationPoint;
import com.ridelink.ride_service.entity.Ride;
import com.ridelink.ride_service.entity.RideStatus;
import com.ridelink.ride_service.entity.VehicleType;
import com.ridelink.ride_service.exception.BadRequestException;
import com.ridelink.ride_service.exception.InvalidStateTransitionException;
import com.ridelink.ride_service.exception.NoDriverAvailableException;
import com.ridelink.ride_service.exception.ResourceNotFoundException;

```

```
import com.ridelink.ride_service.repository.RideRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.time.LocalDateTime;
import java.util.Collections;
import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class RideServiceTest {

    @Mock
    private RideRepository rideRepository;

    @Mock
    private AccountServiceClient accountServiceClient;

    @Mock
    private DriverServiceClient driverServiceClient;

    @InjectMocks
    private RideService rideService;

    private CreateRideRequest sampleRequest;
    private PassengerValidationDto samplePassenger;
    private DriverSummaryDto sampleDriver;
    private Ride sampleRide;

    @BeforeEach
    void setUp() {
        sampleRequest = CreateRideRequest.builder()
            .passengerId("p1")
            .pickupAddress("Colombo Fort")
            .pickupLatitude(6.9344)
            .pickupLongitude(79.8428)
            .serviceArea("Colombo")
            .dropoffAddress("Bambalapitiya")
            .dropoffLatitude(6.8924)
            .dropoffLongitude(79.8553)
            .vehicleType(VehicleType.CAR)
            .estimatedDistanceKm(5.0)
            .build();

        samplePassenger = PassengerValidationDto.builder()
            .id("p1")
            .fullName("Kamal Perera")
            .email("kamal@example.com")
            .phoneNumber("+94771234567")
            .role("PASSENGER")
            .status("ACTIVE")
            .build();

        sampleDriver = DriverSummaryDto.builder()
            .id("d1")
            .userId("u2")
            .driverName("Sunil Silva")
            .phoneNumber("+94719876543")
            .rating(4.8)
```

```

        .totalTrips(45)
        .vehicle(DriverSummaryDto.VehicleSummary.builder()
            .make("Toyota")
            .model("Corolla")
            .licensePlate("CAB-1234")
            .vehicleType(VehicleType.CAR)
            .capacity(4)
            .build())
        .status("ONLINE")
        .build();

sampleRide = Ride.builder()
    .id("r1")
    .passengerId("p1")
    .passengerName("Kamal Perera")
    .passengerPhone("+94771234567")
    .driverId("d1")
    .driverUserId("u2")
    .driverName("Sunil Silva")
    .driverPhone("+94719876543")
    .vehiclePlate("CAB-1234")
    .vehicleModel("Corolla")
    .pickupLocation(LocationPoint.builder()
        .address("Colombo Fort")
        .latitude(6.9344)
        .longitude(79.8428)
        .serviceArea("Colombo")
        .build())
    .dropoffLocation(LocationPoint.builder()
        .address("Bambalapitiya")
        .latitude(6.8924)
        .longitude(79.8553)
        .serviceArea("Colombo")
        .build())
    .vehicleType(VehicleType.CAR)
    .estimatedDistanceKm(5.0)
    .fareAmount(800.0)
    .status(RideStatus.ASSIGNED)
    .requestedAt(LocalDateTime.now())
    .assignedAt(LocalDateTime.now())
    .build();
}

@Test
void createRide_Success() {
    when(accountServiceClient.validatePassenger("p1")).thenReturn(samplePassenger);
    when(rideRepository.findByPassengerIdAndStatusIn(eq("p1"),
anyList())).thenReturn(Collections.emptyList());
    when(driverServiceClient.getAvailableDrivers("Colombo", VehicleType.CAR))
        .thenReturn(List.of(sampleDriver));
    when(rideRepository.save(any(Ride.class))).thenReturn(sampleRide);

    RideResponse response = rideService.createRide(sampleRequest);

    assertNotNull(response);
    assertEquals("p1", response.getPassengerId());
    assertEquals("d1", response.getDriverId());
    assertEquals(RideStatus.ASSIGNED, response.getStatus());

    verify(accountServiceClient, times(1)).validatePassenger("p1");
    verify(driverServiceClient, times(1)).getAvailableDrivers("Colombo", VehicleType.CAR);
    verify(driverServiceClient, times(1)).updateDriverTripStatus("d1", true);
    verify(rideRepository, times(1)).save(any(Ride.class));
}

@Test
void createRide_ActiveRideAlreadyExists_ThrowsBadRequest() {

```

```

        when(accountServiceClient.validatePassenger("p1")).thenReturn(samplePassenger);
        when(rideRepository.findByIdAndStatusIn(eq("p1"), anyList())).thenReturn(List.of(sampleRide));

        assertThrows(BadRequestException.class, () -> rideService.createRide(sampleRequest));

        verify(driverServiceClient, never()).getAvailableDrivers(any(), any());
        verify(rideRepository, never()).save(any());
    }

    @Test
    void createRide_NoDriversAvailable_ThrowsNoDriverAvailableException() {
        when(accountServiceClient.validatePassenger("p1")).thenReturn(samplePassenger);
        when(rideRepository.findByIdAndStatusIn(eq("p1"),
anyList())).thenReturn(Collections.emptyList());
        when(driverServiceClient.getAvailableDrivers("Colombo",
VehicleType.CAR)).thenReturn(Collections.emptyList());

        assertThrows(NoDriverAvailableException.class, () -> rideService.createRide(sampleRequest));

        verify(driverServiceClient, never()).updateDriverTripStatus(any(), anyBoolean());
        verify(rideRepository, never()).save(any());
    }

    @Test
    void acceptRide_FromAssigned_Success() {
        sampleRide.setStatus(RideStatus.ASSIGNED);
        when(rideRepository.findById("r1")).thenReturn(Optional.of(sampleRide));
        when(rideRepository.save(any(Ride.class))).thenAnswer(invocation -> invocation.getArgument(0));

        RideResponse response = rideService.acceptRide("r1");

        assertEquals(RideStatus.ACCEPTED, response.getStatus());
        assertNotNull(response.getAcceptedAt());
        verify(rideRepository, times(1)).save(sampleRide);
    }

    @Test
    void acceptRide_InvalidTransition_ThrowsException() {
        sampleRide.setStatus(RideStatus.IN_PROGRESS);
        when(rideRepository.findById("r1")).thenReturn(Optional.of(sampleRide));

        assertThrows(InvalidStateTransitionException.class, () -> rideService.acceptRide("r1"));
    }

    @Test
    void startRide_FromAccepted_Success() {
        sampleRide.setStatus(RideStatus.ACCEPTED);
        when(rideRepository.findById("r1")).thenReturn(Optional.of(sampleRide));
        when(rideRepository.save(any(Ride.class))).thenAnswer(invocation -> invocation.getArgument(0));

        RideResponse response = rideService.startRide("r1");

        assertEquals(RideStatus.IN_PROGRESS, response.getStatus());
        assertNotNull(response.getStartedAt());
    }

    @Test
    void startRide_InvalidTransition_ThrowsException() {
        sampleRide.setStatus(RideStatus.ASSIGNED);
        when(rideRepository.findById("r1")).thenReturn(Optional.of(sampleRide));

        assertThrows(InvalidStateTransitionException.class, () -> rideService.startRide("r1"));
    }

    @Test
    void completeRide_FromInProgress_Success_ReleasesDriver() {
        sampleRide.setStatus(RideStatus.IN_PROGRESS);

```

```

        when(rideRepository.findById("r1")).thenReturn(Optional.of(sampleRide));
        when(rideRepository.save(any(Ride.class))).thenAnswer(invocation -> invocation.getArgument(0));

        RideResponse response = rideService.completeRide("r1");

        assertEquals(RideStatus.COMPLETED, response.getStatus());
        assertNotNull(response.getCompletedAt());
        verify(driverServiceClient, times(1)).updateDriverTripStatus("d1", false);
    }

    @Test
    void completeRide_InvalidTransition_ThrowsException() {
        sampleRide.setStatus(RideStatus.ACCEPTED);
        when(rideRepository.findById("r1")).thenReturn(Optional.of(sampleRide));

        assertThrows(InvalidStateTransitionException.class, () -> rideService.completeRide("r1"));
        verify(driverServiceClient, never()).updateDriverTripStatus(any(), anyBoolean());
    }

    @Test
    void cancelRide_FromAssigned_Success_ReleasesDriver() {
        sampleRide.setStatus(RideStatus.ASSIGNED);
        when(rideRepository.findById("r1")).thenReturn(Optional.of(sampleRide));
        when(rideRepository.save(any(Ride.class))).thenAnswer(invocation -> invocation.getArgument(0));

        RideResponse response = rideService.cancelRide("r1", "Passenger changed plans");

        assertEquals(RideStatus.CANCELLED, response.getStatus());
        assertEquals("Passenger changed plans", response.getCancellationReason());
        assertNotNull(response.getCancelledAt());
        verify(driverServiceClient, times(1)).updateDriverTripStatus("d1", false);
    }

    @Test
    void cancelRide_AlreadyCompleted_ThrowsException() {
        sampleRide.setStatus(RideStatus.COMPLETED);
        when(rideRepository.findById("r1")).thenReturn(Optional.of(sampleRide));

        assertThrows(InvalidStateTransitionException.class, () -> rideService.cancelRide("r1", "Too late"));
    }

    @Test
    void cancelRide_AlreadyCancelled_ThrowsException() {
        sampleRide.setStatus(RideStatus.CANCELLED);
        when(rideRepository.findById("r1")).thenReturn(Optional.of(sampleRide));

        assertThrows(InvalidStateTransitionException.class, () -> rideService.cancelRide("r1", "Already
cancelled"));
    }

    @Test
    void getRideById_Success() {
        when(rideRepository.findById("r1")).thenReturn(Optional.of(sampleRide));

        RideResponse response = rideService.getRideById("r1");

        assertNotNull(response);
        assertEquals("r1", response.getId());
    }

    @Test
    void getRideById_NotFound_ThrowsException() {
        when(rideRepository.findById("nonexistent")).thenReturn(Optional.empty());

        assertThrows(ResourceNotFoundException.class, () -> rideService.getRideById("nonexistent"));
    }

```

```
@Test
void getRidesByPassengerId_Success() {
    when(rideRepository.findByPassengerIdOrderByRequestedAtDesc("p1")).thenReturn(List.of(sampleRide));

    List<RideResponse> rides = rideService.getRidesByPassengerId("p1");

    assertEquals(1, rides.size());
    assertEquals("p1", rides.get(0).getPassengerId());
}

@Test
void getRidesByDriverId_Success() {
    when(rideRepository.findByDriverIdOrderByRequestedAtDesc("d1")).thenReturn(List.of(sampleRide));

    List<RideResponse> rides = rideService.getRidesByDriverId("d1");

    assertEquals(1, rides.size());
    assertEquals("d1", rides.get(0).getDriverId());
}
}
```

B.4 Fare & Payment Service (Member 4 - Port 8084)

Total Source Files: **37**

File: fare-service/src/main/java/com/ridelink/fare_service/client/RideServiceClient.java

```
package com.ridelink.fare_service.client;

import com.ridelink.fare_service.dto.RideValidationDto;
import com.ridelink.fare_service.exception.BadRequestException;
import com.ridelink.fare_service.exception.ResourceNotFoundException;
import com.ridelink.fare_service.exception.RideServiceException;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;

@Slf4j
@Component
public class RideServiceClient {

    private final RestClient restClient;
    private final String rideServiceUrl;

    public RideServiceClient(RestClient restClient,
                            @Value("${services.ride-service.url:http://localhost:8083}") String
rideServiceUrl) {
        this.restClient = restClient;
        this.rideServiceUrl = rideServiceUrl;
    }

    public RideValidationDto getAndValidateRide(String rideId, String expectedPassengerId) {
        String url = rideServiceUrl + "/api/rides/" + rideId;
        log.info("Calling Ride Service to validate ride at: {}", url);

        try {
            RideValidationDto ride = restClient.get()
                .uri(url)
                .retrieve()
                .onStatus(status -> status.value() == 404, (req, res) -> {
                    throw new ResourceNotFoundException("Ride not found in Ride Service with ID: " +
rideId);
                })
                .onStatus(HttpStatus::is5xxServerError, (req, res) -> {
                    throw new RideServiceException("Ride Service is currently unavailable");
                })
                .body(RideValidationDto.class);

            if (ride == null) {
                throw new ResourceNotFoundException("Ride could not be retrieved from Ride Service");
            }

            if (expectedPassengerId != null && !expectedPassengerId.equals(ride.getPassengerId())) {
                throw new BadRequestException("Passenger ID does not match the ride's registered passenger");
            }

            if ("CANCELLED".equalsIgnoreCase(ride.getStatus())) {
                throw new BadRequestException("Cannot process payment for a CANCELLED ride");
            }
        }
    }
}
```

```

        return ride;
    } catch (ResourceNotFoundException | BadRequestException e) {
        throw e;
    } catch (Exception e) {
        log.error("Failed to connect to Ride Service: {}", e.getMessage());
        throw new RideServiceException("Unable to contact Ride Service: " + e.getMessage());
    }
}
}
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/config/OpenApiConfig.java

```

package com.ridelink.fare_service.config;

import io.swagger.v3.oas.models.Components;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Contact;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.info.License;
import io.swagger.v3.oas.models.security.SecurityRequirement;
import io.swagger.v3.oas.models.security.SecurityScheme;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI fareServiceOpenAPI() {
        final String securitySchemeName = "bearerAuth";
        return new OpenAPI()
            .info(new Info()
                .title("RideLink - Fare & Payment Service API")
                .description("Microservice responsible for Fare Estimation, Final Fare Calculation, Simulated Payment Recording, and Digital Receipt Generation for IT3130 - Application Development (Member 4).")
                .version("1.0.0")
                .contact(new Contact().name("RideLink Backend Team").email("support@ridelink.com"))
                .license(new License().name("Apache 2.0").url("https://springdoc.org")))
            .addSecurityItem(new SecurityRequirement().addList(securitySchemeName))
            .components(new Components()
                .addSecuritySchemes(securitySchemeName,
                    new SecurityScheme()
                        .name(securitySchemeName)
                        .type(SecurityScheme.Type.HTTP)
                        .scheme("bearer")
                        .bearerFormat("JWT"))));
    }
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/config/RestClientConfig.java

```

package com.ridelink.fare_service.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.client.SimpleClientHttpRequestFactory;
import org.springframework.web.client.RestClient;

import java.time.Duration;

@Configuration
public class RestClientConfig {

```

```

@Bean
public RestClient restClient() {
    SimpleClientHttpRequestFactory factory = new SimpleClientHttpRequestFactory();
    factory.setConnectTimeout(Duration.ofSeconds(5));
    factory.setReadTimeout(Duration.ofSeconds(5));

    return RestClient.builder()
        .requestFactory(factory)
        .build();
}
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/config/SecurityConfig.java

```

package com.ridelink.fare_service.config;

import com.ridelink.fare_service.security.JwtAuthenticationFilter;
import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .cors(AbstractHttpConfigurer::disable)
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                // Public documentation & actuator
                .requestMatchers(
                    "/swagger-ui/**",
                    "/swagger-ui.html",
                    "/v3/api-docs/**",
                    "/api-docs/**",
                    "/actuator/**"
                ).permitAll()
                // Public fare estimate endpoint
                .requestMatchers("/api/fares/**").permitAll()
                // Payment endpoints permit all / auth protected via tokens
                .requestMatchers("/api/payments/**").permitAll()
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/controller/FareController.java

```

package com.ridelink.fare_service.controller;

import com.ridelink.fare_service.dto.FareEstimateRequest;
import com.ridelink.fare_service.dto.FareEstimateResponse;
import com.ridelink.fare_service.service.FareCalculationService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api/fares")
@RequiredArgsConstructor
@Tag(name = "Fare Estimation", description = "Endpoints for transparent fare estimation using documented pricing formula")
public class FareController {

    private final FareCalculationService fareCalculationService;

    @PostMapping("/estimate")
    @Operation(summary = "Calculate upfront fare estimate", description = "Calculates estimated fare based on vehicle type, distance, and duration using documented formula.")
    public ResponseEntity<List<FareEstimateResponse>> estimateFare(@Valid @RequestBody FareEstimateRequest request) {
        return ResponseEntity.ok(fareCalculationService.calculateEstimate(request));
    }
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/controller/PaymentController.java

```

package com.ridelink.fare_service.controller;

import com.ridelink.fare_service.dto.PaymentResponse;
import com.ridelink.fare_service.dto.ProcessPaymentRequest;
import com.ridelink.fare_service.dto.ReceiptResponse;
import com.ridelink.fare_service.service.PaymentService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/payments")
@RequiredArgsConstructor
@Tag(name = "Payment Management", description = "Endpoints for simulated payment processing, transaction records, and digital receipts")
public class PaymentController {

    private final PaymentService paymentService;

```

```

    @PostMapping("/process")
    @Operation(summary = "Process simulated payment for a ride", description = "Validates ride with Ride
Service, computes final fare, records payment, and creates digital receipt.")
    public ResponseEntity<PaymentResponse> processPayment(@Valid @RequestBody ProcessPaymentRequest request) {
        PaymentResponse response = paymentService.processPayment(request);
        return new ResponseEntity<>(response, HttpStatus.CREATED);
    }

    @GetMapping("/{id}")
    @Operation(summary = "Get payment transaction by payment ID")
    public ResponseEntity<PaymentResponse> getPaymentById(@PathVariable String id) {
        return ResponseEntity.ok(paymentService.getPaymentById(id));
    }

    @GetMapping("/ride/{rideId}")
    @Operation(summary = "Get payment transaction for a specific ride ID")
    public ResponseEntity<PaymentResponse> getPaymentByRideId(@PathVariable String rideId) {
        return ResponseEntity.ok(paymentService.getPaymentByRideId(rideId));
    }

    @GetMapping("/passenger/{passengerId}")
    @Operation(summary = "Get all payments for a passenger")
    public ResponseEntity<List<PaymentResponse>> getPaymentsByPassengerId(@PathVariable String passengerId) {
        return ResponseEntity.ok(paymentService.getPaymentsByPassengerId(passengerId));
    }

    @GetMapping("/{id}/receipt")
    @Operation(summary = "Get itemized digital receipt by payment ID")
    public ResponseEntity<ReceiptResponse> getReceiptByPaymentId(@PathVariable String id) {
        return ResponseEntity.ok(paymentService.getReceiptByPaymentId(id));
    }

    @GetMapping("/ride/{rideId}/receipt")
    @Operation(summary = "Get itemized digital receipt by ride ID")
    public ResponseEntity<ReceiptResponse> getReceiptByRideId(@PathVariable String rideId) {
        return ResponseEntity.ok(paymentService.getReceiptByRideId(rideId));
    }
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/dto/ApiResponse.java

```

package com.ridelink.fare_service.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class ApiResponse<T> {
    private boolean success;
    private String message;
    private T data;
    @Builder.Default
    private LocalDateTime timestamp = LocalDateTime.now();

    public static <T> ApiResponse<T> success(String message, T data) {
        return ApiResponse.<T>builder()

```

```

        .success(true)
        .message(message)
        .data(data)
        .timestamp(LocalDateTime.now())
        .build();
    }
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/dto/ErrorResponse.java

```

package com.ridelink.fare_service.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;
import java.util.Map;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class ErrorResponse {
    private int status;
    private String error;
    private String message;
    private String path;
    private Map<String, String> validationErrors;
    @Builder.Default
    private LocalDateTime timestamp = LocalDateTime.now();
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/dto/FareEstimateRequest.java

```

package com.ridelink.fare_service.dto;

import com.ridelink.fare_service.entity.VehicleType;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class FareEstimateRequest {

    private String pickupAddress;
    private String dropoffAddress;

    @NotNull(message = "Distance in km is required")
    @Positive(message = "Distance must be positive")
    private Double distanceKm;

    private Double estimatedDurationMinutes;
}

```

```
    private VehicleType vehicleType;
}
```

File: fare-service/src/main/java/com/ridelink/fare_service/dto/FareEstimateResponse.java

```
package com.ridelink.fare_service.dto;

import com.ridelink.fare_service.entity.VehicleType;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class FareEstimateResponse {

    private VehicleType vehicleType;
    private Double distanceKm;
    private Double estimatedDurationMinutes;
    private Double baseFare;
    private Double distanceCharge;
    private Double timeCharge;
    private Double surgeMultiplier;
    private Double estimatedFare;
    @Builder.Default
    private String currency = "LKR";
    private String calculationFormula;
}
```

File: fare-service/src/main/java/com/ridelink/fare_service/dto/PaymentResponse.java

```
package com.ridelink.fare_service.dto;

import com.ridelink.fare_service.entity.PaymentMethod;
import com.ridelink.fare_service.entity.PaymentStatus;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class PaymentResponse {

    private String id;
    private String transactionId;
    private String rideId;
    private String passengerId;
    private Double amount;
    private String currency;
    private PaymentMethod paymentMethod;
    private PaymentStatus status;
    private String failureReason;
    private LocalDateTime paidAt;
}
```

```
    private String receiptNumber;
}
```

File: fare-service/src/main/java/com/ridelink/fare_service/dto/ProcessPaymentRequest.java

```
package com.ridelink.fare_service.dto;

import com.ridelink.fare_service.entity.PaymentMethod;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.NoArgsConstructor;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class ProcessPaymentRequest {

    @NotBlank(message = "Ride ID is required")
    private String rideId;

    @NotBlank(message = "Passenger ID is required")
    private String passengerId;

    @NotNull(message = "Payment method is required (CARD, CASH, WALLET)")
    private PaymentMethod paymentMethod;

    private Double actualDistanceKm;

    private Double actualDurationMinutes;

    /**
     * Optional testing flag to simulate failure scenarios (declined card, insufficient funds)
     * as required by Assignment Workflow 7 & Rubric G2.
     */
    @Builder.Default
    private Boolean simulateFailure = false;
}
```

File: fare-service/src/main/java/com/ridelink/fare_service/dto/ReceiptResponse.java

```
package com.ridelink.fare_service.dto;

import com.ridelink.fare_service.entity.PaymentMethod;
import com.ridelink.fare_service.entity.VehicleType;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class ReceiptResponse {

    private String id;
```

```

    private String receiptNumber;
    private String transactionId;
    private String paymentId;
    private String rideId;
    private String passengerId;
    private String passengerName;
    private String driverName;
    private String vehiclePlate;
    private VehicleType vehicleType;
    private Double distanceKm;
    private Double durationMinutes;

    // Itemized charges
    private Double baseFare;
    private Double distanceCharge;
    private Double timeCharge;
    private Double surgeMultiplier;
    private Double totalAmount;
    @Builder.Default
    private String currency = "LKR";

    private PaymentMethod paymentMethod;
    private LocalDateTime issuedAt;
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/dto/RideValidationDto.java

```

package com.ridelink.fare_service.dto;

import com.ridelink.fare_service.entity.VehicleType;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.NoArgsConstructor;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class RideValidationDto {
    private String id;
    private String passengerId;
    private String passengerName;
    private String driverId;
    private String driverName;
    private String vehiclePlate;
    private VehicleType vehicleType;
    private Double estimatedDistanceKm;
    private String status;
    private Double fareAmount;
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/entity/Payment.java

```

package com.ridelink.fare_service.entity;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

```

```
import java.time.LocalDateTime;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
@Document(collection = "payments")
public class Payment {

    @Id
    private String id;
    private String transactionId;
    private String rideId;
    private String passengerId;
    private Double amount;
    private PaymentMethod paymentMethod;
    private PaymentStatus status;
    private String failureReason;
    private LocalDateTime paidAt;
    private LocalDateTime createdAt;
}
```

File: fare-service/src/main/java/com/ridelink/fare_service/entity/PaymentMethod.java

```
package com.ridelink.fare_service.entity;

public enum PaymentMethod {
    CARD,
    CASH,
    WALLET
}
```

File: fare-service/src/main/java/com/ridelink/fare_service/entity/PaymentStatus.java

```
package com.ridelink.fare_service.entity;

public enum PaymentStatus {
    PENDING,
    COMPLETED,
    FAILED,
    REFUNDED
}
```

File: fare-service/src/main/java/com/ridelink/fare_service/entity/Receipt.java

```
package com.ridelink.fare_service.entity;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.LocalDateTime;

@Data
@Builder
```

```
@NoArgsConstructor
@AllArgsConstructor
@Document(collection = "receipts")
public class Receipt {

    @Id
    private String id;
    private String receiptNumber;
    private String paymentId;
    private String transactionId;
    private String rideId;
    private String passengerId;
    private String passengerName;
    private String driverName;
    private String vehiclePlate;
    private VehicleType vehicleType;
    private Double distanceKm;
    private Double durationMinutes;
    private Double baseFare;
    private Double distanceCharge;
    private Double timeCharge;
    private Double surgeMultiplier;
    private Double totalAmount;
    private PaymentMethod paymentMethod;
    private LocalDateTime issuedAt;
}
```

File: fare-service/src/main/java/com/ridelink/fare_service/entity/VehicleType.java

```
package com.ridelink.fare_service.entity;

public enum VehicleType {
    CAR,
    VAN,
    BIKE,
    TUK
}
```

File: fare-service/src/main/java/com/ridelink/fare_service/exception/BadRequestException.java

```
package com.ridelink.fare_service.exception;

public class BadRequestException extends RuntimeException {
    public BadRequestException(String message) {
        super(message);
    }
}
```

File: fare-service/src/main/java/com/ridelink/fare_service/exception/DuplicatePaymentException.java

```
package com.ridelink.fare_service.exception;

public class DuplicatePaymentException extends RuntimeException {
    public DuplicatePaymentException(String message) {
        super(message);
    }
}
```

File: fare-service/src/main/java/com/ridelink/fare_service/exception/GlobalExceptionHandler.java

```
package com.ridelink.fare_service.exception;

import com.ridelink.fare_service.dto.ErrorResponse;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleResourceNotFoundException(ResourceNotFoundException ex,
        HttpServletRequest request) {
        ErrorResponse errorResponse = ErrorResponse.builder()
            .status(HttpStatus.NOT_FOUND.value())
            .error("Not Found")
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .timestamp(LocalDateTime.now())
            .build();
        return new ResponseEntity<>(errorResponse, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(PaymentFailedException.class)
    public ResponseEntity<ErrorResponse> handlePaymentFailedException(PaymentFailedException ex,
        HttpServletRequest request) {
        ErrorResponse errorResponse = ErrorResponse.builder()
            .status(HttpStatus.PAYMENT_REQUIRED.value())
            .error("Payment Failed")
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .timestamp(LocalDateTime.now())
            .build();
        return new ResponseEntity<>(errorResponse, HttpStatus.PAYMENT_REQUIRED);
    }

    @ExceptionHandler(DuplicatePaymentException.class)
    public ResponseEntity<ErrorResponse> handleDuplicatePaymentException(DuplicatePaymentException ex,
        HttpServletRequest request) {
        ErrorResponse errorResponse = ErrorResponse.builder()
            .status(HttpStatus.CONFLICT.value())
            .error("Duplicate Payment")
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .timestamp(LocalDateTime.now())
            .build();
        return new ResponseEntity<>(errorResponse, HttpStatus.CONFLICT);
    }

    @ExceptionHandler(BadRequestException.class)
    public ResponseEntity<ErrorResponse> handleBadRequestException(BadRequestException ex, HttpServletRequest
        request) {
        ErrorResponse errorResponse = ErrorResponse.builder()
            .status(HttpStatus.BAD_REQUEST.value())
            .error("Bad Request")

```

```

        .message(ex.getMessage())
        .path(request.getRequestURI())
        .timestamp(LocalDateTime.now())
        .build();
    return new ResponseEntity<>(errorResponse, HttpStatus.BAD_REQUEST);
}

@ExceptionHandler(RideServiceException.class)
public ResponseEntity<ErrorResponse> handleRideServiceException(RideServiceException ex, HttpServletRequest request) {
    ErrorResponse errorResponse = ErrorResponse.builder()
        .status(HttpStatus.BAD_GATEWAY.value())
        .error("Ride Service Integration Error")
        .message(ex.getMessage())
        .path(request.getRequestURI())
        .timestamp(LocalDateTime.now())
        .build();
    return new ResponseEntity<>(errorResponse, HttpStatus.BAD_GATEWAY);
}

@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ErrorResponse> handleValidationException(MethodArgumentNotValidException ex,
    HttpServletRequest request) {
    Map<String, String> errors = new HashMap<>();
    for (FieldError fieldError : ex.getBindingResult().getFieldErrors()) {
        errors.put(fieldError.getField(), fieldError.getDefaultMessage());
    }

    ErrorResponse errorResponse = ErrorResponse.builder()
        .status(HttpStatus.BAD_REQUEST.value())
        .error("Validation Failed")
        .message("Input validation failed for one or more fields")
        .path(request.getRequestURI())
        .validationErrors(errors)
        .timestamp(LocalDateTime.now())
        .build();
    return new ResponseEntity<>(errorResponse, HttpStatus.BAD_REQUEST);
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleGeneralException(Exception ex, HttpServletRequest request) {
    ErrorResponse errorResponse = ErrorResponse.builder()
        .status(HttpStatus.INTERNAL_SERVER_ERROR.value())
        .error("Internal Server Error")
        .message(ex.getMessage())
        .path(request.getRequestURI())
        .timestamp(LocalDateTime.now())
        .build();
    return new ResponseEntity<>(errorResponse, HttpStatus.INTERNAL_SERVER_ERROR);
}
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/exception/PaymentFailedException.java

```

package com.ridelink.fare_service.exception;

public class PaymentFailedException extends RuntimeException {
    public PaymentFailedException(String message) {
        super(message);
    }
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/exception/ResourceNotFoundException.java

```
package com.ridelink.fare_service.exception;

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

File: fare-service/src/main/java/com/ridelink/fare_service/exception/RideServiceException.java

```
package com.ridelink.fare_service.exception;

public class RideServiceException extends RuntimeException {
    public RideServiceException(String message) {
        super(message);
    }
}
```

File: fare-service/src/main/java/com/ridelink/fare_service/FareServiceApplication.java

```
package com.ridelink.fare_service;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class FareServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(FareServiceApplication.class, args);
    }
}
```

File: fare-service/src/main/java/com/ridelink/fare_service/repository/PaymentRepository.java

```
package com.ridelink.fare_service.repository;

import com.ridelink.fare_service.entity.Payment;
import com.ridelink.fare_service.entity.PaymentStatus;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface PaymentRepository extends MongoRepository<Payment, String> {

    Optional<Payment> findByRideId(String rideId);

    boolean existsByRideIdAndStatus(String rideId, PaymentStatus status);

    List<Payment> findByPassengerIdOrderByCreatedAtDesc(String passengerId);
}
```

File: fare-service/src/main/java/com/ridelink/fare_service/repository/ReceiptRepository.java

```

package com.ridelink.fare_service.repository;

import com.ridelink.fare_service.entity.Receipt;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface ReceiptRepository extends MongoRepository<Receipt, String> {

    Optional<Receipt> findByPaymentId(String paymentId);

    Optional<Receipt> findByRideId(String rideId);

    Optional<Receipt> findByReceiptNumber(String receiptNumber);
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/security/JwtAuthenticationFilter.java

```

package com.ridelink.fare_service.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.NonNull;
import lombok.RequiredArgsConstructor;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.Collections;
import java.util.List;

@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;

    @Override
    protected void doFilterInternal(
        @NonNull HttpServletRequest request,
        @NonNull HttpServletResponse response,
        @NonNull FilterChain filterChain
    ) throws ServletException, IOException {

        final String authHeader = request.getHeader("Authorization");
        final String jwt;
        final String userEmail;

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }
    }
}

```

```

        jwt = authHeader.substring(7);
        try {
            userEmail = jwtService.extractUsername(jwt);

            if (userEmail != null && SecurityContextHolder.getContext().getAuthentication() == null) {
                if (jwtService.isTokenValid(jwt)) {
                    String role = jwtService.extractRole(jwt);
                    List<SimpleGrantedAuthority> authorities = role != null
                        ? List.of(new SimpleGrantedAuthority("ROLE_" + role.toUpperCase()))
                        : Collections.emptyList();

                    UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(
                        userEmail,
                        null,
                        authorities
                    );
                    authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                    SecurityContextHolder.getContext().setAuthentication(authToken);
                }
            }
        } catch (Exception e) {
            // Invalid JWT token - don't authenticate, let security chain handle
        }

        filterChain.doFilter(request, response);
    }
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/security/JwtService.java

```

package com.ridelink.fare_service.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;
import java.util.function.Function;

@Service
public class JwtService {

    @Value("${jwt.secret:RideLinkSecretKeyForJwtSigning2026SecureKeyWith256BitsMin!}")
    private String secretKey;

    private SecretKey getSigningKey() {
        byte[] keyBytes = secretKey.getBytes(StandardCharsets.UTF_8);
        return Keys.hmacShaKeyFor(keyBytes);
    }

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public String extractRole(String token) {
        Claims claims = extractAllClaims(token);
        return claims.get("role", String.class);
    }

    public String extractUserId(String token) {

```

```

        Claims claims = extractAllClaims(token);
        return claims.get("userId", String.class);
    }

    public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }

    public boolean isTokenValid(String token) {
        try {
            return !isTokenExpired(token);
        } catch (Exception e) {
            return false;
        }
    }

    private boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    private Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    private Claims extractAllClaims(String token) {
        return Jwts.parser()
            .verifyWith(getSigningKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/service/FareCalculationService.java

```

package com.ridelink.fare_service.service;

import com.ridelink.fare_service.dto.FareEstimateRequest;
import com.ridelink.fare_service.dto.FareEstimateResponse;
import com.ridelink.fare_service.entity.VehicleType;
import org.springframework.stereotype.Service;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.List;

@Service
public class FareCalculationService {

    public record RateCard(double baseFare, double perKmRate, double perMinuteRate, double minimumFare) {}

    public static RateCard getRateCard(VehicleType vehicleType) {
        return switch (vehicleType) {
            case CAR -> new RateCard(200.0, 120.0, 10.0, 250.0);
            case VAN -> new RateCard(350.0, 160.0, 15.0, 400.0);
            case TUK -> new RateCard(100.0, 80.0, 8.0, 150.0);
            case BIKE -> new RateCard(80.0, 50.0, 5.0, 100.0);
        };
    }

    /**
     * Calculate fare estimates for a request.
     */
}

```

```

    * If vehicleType is specified, returns estimate for that type.
    * If vehicleType is null, returns estimates for all vehicle types.
    */
    public List<FareEstimateResponse> calculateEstimate(FareEstimateRequest request) {
        double distanceKm = request.getDistanceKm();
        // If duration is not provided, estimate based on average city speed of 25 km/h: (distance / 25) * 60
minutes
        double durationMinutes = request.getEstimatedDurationMinutes() != null &&
request.getEstimatedDurationMinutes() > 0
            ? request.getEstimatedDurationMinutes()
            : Math.max(5.0, Math.round((distanceKm / 25.0) * 60.0));

        List<FareEstimateResponse> estimates = new ArrayList<>();

        if (request.getVehicleType() != null) {
            estimates.add(computeEstimate(request.getVehicleType(), distanceKm, durationMinutes));
        } else {
            for (VehicleType type : VehicleType.values()) {
                estimates.add(computeEstimate(type, distanceKm, durationMinutes));
            }
        }

        return estimates;
    }

    public FareEstimateResponse computeEstimate(VehicleType vehicleType, double distanceKm, double
durationMinutes) {
        RateCard rates = getRateCard(vehicleType);
        double distanceCharge = round(rates.perKmRate() * distanceKm);
        double timeCharge = round(rates.perMinuteRate() * durationMinutes);
        double rawTotal = rates.baseFare() + distanceCharge + timeCharge;
        double surgeMultiplier = 1.0; // Standard rate multiplier

        double finalFare = Math.max(rates.minimumFare(), round(rawTotal * surgeMultiplier));

        String formula = String.format("max(%.2f, round((%.2f + (%.2f * %.2f) + (%.2f * %.2f)) * %.1f, 2))",
            rates.minimumFare(), rates.baseFare(), rates.perKmRate(), distanceKm, rates.perMinuteRate(),
durationMinutes, surgeMultiplier);

        return FareEstimateResponse.builder()
            .vehicleType(vehicleType)
            .distanceKm(distanceKm)
            .estimatedDurationMinutes(durationMinutes)
            .baseFare(rates.baseFare())
            .distanceCharge(distanceCharge)
            .timeCharge(timeCharge)
            .surgeMultiplier(surgeMultiplier)
            .estimatedFare(finalFare)
            .currency("LKR")
            .calculationFormula(formula)
            .build();
    }

    public record FinalFareCalculation(
        double baseFare,
        double distanceCharge,
        double timeCharge,
        double surgeMultiplier,
        double totalFare,
        double durationMinutes
    ) {}

    public FinalFareCalculation calculateFinalFare(VehicleType vehicleType, double distanceKm, Double
durationMinutes) {
        RateCard rates = getRateCard(vehicleType);
        double actualDuration = durationMinutes != null && durationMinutes > 0
            ? durationMinutes

```

```

        : Math.max(5.0, Math.round((distanceKm / 25.0) * 60.0));

double distanceCharge = round(rates.perKmRate() * distanceKm);
double timeCharge = round(rates.perMinuteRate() * actualDuration);
double rawTotal = rates.baseFare() + distanceCharge + timeCharge;
double surgeMultiplier = 1.0;

double finalFare = Math.max(rates.minimumFare(), round(rawTotal * surgeMultiplier));

return new FinalFareCalculation(
    rates.baseFare(),
    distanceCharge,
    timeCharge,
    surgeMultiplier,
    finalFare,
    actualDuration
);
}

private double round(double value) {
    return BigDecimal.valueOf(value).setScale(2, RoundingMode.HALF_UP).doubleValue();
}
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/service/PaymentService.java

```

package com.ridelink.fare_service.service;

import com.ridelink.fare_service.client.RideServiceClient;
import com.ridelink.fare_service.dto.*;
import com.ridelink.fare_service.entity.*;
import com.ridelink.fare_service.exception.DuplicatePaymentException;
import com.ridelink.fare_service.exception.PaymentFailedException;
import com.ridelink.fare_service.exception.ResourceNotFoundException;
import com.ridelink.fare_service.repository.PaymentRepository;
import com.ridelink.fare_service.repository.ReceiptRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.List;
import java.util.UUID;

@Slf4j
@Service
@RequiredArgsConstructor
public class PaymentService {

    private final PaymentRepository paymentRepository;
    private final ReceiptRepository receiptRepository;
    private final RideServiceClient rideServiceClient;
    private final FareCalculationService fareCalculationService;

    public PaymentResponse processPayment(ProcessPaymentRequest request) {
        log.info("Processing simulated payment for ride ID: {} by passenger: {}", request.getRideId(),
            request.getPassengerId());

        // 1. Interservice Validation: Verify Ride with Ride Management Service
        RideValidationDto ride = rideServiceClient.getAndValidateRide(request.getRideId(),
            request.getPassengerId());
        log.info("Ride validated successfully: Status={}, VehicleType={}", ride.getStatus(),
            ride.getVehicleType());
    }
}

```

```

// 2. Prevent Duplicate Payment (Idempotency check)
if (paymentRepository.existsByRideIdAndStatus(request.getRideId(), PaymentStatus.COMPLETED)) {
    throw new DuplicatePaymentException("Payment has already been successfully recorded for ride ID: "
+ request.getRideId());
}

// 3. Simulated Payment Failure (Negative Scenario support for Workflow 7 & Rubric G2)
if (Boolean.TRUE.equals(request.getSimulateFailure())) {
    String txnId = generateTransactionId();
    Payment failedPayment = Payment.builder()
        .transactionId(txnId)
        .rideId(request.getRideId())
        .passengerId(request.getPassengerId())
        .amount(ride.getFareAmount() != null ? ride.getFareAmount() : 0.0)
        .paymentMethod(request.getPaymentMethod())
        .status(PaymentStatus.FAILED)
        .failureReason("Simulated card payment authorization declined by issuer (insufficient
funds)")
        .createdAt(LocalDateTime.now())
        .build();
    paymentRepository.save(failedPayment);
    log.warn("Payment failed for ride ID {}: Simulated decline", request.getRideId());
    throw new PaymentFailedException("Payment processing failed: Transaction declined by simulated card
processor");
}

// 4. Calculate Final Fare using documented formula
double distanceKm = request.getActualDistanceKm() != null && request.getActualDistanceKm() > 0
    ? request.getActualDistanceKm()
    : (ride.getEstimatedDistanceKm() != null ? ride.getEstimatedDistanceKm() : 5.0);

VehicleType vehicleType = ride.getVehicleType() != null ? ride.getVehicleType() : VehicleType.CAR;

FareCalculationService.FinalFareCalculation fareBreakdown = fareCalculationService.calculateFinalFare(
    vehicleType,
    distanceKm,
    request.getActualDurationMinutes()
);

// 5. Create and persist Payment record
String transactionId = generateTransactionId();
LocalDateTime now = LocalDateTime.now();

Payment payment = Payment.builder()
    .transactionId(transactionId)
    .rideId(request.getRideId())
    .passengerId(request.getPassengerId())
    .amount(fareBreakdown.totalFare())
    .paymentMethod(request.getPaymentMethod())
    .status(PaymentStatus.COMPLETED)
    .paidAt(now)
    .createdAt(now)
    .build();

Payment savedPayment = paymentRepository.save(payment);
log.info("Payment recorded successfully: ID={}, TransactionId={}, Amount=LKR {}",
    savedPayment.getId(), transactionId, savedPayment.getAmount());

// 6. Generate digital receipt with itemized breakdown
String receiptNumber = generateReceiptNumber();
Receipt receipt = Receipt.builder()
    .receiptNumber(receiptNumber)
    .paymentId(savedPayment.getId())
    .transactionId(transactionId)
    .rideId(request.getRideId())
    .passengerId(request.getPassengerId())

```

```

        .passengerName(ride.getPassengerName())
        .driverName(ride.getDriverName())
        .vehiclePlate(ride.getVehiclePlate())
        .vehicleType(vehicleType)
        .distanceKm(distanceKm)
        .durationMinutes(fareBreakdown.durationMinutes())
        .baseFare(fareBreakdown.baseFare())
        .distanceCharge(fareBreakdown.distanceCharge())
        .timeCharge(fareBreakdown.timeCharge())
        .surgeMultiplier(fareBreakdown.surgeMultiplier())
        .totalAmount(fareBreakdown.totalFare())
        .paymentMethod(request.getPaymentMethod())
        .issuedAt(now)
        .build();

    receiptRepository.save(receipt);
    log.info("Digital receipt generated: Number={}", receiptNumber);

    return mapToPaymentResponse(savedPayment, receiptNumber);
}

public PaymentResponse getPaymentById(String id) {
    Payment payment = paymentRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Payment not found with ID: " + id));
    String receiptNumber = receiptRepository.findByPaymentId(payment.getId())
        .map(Receipt::getReceiptNumber)
        .orElse(null);
    return mapToPaymentResponse(payment, receiptNumber);
}

public PaymentResponse getPaymentByRideId(String rideId) {
    Payment payment = paymentRepository.findByRideId(rideId)
        .orElseThrow(() -> new ResourceNotFoundException("Payment not found for ride ID: " + rideId));
    String receiptNumber = receiptRepository.findByPaymentId(payment.getId())
        .map(Receipt::getReceiptNumber)
        .orElse(null);
    return mapToPaymentResponse(payment, receiptNumber);
}

public List<PaymentResponse> getPaymentsByPassengerId(String passengerId) {
    return paymentRepository.findByPassengerIdOrderByCreatedAtDesc(passengerId)
        .stream()
        .map(p -> {
            String receiptNo = receiptRepository.findByPaymentId(p.getId())
                .map(Receipt::getReceiptNumber)
                .orElse(null);
            return mapToPaymentResponse(p, receiptNo);
        })
        .toList();
}

public ReceiptResponse getReceiptByPaymentId(String paymentId) {
    Receipt receipt = receiptRepository.findByPaymentId(paymentId)
        .orElseThrow(() -> new ResourceNotFoundException("Receipt not found for payment ID: " +
paymentId));
    return mapToReceiptResponse(receipt);
}

public ReceiptResponse getReceiptByRideId(String rideId) {
    Receipt receipt = receiptRepository.findByRideId(rideId)
        .orElseThrow(() -> new ResourceNotFoundException("Receipt not found for ride ID: " + rideId));
    return mapToReceiptResponse(receipt);
}

private String generateTransactionId() {
    String timestamp = LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyyMMddHHmmss"));
    String random = UUID.randomUUID().toString().substring(0, 6).toUpperCase();
}

```

```

        return "TXN-" + timestamp + "-" + random;
    }

    private String generateReceiptNumber() {
        String timestamp = LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyyMMdd"));
        String random = UUID.randomUUID().toString().substring(0, 6).toUpperCase();
        return "REC-" + timestamp + "-" + random;
    }

    private PaymentResponse mapToPaymentResponse(Payment payment, String receiptNumber) {
        return PaymentResponse.builder()
            .id(payment.getId())
            .transactionId(payment.getTransactionId())
            .rideId(payment.getRideId())
            .passengerId(payment.getPassengerId())
            .amount(payment.getAmount())
            .currency("LKR")
            .paymentMethod(payment.getPaymentMethod())
            .status(payment.getStatus())
            .failureReason(payment.getFailureReason())
            .paidAt(payment.getPaidAt())
            .receiptNumber(receiptNumber)
            .build();
    }

    private ReceiptResponse mapToReceiptResponse(Receipt receipt) {
        return ReceiptResponse.builder()
            .id(receipt.getId())
            .receiptNumber(receipt.getReceiptNumber())
            .transactionId(receipt.getTransactionId())
            .paymentId(receipt.getPaymentId())
            .rideId(receipt.getRideId())
            .passengerId(receipt.getPassengerId())
            .passengerName(receipt.getPassengerName())
            .driverName(receipt.getDriverName())
            .vehiclePlate(receipt.getVehiclePlate())
            .vehicleType(receipt.getVehicleType())
            .distanceKm(receipt.getDistanceKm())
            .durationMinutes(receipt.getDurationMinutes())
            .baseFare(receipt.getBaseFare())
            .distanceCharge(receipt.getDistanceCharge())
            .timeCharge(receipt.getTimeCharge())
            .surgeMultiplier(receipt.getSurgeMultiplier())
            .totalAmount(receipt.getTotalAmount())
            .currency("LKR")
            .paymentMethod(receipt.getPaymentMethod())
            .issuedAt(receipt.getIssuedAt())
            .build();
    }
}

```

File: fare-service/src/main/java/com/ridelink/fare_service/TestController.java

```

package com.ridelink.fare_service;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.beans.factory.annotation.Value;

@RestController
public class TestController {

    @Value("${spring.data.mongodb.uri}")

```

```

private String mongoUri;

@GetMapping("/test/db")
public String testDatabase() {
    try {
        // Basic connection test
        return "✅ Fare Database connection configured! URI: " +
            mongoUri.substring(0, mongoUri.indexOf("@")) + "@cluster...";
    } catch (Exception e) {
        return "❌ Fare Database connection failed: " + e.getMessage();
    }
}

@PostMapping("/test/create-sample-payment")
public String createSamplePayment() {
    try {
        // Since fare-service might not have entities yet, just return success
        return "✅ Fare service is ready! ridelink_payment_db will be created when first payment is
processed.";
    } catch (Exception e) {
        return "❌ Failed to create payment: " + e.getMessage();
    }
}
}

```

File: fare-service/src/test/java/com/ridelink/fare_service/controller/PaymentControllerTest.java

```

package com.ridelink.fare_service.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.ridelink.fare_service.dto.PaymentResponse;
import com.ridelink.fare_service.dto.ProcessPaymentRequest;
import com.ridelink.fare_service.dto.ReceiptResponse;
import com.ridelink.fare_service.entity.PaymentMethod;
import com.ridelink.fare_service.entity.PaymentStatus;
import com.ridelink.fare_service.entity.VehicleType;
import com.ridelink.fare_service.exception.DuplicatePaymentException;
import com.ridelink.fare_service.exception.GlobalExceptionHandler;
import com.ridelink.fare_service.exception.PaymentFailedException;
import com.ridelink.fare_service.exception.ResourceNotFoundException;
import com.ridelink.fare_service.service.PaymentService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.setup.MockMvcBuilders;

import java.time.LocalDateTime;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@ExtendWith(MockitoExtension.class)
class PaymentControllerTest {

    private MockMvc mockMvc;

```

```

private ObjectMapper objectMapper;

@Mock
private PaymentService paymentService;

@InjectMocks
private PaymentController paymentController;

private PaymentResponse samplePaymentResponse;
private ReceiptResponse sampleReceiptResponse;

@BeforeEach
void setUp() {
    objectMapper = new ObjectMapper();
    objectMapper.findAndRegisterModules();
    mockMvc = MockMvcBuilders.standaloneSetup(paymentController)
        .setControllerAdvice(new GlobalExceptionHandler())
        .build();

    samplePaymentResponse = PaymentResponse.builder()
        .id("pay1")
        .transactionId("TXN-20260926-123456")
        .rideId("r1")
        .passengerId("p1")
        .amount(950.0)
        .currency("LKR")
        .paymentMethod(PaymentMethod.CARD)
        .status(PaymentStatus.COMPLETED)
        .paidAt(LocalDateTime.now())
        .receiptNumber("REC-20260926-654321")
        .build();

    sampleReceiptResponse = ReceiptResponse.builder()
        .id("rec1")
        .receiptNumber("REC-20260926-654321")
        .paymentId("pay1")
        .transactionId("TXN-20260926-123456")
        .rideId("r1")
        .passengerId("p1")
        .passengerName("Kamal Perera")
        .driverName("Sunil Silva")
        .vehiclePlate("CAB-1234")
        .vehicleType(VehicleType.CAR)
        .distanceKm(5.0)
        .durationMinutes(15.0)
        .baseFare(200.0)
        .distanceCharge(600.0)
        .timeCharge(150.0)
        .surgeMultiplier(1.0)
        .totalAmount(950.0)
        .currency("LKR")
        .paymentMethod(PaymentMethod.CARD)
        .issuedAt(LocalDateTime.now())
        .build();
}

@Test
void processPayment_ValidRequest_Returns201Created() throws Exception {
    ProcessPaymentRequest request = ProcessPaymentRequest.builder()
        .rideId("r1")
        .passengerId("p1")
        .paymentMethod(PaymentMethod.CARD)
        .actualDistanceKm(5.0)
        .actualDurationMinutes(15.0)
        .build();
}

```

```

when(paymentService.processPayment(any(ProcessPaymentRequest.class))).thenReturn(samplePaymentResponse);

mockMvc.perform(post("/api/payments/process")
    .contentType(MediaType.APPLICATION_JSON)
    .content(objectMapper.writeValueAsString(request)))
    .andExpect(status().isCreated())
    .andExpect(jsonPath("$.id").value("pay1"))
    .andExpect(jsonPath("$.status").value("COMPLETED"))
    .andExpect(jsonPath("$.receiptNumber").value("REC-20260926-654321"));
}

@Test
void processPayment_MissingFields_Returns400BadRequest() throws Exception {
    ProcessPaymentRequest request = ProcessPaymentRequest.builder()
        .rideId("")
        .passengerId("")
        .build();

    mockMvc.perform(post("/api/payments/process")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(request)))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.status").value(400));
}

@Test
void processPayment_Duplicate_Returns409Conflict() throws Exception {
    ProcessPaymentRequest request = ProcessPaymentRequest.builder()
        .rideId("r1")
        .passengerId("p1")
        .paymentMethod(PaymentMethod.CARD)
        .build();

    when(paymentService.processPayment(any(ProcessPaymentRequest.class)))
        .thenThrow(new DuplicatePaymentException("Payment already recorded"));

    mockMvc.perform(post("/api/payments/process")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(request)))
        .andExpect(status().isConflict())
        .andExpect(jsonPath("$.error").value("Duplicate Payment"));
}

@Test
void processPayment_PaymentFailed_Returns402PaymentRequired() throws Exception {
    ProcessPaymentRequest request = ProcessPaymentRequest.builder()
        .rideId("r1")
        .passengerId("p1")
        .paymentMethod(PaymentMethod.CARD)
        .simulateFailure(true)
        .build();

    when(paymentService.processPayment(any(ProcessPaymentRequest.class)))
        .thenThrow(new PaymentFailedException("Transaction declined"));

    mockMvc.perform(post("/api/payments/process")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(request)))
        .andExpect(status().isPaymentRequired())
        .andExpect(jsonPath("$.error").value("Payment Failed"));
}

@Test
void getPaymentById_Success() throws Exception {
    when(paymentService.getPaymentById("pay1")).thenReturn(samplePaymentResponse);

    mockMvc.perform(get("/api/payments/pay1"))

```

```

        .andExpect(status().isOk())
        .andExpect(jsonPath("$.id").value("pay1"))
        .andExpect(jsonPath("$.amount").value(950.0));
    }

    @Test
    void getPaymentById_NotFound_Returns404() throws Exception {
        when(paymentService.getPaymentById("nonexistent"))
            .thenReturn(new ResourceNotFoundException("Payment not found"));

        mockMvc.perform(get("/api/payments/nonexistent"))
            .andExpect(status().isNotFound())
            .andExpect(jsonPath("$.status").value(404));
    }

    @Test
    void getReceiptByPaymentId_Success() throws Exception {
        when(paymentService.getReceiptByPaymentId("pay1")).thenReturn(sampleReceiptResponse);

        mockMvc.perform(get("/api/payments/pay1/receipt"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.receiptNumber").value("REC-20260926-654321"))
            .andExpect(jsonPath("$.passengerName").value("Kamal Perera"))
            .andExpect(jsonPath("$.totalAmount").value(950.0));
    }
}

```

File: fare-service/src/test/java/com/ridelink/fare_service/FareServiceApplicationTests.java

```

package com.ridelink.fare_service;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.TestPropertySource;

@SpringBootTest
@TestPropertySource(properties = {
    "spring.data.mongodb.uri=mongodb://localhost:27017/ridelink_fare_test_db",
    "services.ride-service.url=http://localhost:8083"
})
class FareServiceApplicationTests {

    @Test
    void contextLoads() {
    }

}

```

File: fare-service/src/test/java/com/ridelink/fare_service/service/FareCalculationServiceTest.java

```

package com.ridelink.fare_service.service;

import com.ridelink.fare_service.dto.FareEstimateRequest;
import com.ridelink.fare_service.dto.FareEstimateResponse;
import com.ridelink.fare_service.entity.VehicleType;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

class FareCalculationServiceTest {

```

```

private FareCalculationService fareCalculationService;

@BeforeEach
void setUp() {
    fareCalculationService = new FareCalculationService();
}

@Test
void calculateEstimate_SingleVehicleType_Car() {
    FareEstimateRequest request = FareEstimateRequest.builder()
        .distanceKm(10.0)
        .estimatedDurationMinutes(20.0)
        .vehicleType(VehicleType.CAR)
        .build();

    List<FareEstimateResponse> estimates = fareCalculationService.calculateEstimate(request);

    assertEquals(1, estimates.size());
    FareEstimateResponse carEstimate = estimates.get(0);

    assertEquals(VehicleType.CAR, carEstimate.getVehicleType());
    assertEquals(200.0, carEstimate.getBaseFare());
    assertEquals(1200.0, carEstimate.getDistanceCharge()); // 10 km * 120
    assertEquals(200.0, carEstimate.getTimeCharge()); // 20 min * 10
    assertEquals(1600.0, carEstimate.getEstimatedFare()); // 200 + 1200 + 200 = 1600
    assertNotNull(carEstimate.getCalculationFormula());
}

@Test
void calculateEstimate_AllVehicleTypes_WhenVehicleTypeNull() {
    FareEstimateRequest request = FareEstimateRequest.builder()
        .distanceKm(5.0)
        .build();

    List<FareEstimateResponse> estimates = fareCalculationService.calculateEstimate(request);

    assertEquals(4, estimates.size()); // CAR, VAN, BIKE, TUK
}

@Test
void calculateEstimate_EnforcesMinimumFareFloor() {
    FareEstimateRequest request = FareEstimateRequest.builder()
        .distanceKm(0.1) // very short trip
        .estimatedDurationMinutes(1.0)
        .vehicleType(VehicleType.VAN)
        .build();

    List<FareEstimateResponse> estimates = fareCalculationService.calculateEstimate(request);

    // VAN min fare is 400.0
    assertTrue(estimates.get(0).getEstimatedFare() >= 400.0);
}

@Test
void calculateFinalFare_ComputesAccurateBreakdown() {
    FareCalculationService.FinalFareCalculation result =
        fareCalculationService.calculateFinalFare(VehicleType.TUK, 4.0, 10.0);

    // TUK: Base 100, PerKm 80 * 4 = 320, PerMin 8 * 10 = 80 -> Total = 500.0
    assertEquals(100.0, result.baseFare());
    assertEquals(320.0, result.distanceCharge());
    assertEquals(80.0, result.timeCharge());
    assertEquals(500.0, result.totalFare());
}
}

```

File: fare-service/src/test/java/com/ridelink/fare_service/service/PaymentServiceTest.java

```
package com.ridelink.fare_service.service;

import com.ridelink.fare_service.client.RideServiceClient;
import com.ridelink.fare_service.dto.PaymentResponse;
import com.ridelink.fare_service.dto.ProcessPaymentRequest;
import com.ridelink.fare_service.dto.ReceiptResponse;
import com.ridelink.fare_service.dto.RideValidationDto;
import com.ridelink.fare_service.entity.*;
import com.ridelink.fare_service.exception.DuplicatePaymentException;
import com.ridelink.fare_service.exception.PaymentFailedException;
import com.ridelink.fare_service.exception.ResourceNotFoundException;
import com.ridelink.fare_service.repository.PaymentRepository;
import com.ridelink.fare_service.repository.ReceiptRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.Spy;
import org.mockito.junit.jupiter.MockitoExtension;

import java.time.LocalDateTime;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class PaymentServiceTest {

    @Mock
    private PaymentRepository paymentRepository;

    @Mock
    private ReceiptRepository receiptRepository;

    @Mock
    private RideServiceClient rideServiceClient;

    @Spy
    private FareCalculationService fareCalculationService;

    @InjectMocks
    private PaymentService paymentService;

    private ProcessPaymentRequest sampleRequest;
    private RideValidationDto sampleRide;
    private Payment samplePayment;
    private Receipt sampleReceipt;

    @BeforeEach
    void setUp() {
        sampleRequest = ProcessPaymentRequest.builder()
            .rideId("r1")
            .passengerId("p1")
            .paymentMethod(PaymentMethod.CARD)
            .actualDistanceKm(5.0)
            .actualDurationMinutes(15.0)
            .simulateFailure(false)
            .build();

        sampleRide = RideValidationDto.builder()

```

```

        .id("r1")
        .passengerId("p1")
        .passengerName("Kamal Perera")
        .driverId("d1")
        .driverName("Sunil Silva")
        .vehiclePlate("CAB-1234")
        .vehicleType(VehicleType.CAR)
        .estimatedDistanceKm(5.0)
        .status("COMPLETED")
        .fareAmount(800.0)
        .build();

samplePayment = Payment.builder()
    .id("pay1")
    .transactionId("TXN-20260926-123456")
    .rideId("r1")
    .passengerId("p1")
    .amount(950.0)
    .paymentMethod(PaymentMethod.CARD)
    .status(PaymentStatus.COMPLETED)
    .paidAt(LocalDateTime.now())
    .createdAt(LocalDateTime.now())
    .build();

sampleReceipt = Receipt.builder()
    .id("rec1")
    .receiptNumber("REC-20260926-654321")
    .paymentId("pay1")
    .transactionId("TXN-20260926-123456")
    .rideId("r1")
    .passengerId("p1")
    .passengerName("Kamal Perera")
    .driverName("Sunil Silva")
    .vehiclePlate("CAB-1234")
    .vehicleType(VehicleType.CAR)
    .distanceKm(5.0)
    .durationMinutes(15.0)
    .baseFare(200.0)
    .distanceCharge(600.0)
    .timeCharge(150.0)
    .surgeMultiplier(1.0)
    .totalAmount(950.0)
    .paymentMethod(PaymentMethod.CARD)
    .issuedAt(LocalDateTime.now())
    .build();
}

@Test
void processPayment_Success() {
    when(rideServiceClient.getAndValidateRide("r1", "p1")).thenReturn(sampleRide);
    when(paymentRepository.existsByRideIdAndStatus("r1", PaymentStatus.COMPLETED)).thenReturn(false);
    when(paymentRepository.save(any(Payment.class))).thenReturn(samplePayment);
    when(receiptRepository.save(any(Receipt.class))).thenReturn(sampleReceipt);

    PaymentResponse response = paymentService.processPayment(sampleRequest);

    assertNotNull(response);
    assertEquals("pay1", response.getId());
    assertEquals(PaymentStatus.COMPLETED, response.getStatus());
    assertEquals(PaymentMethod.CARD, response.getPaymentMethod());
    assertNotNull(response.getReceiptNumber());

    verify(rideServiceClient, times(1)).getAndValidateRide("r1", "p1");
    verify(paymentRepository, times(1)).save(any(Payment.class));
    verify(receiptRepository, times(1)).save(any(Receipt.class));
}

```

```

@Test
void processPayment_DuplicatePayment_ThrowsDuplicatePaymentException() {
    when(rideServiceClient.getAndValidateRide("r1", "p1")).thenReturn(sampleRide);
    when(paymentRepository.existsByRideIdAndStatus("r1", PaymentStatus.COMPLETED)).thenReturn(true);

    assertThrows(DuplicatePaymentException.class, () -> paymentService.processPayment(sampleRequest));

    verify(paymentRepository, never()).save(any(Payment.class));
    verify(receiptRepository, never()).save(any(Receipt.class));
}

@Test
void processPayment_SimulateFailure_ThrowsPaymentFailedException() {
    sampleRequest.setSimulateFailure(true);
    when(rideServiceClient.getAndValidateRide("r1", "p1")).thenReturn(sampleRide);
    when(paymentRepository.existsByRideIdAndStatus("r1", PaymentStatus.COMPLETED)).thenReturn(false);

    assertThrows(PaymentFailedException.class, () -> paymentService.processPayment(sampleRequest));

    verify(paymentRepository, times(1)).save(argThat(p -> p.getStatus() == PaymentStatus.FAILED));
    verify(receiptRepository, never()).save(any(Receipt.class));
}

@Test
void getPaymentById_Success() {
    when(paymentRepository.findById("pay1")).thenReturn(Optional.of(samplePayment));
    when(receiptRepository.findByPaymentId("pay1")).thenReturn(Optional.of(sampleReceipt));

    PaymentResponse response = paymentService.getPaymentById("pay1");

    assertNotNull(response);
    assertEquals("pay1", response.getId());
    assertEquals("REC-20260926-654321", response.getReceiptNumber());
}

@Test
void getPaymentById_NotFound_ThrowsException() {
    when(paymentRepository.findById("nonexistent")).thenReturn(Optional.empty());

    assertThrows(ResourceNotFoundException.class, () -> paymentService.getPaymentById("nonexistent"));
}

@Test
void getReceiptByPaymentId_Success() {
    when(receiptRepository.findByPaymentId("pay1")).thenReturn(Optional.of(sampleReceipt));

    ReceiptResponse response = paymentService.getReceiptByPaymentId("pay1");

    assertNotNull(response);
    assertEquals("REC-20260926-654321", response.getReceiptNumber());
    assertEquals("Kamal Perera", response.getPassengerName());
    assertEquals(950.0, response.getTotalAmount());
}
}

```